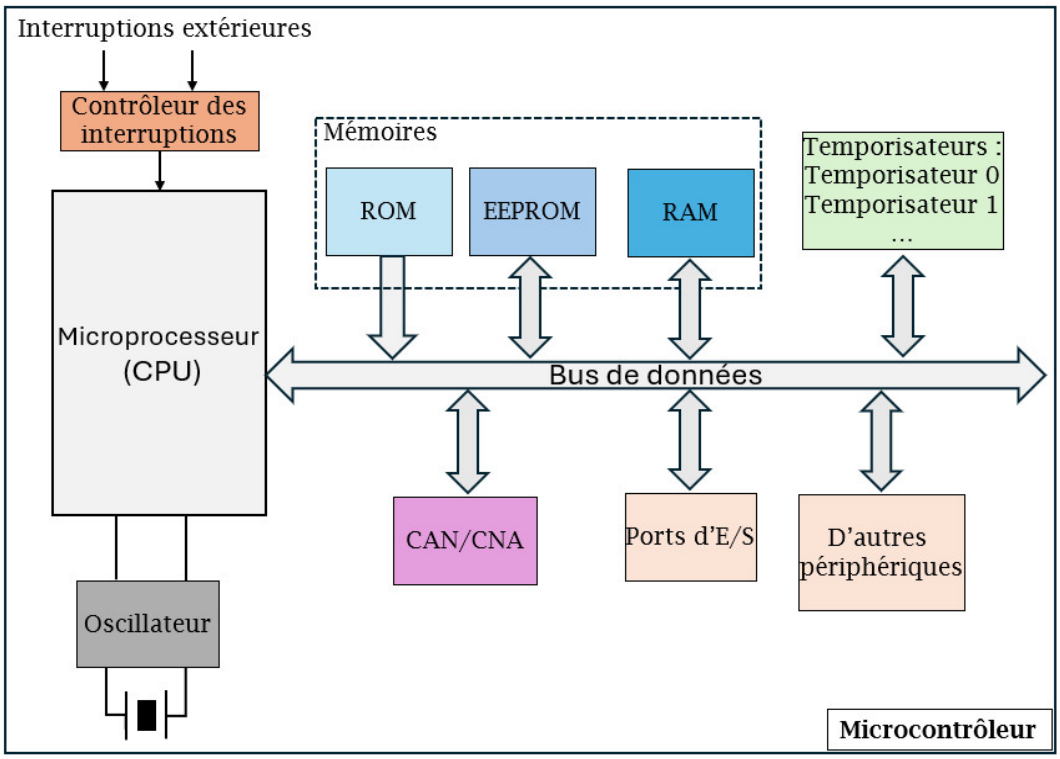
كلية العلوم - أكاديـر
ⵜⴰⵖⵡⴰⵍⵜ ⵏ ⵜⵣⴰⵡⵉⵏ - ⴰⴳⴰⴷⵉⵔ
FACULTÉ DES SCIENCES - AGADIR

Département de Physique

Licence PC-P
Filière Électronique & Systèmes

Semestre 6

Cours de Systèmes à microprocesseur & à microcontrôleur



Responsable : Prof. Said AGOUNAD

2025-2026

Note : Ce cours est destiné aux étudiants de parcours électronique et systèmes, systèmes embarqués, génie électrique, génie industriel, automatique, automatisme, etc. Son objectif principal est d'aider les étudiants à comprendre l'architecture des systèmes à microprocesseur et à microcontrôleur, et apprendre sur les modes et le langage de communication entre le processeur et ses périphériques. Il est essentiel dans de nombreux secteurs professionnels, académiques et de recherche, élargissant ainsi vos opportunités.

Ce polycopié ne peut en aucun cas remplacer le cours magistral, il l'accompagne et le complète. Plus explications et d'informations complémentaires sont données pendant les séances de cours magistral.

Note : Science is a great weapon. From the early days until now and in the future only science can improve the life of humanity and bring enlightenment to them. Currently, there are many challenges in the world, so only young talented scientists can find effective solutions to them.

Table des matières

Partie I – Systèmes à microprocesseur **1**

Chapitre I – Généralités sur les systèmes à microprocesseur **2**
I Architecture d'un système à microprocesseur 2
I.1 Signal d'horloge 3
I.2 Bus 3
I.2.1 Bus de données 3
I.2.2 Bus de contrôles 3
I.2.3 Bus d'adresses 4
I.3 Mémoires 4
I.3.1 Définitions 4
I.3.2 Schéma fonctionnel d'une mémoire 5
I.3.3 Capacité d'une mémoire 6
I.3.4 Interfaçage microprocesseur/mémoire 6
I.3.5 Décodage d'adresses 7
I.3.6 Chronogrammes de lecture / écriture en mémoire 9
I.4 Microprocesseur 10
I.4.1 Définition 10
I.4.2 Historique des microprocesseurs 10
I.4.3 Architecture de base d'un microprocesseur 12
I.4.3.1 Unité de traitement : 13
I.4.3.2 Unité de commande : 13
I.4.3.3 Registres d'adresses et données : 14
I.4.4 Fonctionnement d'un microprocesseur 14
I.4.4.1 Récupération de l'instruction : 15
I.4.4.2 Décodage de l'instruction et recherche de l'opérande : . 15
I.4.4.3 Exécution de l'instruction 16
I.4.5 Forme d'instructions : Architectures CISC et RISC 16
I.4.5.1 Architecture CISC 17
I.4.5.2 Architecture RISC 18
I.4.5.3 Comparaison entre CISC et RISC 18
I.4.6 Procédure de transfert des données : Architectures de Von Neumann et Harvard 19
I.4.6.1 Architecture de Von Neumann 19
I.4.6.2 Architecture de Harvard 19

I.4.7 Procédure d'exécution des instructions : Architectures classique et pipeline . . . 21
I.4.7.1 Architecture pipeline . . . 21
I.4.7.2 Gain de performance . . . 22
I.4.7.3 Problèmes et solutions des pipelines . . . 22
II Exemples de systèmes à microprocesseur . . . 23
II.1 Ordinateurs . . . 23
II.1.1 Définition . . . 23
II.1.2 Évolution des ordinateurs avec l'évolution des microprocesseurs 24
II.1.3 Carte mère et le microprocesseur . . . 26
II.1.4 Périphériques . . . 27
II.2 Microcontrôleur . . . 27
III Conclusion . . . 27

Chapitre II – Architecture et fonctionnement du microprocesseur Intel 8086 28
I Architecture externe du µp 8086 . . . 28
I.1 Brochage . . . 28
I.2 Fonctions des broches . . . 28
I.2.1 Signal d'horloge : CLK . . . 30
I.2.2 Bus d'adresses . . . 30
I.2.3 Bus de données . . . 31
I.2.4 Bus de contrôle . . . 31
I.2.4.1 Bus de contrôle du microprocesseur . . . 31
I.2.4.2 Bus de contrôle des périphériques . . . 32
I.2.5 Interruptions 8086 . . . 35
I.2.6 Signaux de demande d'accès à la mémoire . . . 36
I.2.7 Signaux d'états . . . 36
I.2.8 Broches d'alimentation et de la masse . . . 36
II Architecture interne du µp 8086 . . . 36
II.1 Unité d'interface de bus . . . 37
II.1.1 Registres de segments . . . 38
II.1.2 Pointeur d'instruction . . . 38
II.1.3 Bloc logique de génération d'adresses et de contrôle des bus . . . 38
II.1.4 File d'attente d'instructions . . . 39
II.2 Unité d'exécution . . . 39
II.2.1 Unité arithmétique et logique . . . 39
II.2.2 Registres généraux . . . 39
II.2.3 Registres de pointeurs et d'index . . . 40
II.2.4 Registre d'état . . . 40
III Organisation de la mémoire par le 8086 . . . 42

Chapitre III – Programmation en assembleur du µp 8086 . . . 44
I Jeu d'instruction de µp 8086 . . . 44
I.1 Instructions de transfert de données et modes d'adressage . . . 45
I.1.1 Instructions de transfert de données . . . 45
I.1.2 Modes d'adressage . . . 45
I.2 Instructions arithmétiques . . . 47

I.3 Instructions logiques ... 49
I.4 Instructions de décalage et rotation ... 50
I.4.1 Instructions de décalage ... 50
I.4.2 Instructions de rotations ... 52
I.5 Instructions de branchement ... 53
I.5.1 Instruction de branchement inconditionnel ... 53
I.5.2 Instructions de branchements conditionnels ... 53
I.5.3 Appel de sous-programmes ... 55
II Directives de langage assembleur ... 57

Chapitre IV – Interfaces d'entrées/sorties et interruptions de µp 8086 ... 60
I Généralités sur les interfaces d'entrées/sorties ... 60
I.1 Définitions ... 60
I.2 Adressage du port d'une interface d'E/S ... 61
II Interfaces E/S de µp 8086 ... 62
II.1 Gestion des interfaces d'E/S par le µp 8086 ... 62
II.2 Interface parallèle 8255 ... 64
II.2.1 Brochage du 8255 ... 64
II.2.2 Accès aux ports du 8255 ... 64
II.2.3 Modes de fonctionnement de l'interface 8255 ... 65
II.2.4 Connections du 8255 avec le microprocesseur 8086 ... 66
II.2.5 Exemple d'application en mode 0 ... 68
II.3 Interface série 8250 ... 69
II.3.1 Généralités sur les interfaces séries ... 69
II.3.2 Brochage du 8250 ... 70
II.3.3 Registres du 8250 ... 71
II.3.3.1 Registres de contrôle de protocole ... 72
II.3.3.2 Registres de contrôle de la ligne ... 73
II.3.3.3 Registres de contrôle d'interruptions ... 73
II.3.3.4 Registres de transfert ... 74
II.3.3.5 Registre à usage général ... 76
II.3.4 Exemples de programmation du 8250 ... 76
II.3.4.1 Programmation directe ... 76
II.3.4.2 Programmation en utilisant les interruptions BIOS/DOS ... 76
III Interruptions du 8086 ... 78
III.1 Définition ... 78
III.2 Traitement de demandes d'interruptions par le µp ... 78
III.3 Adresses des sous-programmes d'interruptions ... 79
III.4 Interruptions du 8086 ... 79
III.5 Contrôleur d'interruptions 8259 ... 80

Partie II – Systèmes à microcontrôleur ... 82

Chapitre V – Microcontrôleurs : Arduino uno ... 83
I Généralités sur les microcontrôleurs ... 83

I.1 Définition . . . 83
I.2 Domaines d'applications . . . 83
I.3 Description matérielle des microcontrôleurs . . . 84
I.3.1 Architecture interne des microcontrôleurs . . . 84
I.3.2 Boîtiers de microcontrôleurs . . . 86
I.4 Familles de microcontrôleurs . . . 86
II Arduino Uno . . . 88
II.1 Définition . . . 88
II.2 Présentation de la carte Arduino Uno . . . 88
II.2.1 Microcontroleur ATmega 328 . . . 88
II.2.1.1 Caractéristiques de ATmega 328 : . . . 88
II.2.1.2 Brochage de ATmega 328 . . . 90
II.2.2 Schéma simplifié de la carte Arduino UNO . . . 92
II.3 Programmation de la carte Arduino Uno . . . 93
II.3.1 Environnement de développement (IDE) . . . 93
II.3.2 Programmation et configuration de la carte Arduino . . . 93
II.3.3 Langage Arduino . . . 94

Partie I

Systèmes à microprocesseur

Chapitre I

Généralités sur les systèmes à microprocesseur

Le développement de l'électronique conduit à l'apparition du microprocesseur en 1971. Dans cette année, la société Intel a réussi, pour la première fois, à placer tous les composants qui constituent un processeur sur un seul circuit intégré, donnant ainsi naissance au microprocesseur. Cette miniaturisation a permis de réduire des distances entre les composants ce qui augmente les vitesses de fonctionnement des microprocesseurs ; de diminuer les coûts, grâce au remplacement de plusieurs composants par un seul ; d'augmenter la fiabilité, en éliminant les connexions entre les composants d'un processeur, on élimine donc l'un des principaux origines des pannes ; de réduire la consommation énergétique ; de créer des ordinateurs bien plus petits.

Le rôle de ce chapitre est de fournir quelques généralités sur les systèmes à microprocesseurs, en présentant les éléments principaux d'un système à microprocesseur, l'histoire des microprocesseurs, leur fonctionnement, etc.

I Architecture d'un système à microprocesseur

Un système à microprocesseur est un système électronique programmable équipé d'éléments nécessaires pour réaliser une fonction donnée. Comme le montre la figure 1, les éléments principaux de ce système sont : le microprocesseur, l'horloge, les mémoires, les interfaces entrées/sorties et les bus.

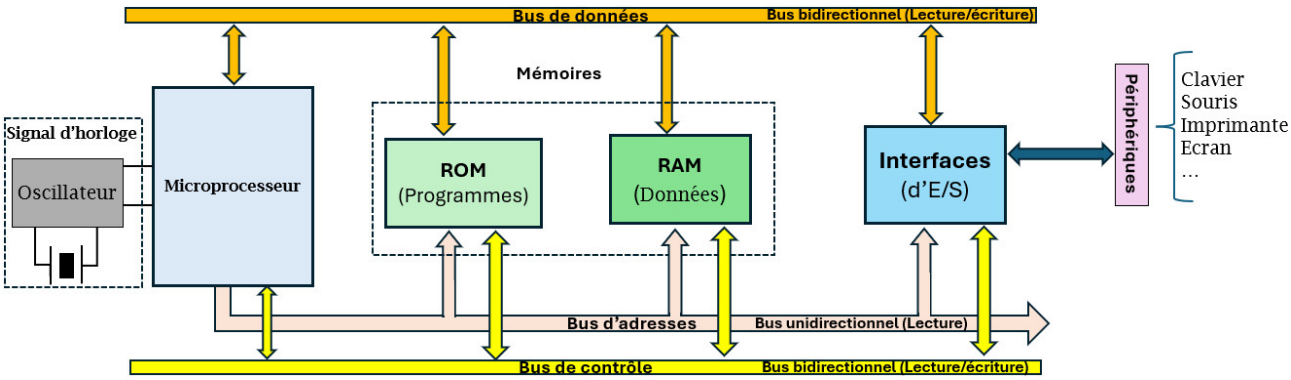


Figure 1 – Système à microprocesseur.

I.1 Signal d'horloge

Signal d'horloge ou tout simplement l'horloge est un signal périodique qui synchronise toutes les opérations internes du microprocesseur (récupération d'instructions, traitement des données). Leur fréquence (vitesse du microprocesseur), mesurée en Hz ou plus souvent en GHz, indique le nombre de cycles (impulsions) que le microprocesseur exécute par seconde. Par exemple, pour une horloge de fréquence 3 GHz, le microprocesseur effectue 3 milliards de cycles par seconde.

La durée d'exécution d'une instruction dépend de la période de l'horloge et le nombre de cycles nécessaires à leur exécution.

Exemple : La durée d'exécution de l'instruction qui s'exécute en trois cycles d'horloge de fréquence 5 MHz (période T = 1/f = 0, 2 s) est : $3 \times 0{,}2 = 0{,}6$ s.

Rappel : Le signal d'horloge est souvent généré par un oscillateur électrique à quartz dont les circuits peuvent être internes ou externes au microprocesseur. Le quartz est un matériau cristallin de roche naturelle de silice (dioxyde de silicium : SiO2). Il a la particularité de vibrer (résonner) à une fréquence précise lorsqu'il est soumis à une excitation électrique.

I.2 Bus

Un bus est constitué de fils électriques qui servent à transporter des informations entre deux éléments ou plus. Dans le cas d'un système à microprocesseur, la communication et l'échange des informations entre les différents éléments de ce système sont assurés par le biais de trois bus : *bus de données, bus d'adresses et bus de contrôles* (voir la figure 1).

I.2.1 Bus de données

Définition :

Le bus de données est en quelque sorte l'autoroute qui assure le transport des données du microprocesseur vers ses périphériques (comme les mémoires), et vice-versa. Plutôt que d'utiliser deux bus, une pour la transmission de données et l'autre pour la réception, ce qui pour un microprocesseur à 8 bits représente 16 connexions d'entrée-sortie, les constructeurs ont préféré d'utiliser un seul bus dont le sens de transmission peut être alternativement inversé grâce à une connexion supplémentaire sur laquelle est appliquée un niveau logique dont la valeur décide du sens de circulation. C'est donc un bus à deux sens, on dit qu'il est *bidirectionnel*

Plusieurs organes sont reliés à ce bus, mais un organe ne peut y expédier des données vers un autre organe qu'une fois reçu le signal correspondant émis par la logique de décodage et de séquencement en fonction de l'instruction en cours.

Exemple :

Un microprocesseur 8 bits (Ex. Motorola 6809) possède un bus de données qui contient 8 fils électriques, notés D0 à D7. Il peut transmettre 8 bits (1 octet) à la fois.

Les octets circulant sur le bus de données constituent les codes :

— des *instructions* en provenance de la partie mémoire programme ;

— ou des *données* en provenance ou à destination d'une mémoire ou d'un circuit périphérique.

I.2.2 Bus de contrôles

Comme le bus de données ce bus est aussi bidirectionnel sert à :

— synchroniser le système : Horloge ;

— réaliser l'initialisation de l'unité centrale : Reset ;

— contrôler les échanges (lecture/écriture) : R/W (read/write) ;

— provoquer des exécutions de programmes particuliers (interruptions) : IRQ. On rappelle que IRQ (en anglais Interrupt ReQuest) est une interruption matérielle (signal matériel) déclenchée par un périphérique d'entrée-sortie et envoyée au processeur qui arrête temporairement un programme en cours d'exécution et permet à un programme spécial, un gestionnaire d'interruption, de s'exécuter à la place.

I.2.3 Bus d'adresses

Le bus d'adresses est un bus unidirectionnel qui transporte les adresses des données du microprocesseur vers les mémoires et les périphériques. Une adresse désigne l'emplacement où se trouve une donnée.

Le nombre de bits d'adresses permet de déterminer le nombre de cases mémoires que le microprocesseur peut adresser : c-à-d **l'espace adressable**.
Exemple : Pour un bus d'adresse de 16 bits, notés A0 à A15, le microprocesseur peut accéder à 2^{16} octets (=65 536 octets =64 Kilo octets ; noté que 1 kilo-octet=1024 octets) si le bus de données est de 8 bits.

I.3 Mémoires

I.3.1 Définitions

Une mémoire est un dispositif électronique qui sert à stocker des données. On distingue généralement entre deux types de mémoires : Mémoires vives (RAM) et les Mémoires mortes (ROM) :

▷ RAM (Random Access Memory)

C'est une mémoire volatile, c'est-à-dire qu'elle perd son contenu en cas de coupure d'alimentation. Elle est utilisée pour stocker temporairement des données et des programmes, et peut être lue ou écrite directement par le microprocesseur. Il existe deux types de mémoires RAM :

— *Static RAM (SRAM)* qui stocke les données à l'aide de bascules à semiconducteurs (bascules RS). Elle conserve les données, une fois écrites, sans autre manipulation tant que la source d'énergie est maintenue. Elle est très rapide, mais il est compliqué à réaliser. La SRAM est généralement utilisée pour réaliser les registres du microprocesseur et les mémoires caches.

— *Dynamic RAM (DRAM)* qui stocke les données sous forme de charges électriques dans des condensateurs, qui doivent être régulièrement rafraîchis pour éviter leur perte. Elle est moins coûteuse et c'est elle qui est utilisée pour la réalisation de la mémoire vive des ordinateurs.

▷ ROM (Read Only Memory)

Est une mémoire non-volatile (permanente), c'est-à-dire qu'elle conserve son contenu même en l'absence d'alimentation électrique. Comme son nom l'indique, elle est en principe accessible uniquement en lecture par le microprocesseur. Elle contient de façon permanente des données essentielles ou des programmes de démarrage (comme le BIOS dans les ordinateurs). En plus des ROM classiques qui sont livré avec des données permanentes, il existe d'autres variantes :

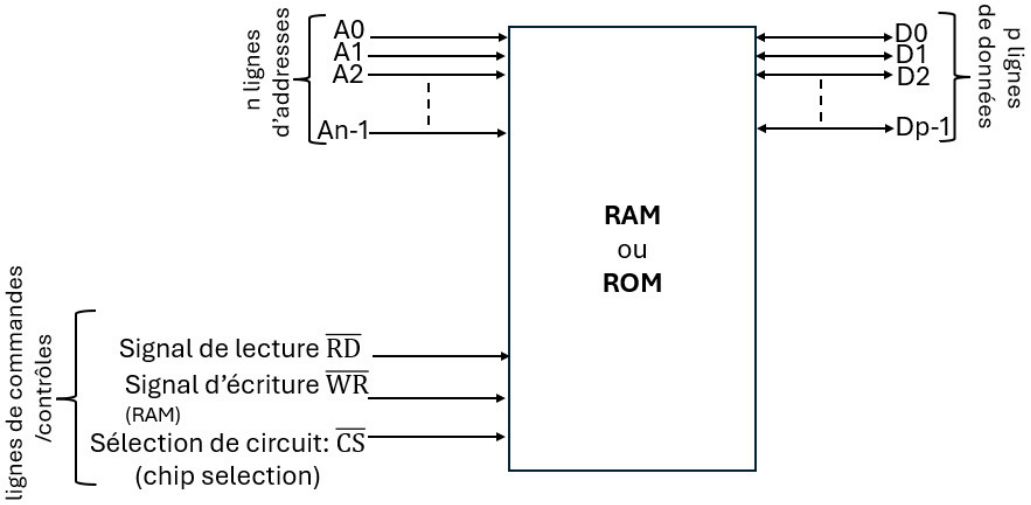


FIGURE 2 – Architecture externe d'une mémoire.

— *Programmable ROM (PROM)* : Mémoire morte programmable une seule fois par l'utilisateur après sa fabrication. Une fois les données inscrites, elles ne peuvent plus être modifiées ni effacées. Nécessite un programmateur spécialisé (par exemple : application d'une tension de programmation 21 ou 25 V pendant 20 ms).

— *Erasable PROM (EPROM)* : Mémoire morte programmable et effaçable. Programmable électriquement avec un programmateur et effaçable par exposition à un rayonnement ultraviolet.

— *Electrically Erasable PROM (EEPROM)* : Mémoire morte programmable et effaçable électriquement, cellule par cellule. Elle peut être réécrite plusieurs fois sans extraction de la carte et sans programmateur, ce qui la rend très pratique pour stocker des paramètres de configuration des ordinateurs.

I.3.2 Schéma fonctionnel d'une mémoire

La communication entre les mémoires et le microprocesseur se fait moyennant de différentes broches (voir figure 2) :

— n broches d'adresses (A0 à An-1) permettant de sélectionner une case mémoire précise.

— p broches de données (D0 à Dp-1) permettant à chaque case mémoire de fournir ou recevoir une information codée sur p bits.

— 3 broches de commande/contrôle :

  — $\overline{RD}$ (Read) : active la lecture et place sur les lignes de sortie la donnée contenue à l'adresse sélectionnée.

  — $\overline{WR}$ (Write) : autorise l'écriture d'une donnée à l'adresse choisie (uniquement dans les mémoires de type RAM).

  — $\overline{CS}$ (Chip Select) : permet de sélectionner le circuit mémoire concerné, en particulier lorsqu'il existe plusieurs circuits dans le système.

La barre qui se trouve sur des broches comme $\overline{RD}$ signifie que cette broche est activée à l'état bas.

I.3.3 Capacité d'une mémoire

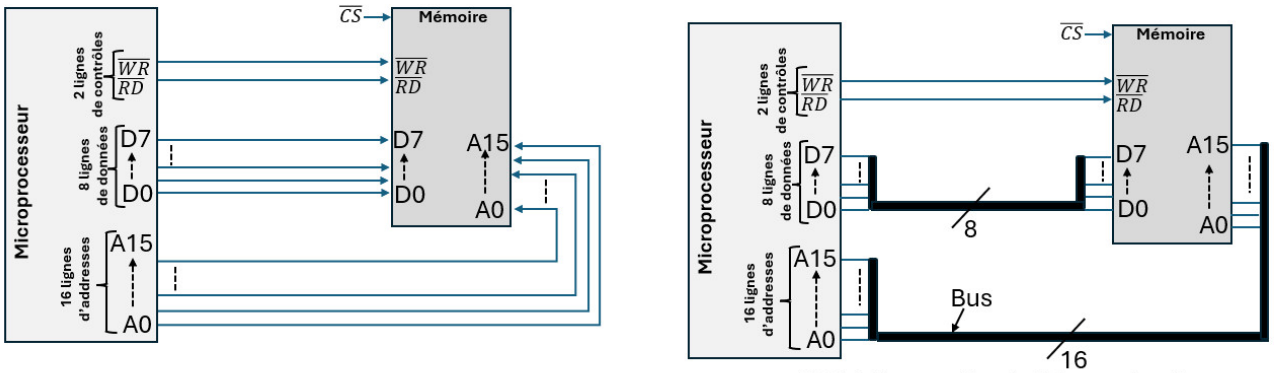
La capacité d'une mémoire (taille) est le nombre d'informations (quantité) que la mémoire peut mémoriser. Elle peut être exprimée en :
- Bit (0 ou 1) : est un élément de base pour la représentation des informations.
- Octet : est une suite de 8 bits codant une information. Il y a des multiples d'un octet tels que :
  + Kilo-octet (Ko) : 1 Ko =1024 octets $=2^{10}$ octets
  + Méga-octet (Mo) : 1 Mo =1024 Ko $=2^{20}$ octets
  + Géga-octet (Go) : 1 Go =1024 Mo $=2^{30}$ octets
  + Téra-octet (To) : 1 To =1024 Go $=2^{40}$ octets

La capacité de la mémoire dépend du nombre de lignes d'adresses et du nombre de bits p qu'une case mémoire peut stocker, c.à.d., la taille d'une case mémoire qui peut être 8, 16, 32, 64 bits ou plus, suivant l'architecture du microprocesseur. En effet, n lignes d'adresses permettent d'adresser 2^n cases mémoire, alors la capacité d'une mémoire est $2^n \times p$ bits, si $p = 8$, alors la taille de la mémoire est 2^n octets.
Exemple : 8 bits d'adresses permettent d'adresser $2^8 = 256$ cases mémoire, c.à.d., la taille de mémoire est 256 octets (si $p = 8$). 16 bits d'adresses permettent d'adresser $2^{16} = 65536$ cases mémoire, c.à.d., la taille de mémoire est 64 Ko (si $p = 8$).

I.3.4 Interfaçage microprocesseur/mémoire

Un microprocesseur peut être relié à un ou plusieurs mémoires en même temps qui sont reliées en permanence sur le bus de données. Dans le dernier cas, la sélection d'une seule mémoire pour la lecture ou l'écriture se fait à l'aide de décodeur d'adresse qui commande les broches $\overline{CS}$ des différentes mémoires. Lorsque une broche est active (à l'état bas), le boîtier de mémoire correspondante est sélectionné pour la lecture ou l'écriture. Lorsque $\overline{CS}$ est inactive (à l'état haut), la mémoire est mise hors service : ses broches de données (D0 à Dp-1) passent en haute impédance, donc la mémoire est déconnectée du bus de données du microprocesseur.

Ce mécanisme rend possible la connexion de plusieurs boîtiers mémoire sur un même bus d'adresses et de données. Pour éviter les conflits, un seul boîtier mémoire doit être activé à la fois. Donc un seul signal $\overline{CS}$ doit être actif à un instant donné.
Exemple 1 : La figure 3 montre le schéma détaillé de l'interfaçage microprocesseur/1 boîtiers mémoire d'une capacité de 64 ko sur un bus d'adresse de 16 bits.



(a) Schéma détaillé de l'interfaçage microprocesseur–mémoire

(b) Schéma condensée (plus pratique)

FIGURE 3 – Interfaçage microprocesseur/1 boîtier mémoire.

Exemple 2 : La figure 4 montre le schéma détaillé de l'interfaçage microprocesseur/2 boîtiers

mémoire d'une capacité de 32 Ko chacune (15 lignes d'adresses) sur un bus d'adresse de 16 bits.

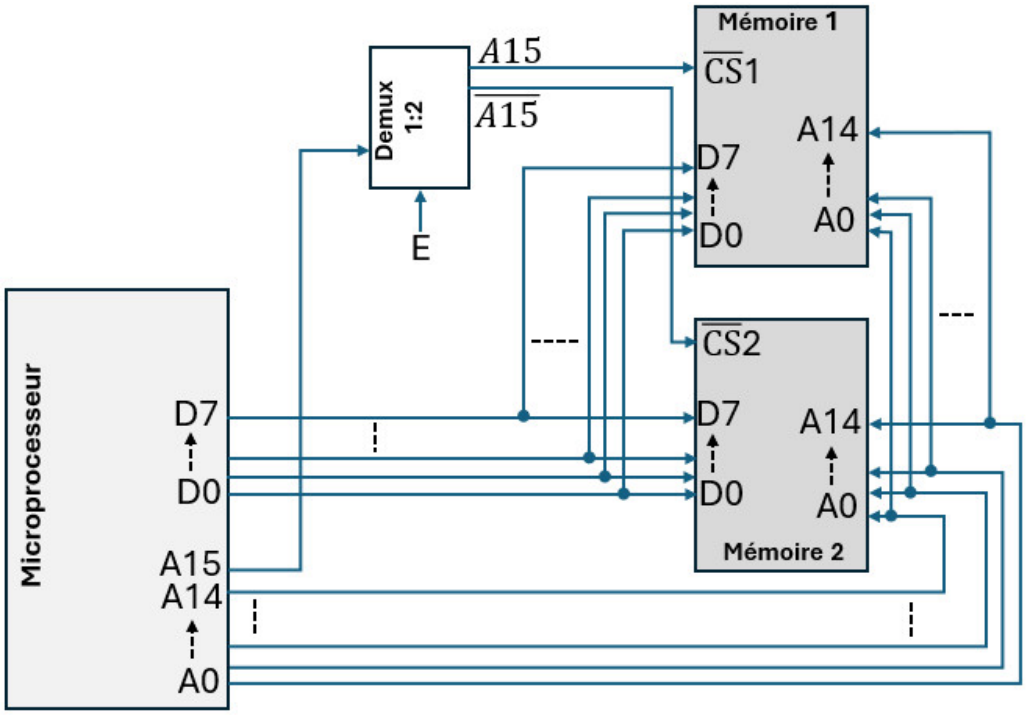


FIGURE 4 – Interfaçage microprocesseur/2 boîtiers mémoire.

Exemple 3 : Le schéma détaillé de l'interfaçage microprocesseur/4 boîtiers mémoire d'une capacité de 256 octets chacune (8 lignes d'adresses) sur un bus d'adresse de 10 bits est représenté sur la figure 5.

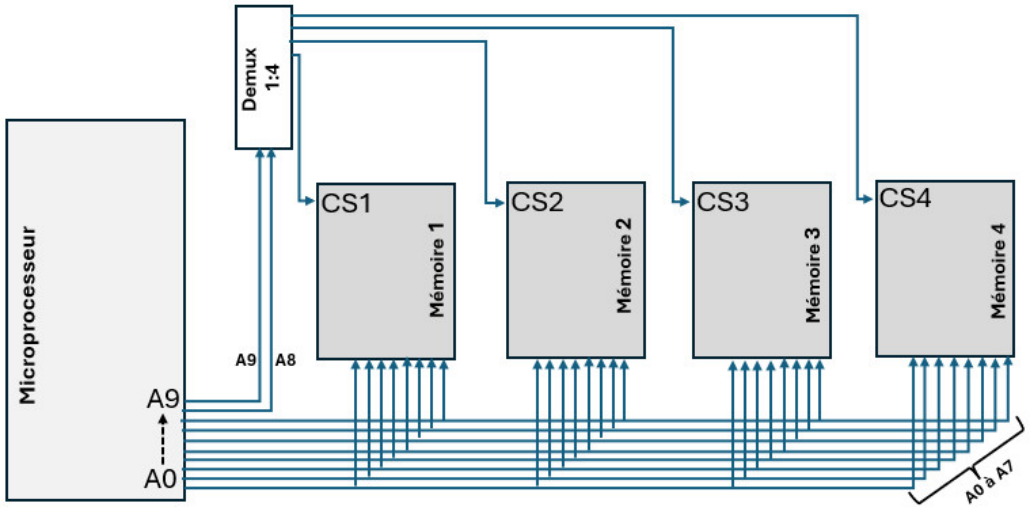


FIGURE 5 – Interfaçage microprocesseur/4 boîtiers mémoire.

I.3.5 Décodage d'adresses

Le décodage d'adresses est une opération qui permet d'allouer à chaque mémoire (ou périphérique) une zone de l'espace adressable. La capacité d'adressage du microprocesseur est

souvent plus grande que la capacité des mémoires implantées. Donc, il est possible d'ajouter d'autres boîtiers mémoire afin d'exploiter tout l'espace adressable.

Exemple 1 : Connexion de 3 boîtiers mémoire de 8 Ko sur un bus d'adresse de 16 bits. La figure 6 montre le schéma de l'interfaçage entre le microprocesseur et ces 3 boîtiers. Les lignes d'adresses A13, A14 et A15 sont utilisées pour sélectionner un seul boîtier mémoire à la fois à l'aide de circuit de décodage d'adresses (circuit de sélection) qui active le signal CS (Chip Select) du boîtier correspondant à la plage d'adresses concernée. Ils peuvent prendre des valeurs binaires de 000 à 111, soit 8 combinaisons possibles, ce qui permet de gérer jusqu'à 8 boîtiers mémoires distincts. Toutefois, il est impératif qu'un seul boîtier mémoire soit sélectionné à la fois pour éviter tout conflit sur le bus de données.

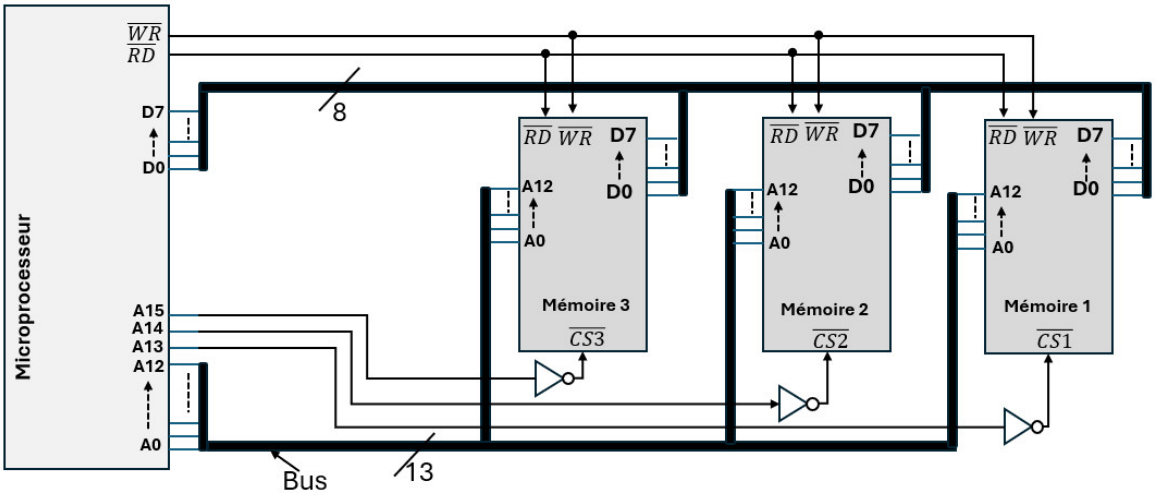


FIGURE 6 – Schéma de l'interfaçage entre le microprocesseur/3 boîtiers mémoire.

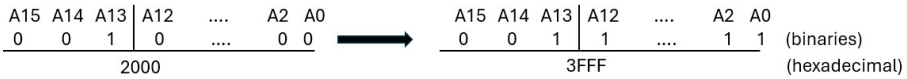
Dans ce cas, le circuit de décodage utilisé est un circuit avec trois inverseurs, chacun monté sur une ligne d'adresse, qui permet de sélectionner l'un des 3 boîtiers mémoire à partir des lignes d'adresses A13, A14 et A15.

Chaque boîtier (mémoire 1, mémoire 2, mémoire 3) possède 13 lignes d'adresses (A0–A12). Donc, chaque mémoire a une taille de 2^{13} octets = 8192 octets = 8 Ko.

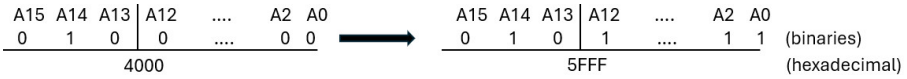
La question qui se pose maintenant est la suivante : *Quelle est la plage d'adresse occupée par chacune de ces mémoires ?*

Réponse :

- Pour atteindre la mémoire N°1, il faut mettre à 1 le bit A13 et à 0 les bits A14 et A15. La plage d'adresses occupée par cette mémoire est donc



- De même, pour la mémoire N°2, on doit avoir A13 = 0, A14 = 1 et A15 = 0 d'où la plage d'adresses occupée cette mémoire :



- Pour la mémoire N°3, on doit avoir A13 = 0, A14 = 0 et A15 = 1 d'où la plage d'adresses occupée cette mémoire :

On en déduit la cartographie ou mapping de la mémoire visible par le microprocesseur :

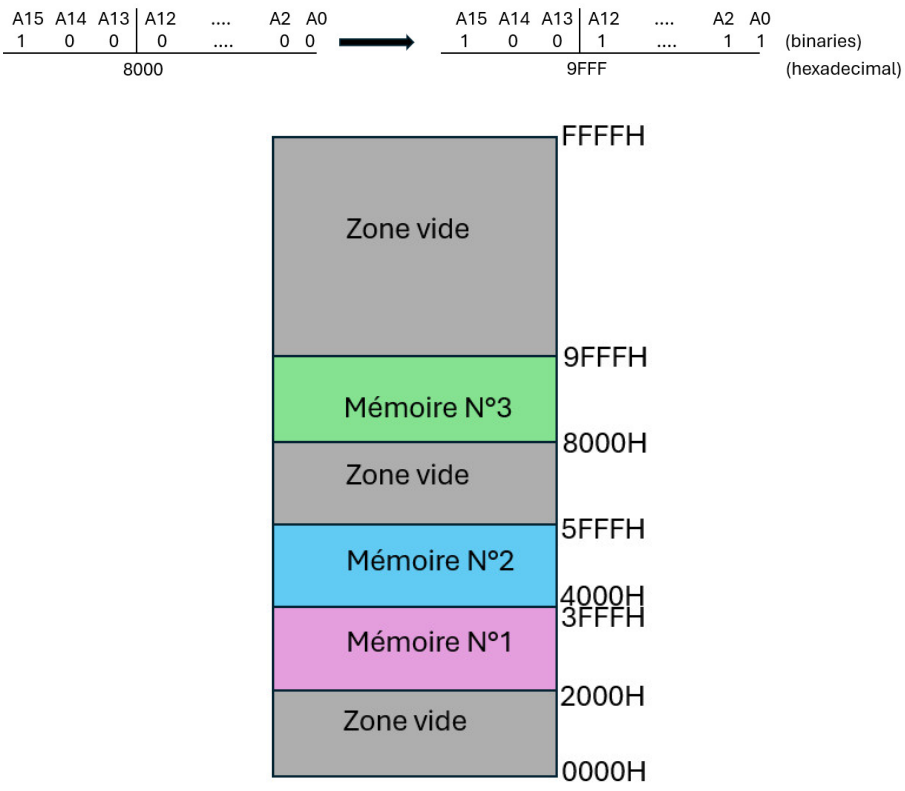


FIGURE 7 – Cartographie ou mapping de la mémoire visible par le microprocesseur

I.3.6 Chronogrammes de lecture / écriture en mémoire

La lecture ou l'écriture sur une mémoire se fait suivant un ensemble d'étapes bien définies qu'on peut illustrer à l'aide d'un chronogramme.

▷ Chronogramme de lecture en mémoire

Les étapes de lecture d'une case mémoire sont :

— Présentation de l'adresse de la case mémoire (donnée) que le microprocesseur veut lire ;

— Mettre l'entrée de lecture $\overline{RD}$ à l'état actif "dans ce cas c'est 0" ;

— Sélection du boîtier de la mémoire, en appliquant sur les entrées de sélection, $\overline{CS}$, un niveau actif, "dans ce cas c'est 0".

— Mettre la valeur à lire, pendant une durée donnée, sur les bus de données.

Le chronogramme de lecture en mémoire est présenté sur la figure 8.

Explication du chronogramme :

- Les zones hachurées indiquent que l'état des entrées concernées n'est pas stable.
- Sur un tracé donné : Le niveau intermédiaire entre l'état 1 et de l'état 0 indique que le bus de données est à l'état «haute impédance», c.à.d., le circuit de sortie est déconnecté.
- Une fois, les entrées de sélection sont activées, le bus de données bascule de l'état «haute impédance» à l'état actif, mais ne donne pas cependant la donnée correspondante à l'adresse désirée. En effet, le temps de propagation de l'état des entrées de lecture/écriture et d'adresse est plus grande que celui des entrées de sélection.

Remarque : La durée qui s'écoule entre le moment où une adresse est appliquée sur le bus d'adresses et le moment où la donnée correspondante devient disponible et stable sur le bus

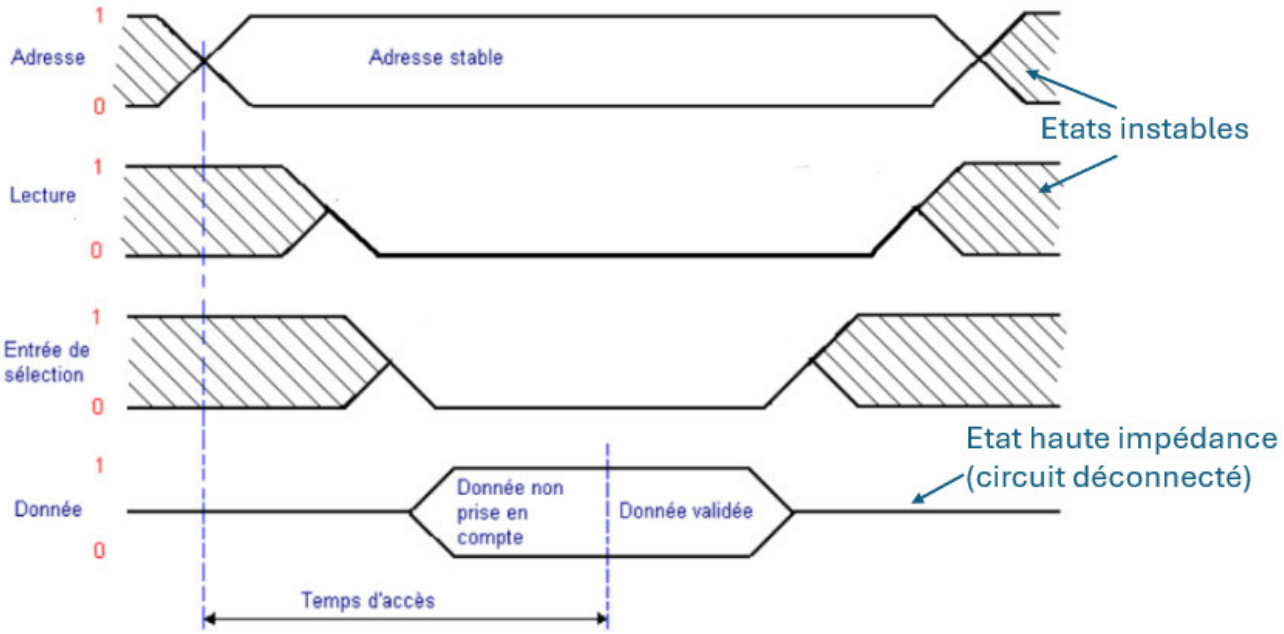


FIGURE 8 – Chronogramme de lecture en mémoire, $\overline{RD}$ actif à l'état bas.

de données est définie comme le *temps d'accès au mémoire*, est une caractéristique essentielle d'une mémoire. En d'autres termes, c'est le délai nécessaire pour que la mémoire « réponde » à une requête de lecture.

▷ Chronogramme de l'écriture en mémoire

L'écriture en mémoire se fait de la même façon que dans le cas de la lecture. La figure 9 montre le chronogramme de lecture en mémoire. En premier temps, le microprocesseur présente l'adresse de case mémoire où il veut mémoriser une donnée. Ensuite, il met l'entrée d'écriture à l'état actif, dans ce cas c'est 0. Après, il active les entrées de sélection d'une boîtier mémoire et enfin il applique la valeur à mémoriser, pendant un temps au moins égal au temps d'accès de la mémoire, sur les bus de données.

I.4 Microprocesseur

I.4.1 Définition

Le microprocesseur est un circuit intégré complexe caractérisé par une très grande intégration. Il est chargé d'assurer l'ordonnancement des tâches et leur exécution en lisant le programme, en donnant les ordres pour aller rechercher des données en mémoire, en effectuant les divers traitements imposés par le programme, en affichant les résultats, etc. En fait, il réalise trois grandes fonctions :

+ aller rechercher un mot d'instruction en mémoire ;
+ déterminer la nature de la tâche à exécuter ;
+ exécuter cette tâche.

I.4.2 Historique des microprocesseurs

Depuis son invention jusqu'à aujourd'hui, le microprocesseur a connu une grande progression. Le premier microprocesseur a été inventé en 1969 par les physiciens de Intel : Marcian Hoff

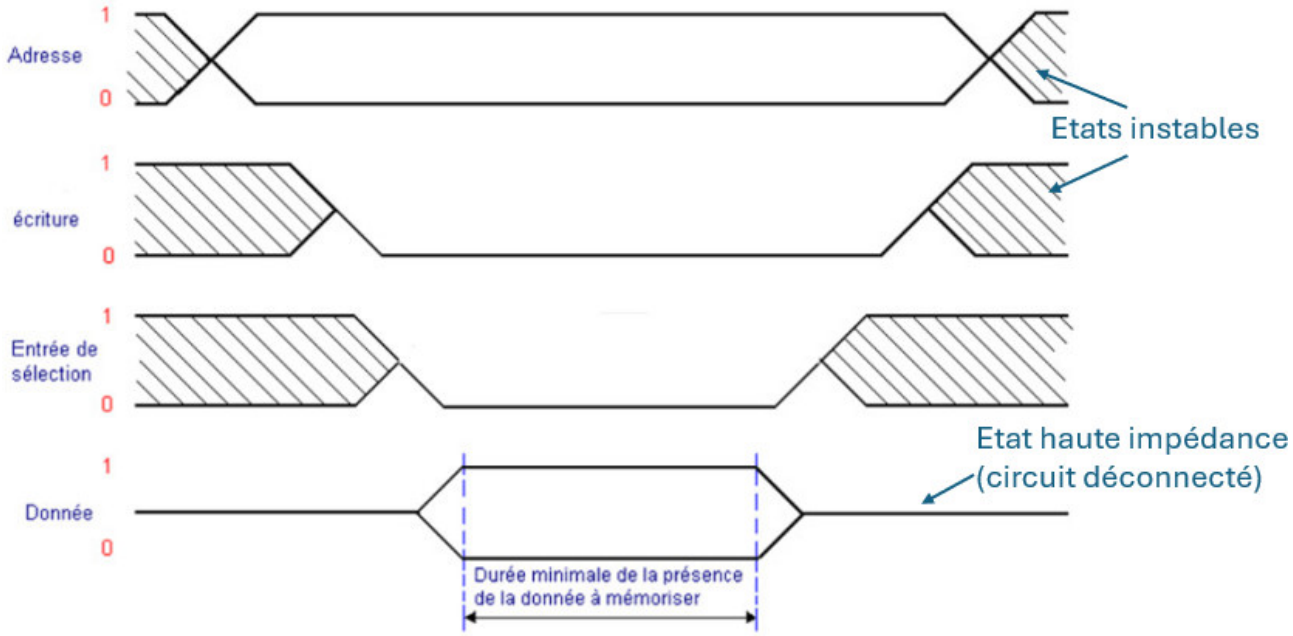


FIGURE 9 – Chronogramme de l'écriture en mémoire, $\overline{WR}$ actif à l'état bas.



Intel 8086 : 40 broches
DIP (Dual In-line Package)



Intel i9 : 1151–1700 pads
PGA (Pin Grid Array) or LGA (Land Grid Array)

FIGURE 10 – Exemples de microprocesseurs.

(ingénieur docteur en électronique américain) et Federico Faggin (physicien et inventeur italien). Le tableau I.1 résume les étapes les plus importantes dans l'évolution des microprocesseurs.

Date	Invention	Fabriquant	Horloge	Géométrie	Nombre de transistors	Loi de Moore
1971	Lancement du premier microprocesseur commercial, 4004 (4 bits)	Intel	740 kHz	10 µm	2 250	2 250
1974	6800 (8 bits)	Motorola	2 MHz	-	4 100	6 750
1974	8080 (8 bits)	Intel	2 MHz	6 µm	6 000	6 750
1978	8086 (16 bits)	Intel	5 MHz	3 µm	29 000	27 000
1979	6809 (8 bits)	Motorola	1 MHz	5 µm	40 000	36 000
1984	68020 (32 bits)	Motorola	16 MHz	2 µm	190 000	216 000
1985	80386 (32 bits)	Intel	16 MHz	1,5 µm	275 000	288 000
1991	R-4000 (64 bits)	MIPS	100 MHz	0,8 µm	1,35 millions	2,3 millions
1993	Pentium (64 bits)	Intel	60 MHz	0,8 µm	3,1 millions	4,6 millions
2006	Core 2	Intel	3 GHz	65 nm	291 millions	442,5 millions
2007	Phenom	AMD	2,6 GHz	65 nm	463 millions	590 millions
2008	Core i7	Intel	3,2 GHz	45 nm	730 millions	885 millions
2017	Ryzen 1000 (nbr. de cœurs 4-16)	AMD	3,2-4,1 GHz	14 nm	4 800 millions	-
2017	Core i9 (nbr. de coeurs 8 à 18)	Intel	2,60 GHz à 4,30 GHz	14 nm	-	-
2021	Rocket Lake	Intel	1,3 GHz à 5,3 GHz	14 nm	-	-
2023	Meteor Lake	Intel	1,3 GHz à 5,3 GHz	14 nm	-	-

TABLE I.1 – Quelques dates importantes dans les l'évolution des microprocesseurs.

I.4.3 Architecture de base d'un microprocesseur

Comme le montre la figure 11, un microprocesseur est construit autour de trois éléments principaux :
- unité de traitement ;
- unité de commande ;
- registres de données et d'adresses.

Ces trois éléments sont reliés entre eux par des bus internes permettant les échanges d'informations.

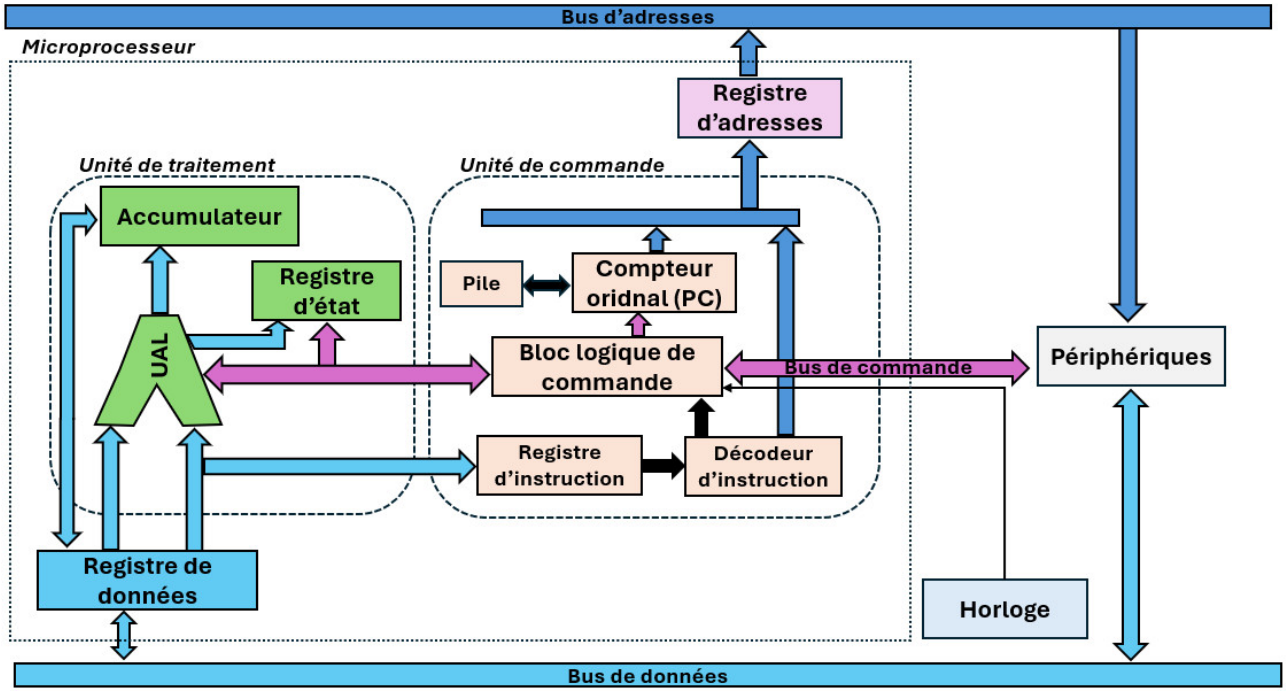


FIGURE 11 – Architecture de base d'un microprocesseur.

I.4.3.1 Unité de traitement : L'unité de traitement représente le cœur du microprocesseur, dont le rôle est le traitement des données. Elle est constitué des éléments suivants :

— **Unité Arithmétique et Logique** (UAL, ou ALU de l'anglais "arithmetic and logic unit") est un circuit qui réalise les opérations arithmétiques (Addition, soustraction, etc.) et logiques (ET, OU, Comparaison, Décalage, etc...). Les données à traiter arrivent dans l'UAL par l'intermédiaire du registre de données. Une fois l'opération à exécuter est réalisée, le résultat s'inscrit dans un autre registre, appelé *accumulateur*.

— **Accumulateur** est registre de travail qui serve à stocker un opérande au début d'une opération arithmétique et le résultat à la fin de l'opération.

— **Registre d'état** est souvent composé de 8 bits indépendants. Chacun de ces bits est un indicateur dont l'état dépend du résultat de la dernière opération réalisée par l'UAL. On les appelle *indicateur d'état ou flag ou drapeaux*. Dans un programme le résultat du test de leur état conditionne souvent le déroulement de la suite du programme. A titre d'exemple, on peut citer les indicateurs de :

  + Retenue intermédiaire (Auxiliary-Carry : AC)
  + Retenue (carry : C)
  + Zéro (Z)
  + Signe (Sign : S)
  + Débordement (overflow : OV ou V)
  + Parité (Parity : P)

I.4.3.2 Unité de commande : Le rôle principale de l'unité de commande est de séquencer le déroulement des instructions. En premier temps, elle se charge de rechercher l'instruction à exécuter en mémoire. Puis, elle en assure le décodage pour déterminer la nature de la tâche à exécuter, ensuite, effectue la préparation de l'instruction suivante. Elle est constituée de :

— **Compteur ordinal**, aussi appelé pointeur d'instruction ou compteur de programme (en anglais Instruction Pointer ou Program Counter), est un registre spécial du microprocesseur dont le contenu est l'adresse mémoire de la prochaine instruction à exécuter.

— **Pile** : Le rôle principal de la pile est de sauvegarder le contenu de compteur ordinal, chaque fois qu'il y a un saut de programme afin par exemple d'appeler un sous-programme. Ce dernier est hébergé dans un emplacement de la mémoire et appelé chaque fois que c'est nécessaire, en chargeant le compteur ordinal avec l'adresse correspondant à la première instruction. Quand son déroulement est terminé, la valeur initiale du compteur ordinal - sauvegardée dans la pile - réintégrera ce dernier, de façon à assurer la reprise du programme principal.

— **Registre d'instruction et le décodeur d'instruction :** chacune des instructions à exécuter est reçu par le registre instruction puis son décodage est effectué au niveau de décodeur d'instruction. On rappelle que chaque instruction est codée sous forme binaire.

— **Bloc logique de commande (ou séquenceur)** : son rôle est d'organiser l'exécution des instructions au rythme d'une horloge. Il génère tous les signaux de synchronisation internes/externes (bus de commande) du microprocesseur en fonction des différents signaux de commande provenant du registre d'état ou du décodeur d'instruction par exemple. C'est donc un automate réalisé soit de façon microprogrammée, on parle alors de micro-microprocesseur, soit de façon câblée.

I.4.3.3 Registres d'adresses et données : Ces registres sont des petites mémoires très rapides intégrées directement dans le microprocesseur (proche de l'UAL et de l'unité de contrôle). Ils servent à stocker temporairement des données, des instructions ou des adresses pendant l'exécution des programmes. Contrairement à la mémoire RAM, les registres ont une capacité très limitée (quelques dizaines de bits), mais disposent d'un temps d'accès très rapide.

Le registre d'adresses contient l'adresse servant à repérer l'emplacement mémoire que le microprocesseur désire utiliser. Au début d'un cycle de recherche et d'exécution, les contenus du registre d'adresses et du compteur ordinal sont donc identiques. Mais contrairement au compteur ordinal, le registre d'adresses n'est pas automatiquement incrémenté pendant l'exécution de l'instruction ; il reste sous le contrôle du programme qui peut lui demander par exemple de rester "pointé" sur une adresse donnée, et modifier ainsi le contenu de l'emplacement de mémoire correspondant, en fonction des résultats du traitement obtenus.

Le registres de données appelé aussi registre tampon contient la donnée à écrire en mémoire ou lue. Il est directement connecté au bus de données.

I.4.4 Fonctionnement d'un microprocesseur

Un microprocesseur sous tension est toujours prêt à exécuter les différentes instructions d'un programme qu'est déposé dans la mémoire, mais seulement si on lui en donne l'ordre en "levant un drapeau". Une instruction peut être codée sur un ou plusieurs octets. Le format générale d'une instruction est la suivante :

Opération	opérande 1	,	opérande 2

Exemple :

$$\underbrace{\text{Additionner}}_{\text{opération}}, \underbrace{\text{donnée de case mémoire 1, donnée de la case mémoire 2}}_{\text{opérandes}}$$

L'exécution d'une instruction par le microprocesseur se fait en trois phases :

— Récupération de l'instruction ;

— Décodage de l'instruction et recherche de l'opérande ;

— Exécution de l'instruction.

I.4.4.1 Récupération de l'instruction : L'adresse de l'instruction à exécuter se trouve toujours dans le compteur de programme. Par exemple, supposons qu'il contienne la valeur \$8000H (en hexadécimal sur 16 bits). Ce nombre représente donc l'adresse de l'instruction à effectuer. Pour l'identifier, le µP va devoir en premier lieu aller la chercher (to fetch en anglais) ce qui implique toute une succession d'opérations élémentaires (voir la figure 12) :

— Transfert le contenu de compteur ordinal qui contient l'adresse de l'instruction suivante du programme vers le bus d'adresses ;

— Envoi d'un ordre de lecture à la mémoire centrale (Read : $\overline{RD}$) ;

— Au bout d'un certain temps (temps d'accès à la mémoire), le contenu de la case mémoire sélectionnée est disponible sur le bus des données ;

— L'instruction est stockée dans le registre instruction du processeur.

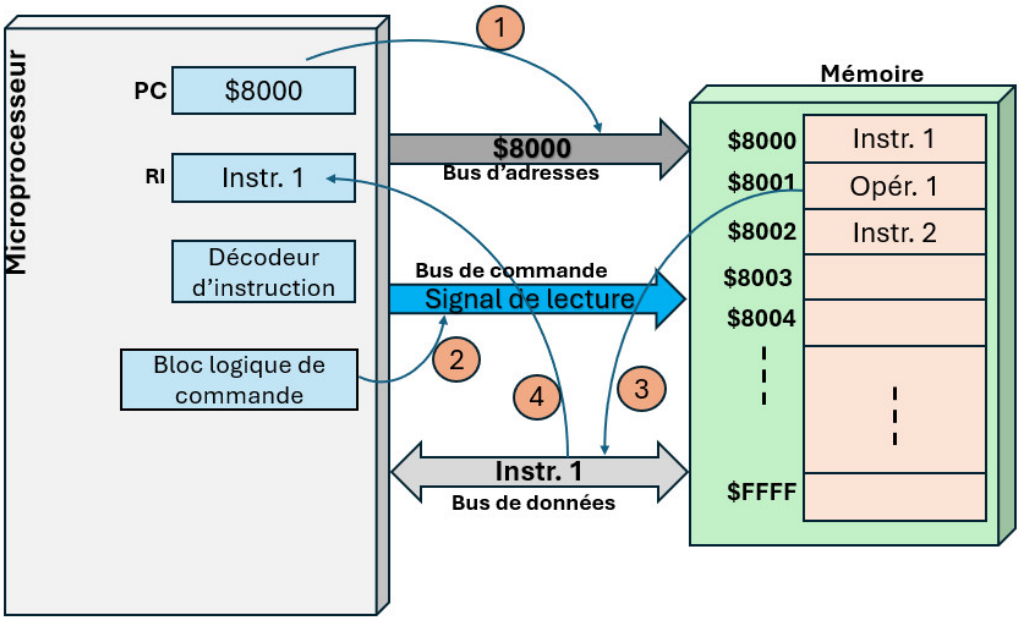


FIGURE 12 – Étapes de récupération de l'instruction.

I.4.4.2 Décodage de l'instruction et recherche de l'opérande : Le registre d'instruction contient maintenant le premier mot de l'instruction qui peut être codée sur plusieurs mots. Ce premier mot contient le code opératoire qui définit la nature de l'opération à effectuer (addition, soustraction, rotation, ...) et le nombre de mots de l'instruction.

Le déroulement de la deuxième phase est comme suit :

— Décodage :

  — Lecture éventuelle des autres mots d'instruction (selon le format) ;

  — L'unité de commande transforme l'instruction en une suite de commandes élémentaires nécessaires au traitement de l'instruction.

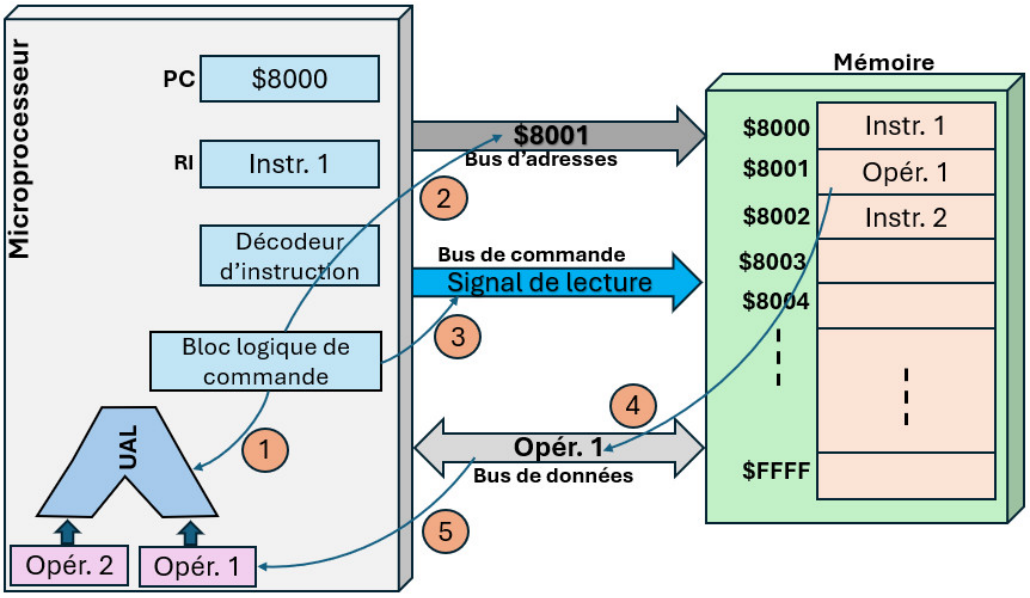


FIGURE 13 – Décodage de l'instruction et recherche de l'opérande.

— Recherche d'opérandes. Si l'instruction à besoin des opérandes, alors ils peuvent être obtenu soit par :
  — Accès aux registres (si mode registre) ;
  — Calcul d'adresse et recherche d'opérandes mémoire (si mode mémoire), puis stocker l'opérande dans un registre.

La figure 13 montre une illustration de la phase de décodage et de recherche des opérandes dans le cas où l'opérande 1 est stocké dans la mémoire par contre l'opérande 2 est stocké dans le registre.

I.4.4.3 Exécution de l'instruction

— Exécution. Deux types de l'exécution peuvent avoir lieu :
  — Calcul de l'adresse de branchement pour les instructions de contrôle
  — Calcul dans l'ALU pour les instructions arithmétiques ou logiques. Pour cela, le microprogramme réalisant l'instruction est exécuté.
— Les drapeaux sont positionnés (registre d'état).
— Écriture de résultats. Suivant le type d'exécution effectué, on distingue deux types d'écriture des résultats :
  — Modification de PC pour les instructions de contrôle ;
  — Modification de l'opérande destination pour les instructions arithmétiques ou logiques ;
— L'unité de commande positionne le PC pour l'instruction suivante.

La figure 14 montre une illustration de la phase de l'exécution d'une instruction et de stocker dans l'accumulateur.

I.4.5 Forme d'instructions : Architectures CISC et RISC

En se basant sur le jeu d'instruction, on distingue entre deux grandes familles des microprocesseurs :

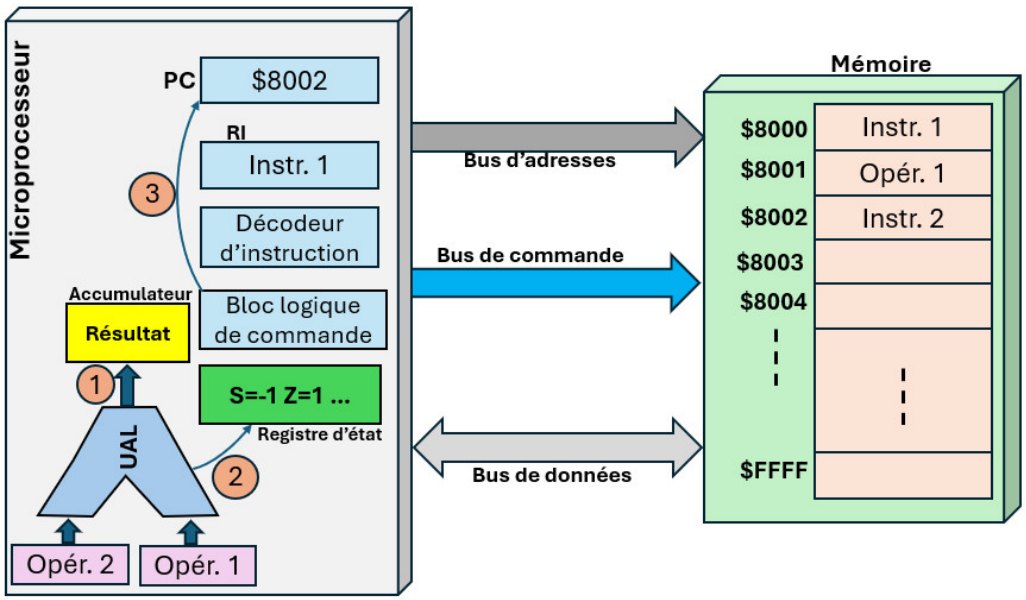


FIGURE 14 – Exécution d'une instruction et stockage dans l'accumulateur.

— Microprocesseur de type CISC (Complex Instruction Set Computer), on dit aussi microprocesseur à jeu d'instruction complexe ; Exemple : Motorola 6809, Intel 8086, Motorola 680x0 ;

— Microprocesseur de type RISC (Reduced Instruction Set Computer), on dit aussi microprocesseur à jeu d'instruction réduit. Exemple : SPARC, POWER, PowerPC, MIPS et ARM.

I.4.5.1 Architecture CISC :

Contexte : Au début la conception de microprocesseur de type CISC était la seule envisageable. En effet,

+ vue que la mémoire travaillait lentement par rapport au processeur, on pensait qu'il était plus intéressant de soumettre au microprocesseur des instructions complexes. Ainsi, plutôt que de coder une opération complexe par plusieurs instructions plus petites (qui demanderaient autant d'accès mémoire très lent), il semblait préférable d'ajouter au jeu d'instructions du microprocesseur une instruction complexe qui se chargerait de réaliser cette opération.
+ le développement des langages de haut niveau pose de nombreux problèmes quant à la conception de compilateurs. On a donc eu tendance à incorporer au niveau processeur des instructions plus proches de la structure de ces langages.

Principe : Pour ces raisons, l'architecture CISC a été développée où le microprocesseur doit exécuter des tâches complexes par instruction unique. Pour une tâche donnée, un microprocesseur exécute ainsi un petit nombre d'instructions mais chacune nécessite un plus grand nombre de cycles d'horloge. Le code machine de ces instructions varie d'une instruction à l'autre et nécessite donc un décodeur complexe (micro-code).

Exemple :

ADD AX, [B200] ; additionne le contenu de la case mémoire qui se trouve à l'adresse B200 au contenu du registre AX.

Dans ce cas, une seule instruction fait plusieurs choses : accéder à la mémoire, lire la donnée, puis l'additionner avec le contenu du registre AX.

I.4.5.2 Architecture RISC :

context, cette architecture consiste à réduire le jeu d'instruction des microprocesseurs. En effet, des études statistiques menées au cours des années 70 ont clairement montré que les programmes générés par les compilateurs se contentaient le plus souvent d'affectations, d'additions et de multiplications par des constantes. Ainsi, 80% des traitements des langages de haut niveau faisaient appel à seulement 20% des instructions du microprocesseur. D'où l'idée de réduire le jeu d'instructions à celles le plus couramment utilisées et d'en améliorer la vitesse de traitement.

Principe : Architecture RISC est donc une architecture dans laquelle le nombre d'instructions est réduit (chargement, branchement, appel sous-programme). Elles peuvent donc être réalisées à partir de séquenceur câblé. Leur réalisation libère de la surface permettant d'augmenter le nombre de registres ou d'unités de traitement par exemple. Chacune de ces instructions s'exécutent ainsi en un cycle d'horloge. Bien souvent, ces instructions ne disposent que d'un seul mode d'adressage. Les accès à la mémoire s'effectuent seulement à partir de deux instructions (Load et Store). Par contre, les instructions complexes doivent être réalisées à partir de séquences basées sur les instructions élémentaires, ce qui nécessite un compilateur très évolué dans le cas de programmation en langage de haut niveau.

Exemple :

LOAD R1, 1234H ; charge le contenu de la case mémoire d'adresse 1234H dans le registre R1
ADD R2, R1 ; additionne R2 avec R1

Dans ce cas, deux instructions sont utilisé pour faire l'addition : une pour charger, une pour additionner. Mais chaque instruction est très simple et rapide.

I.4.5.3 Comparaison entre CISC et RISC

: Le tableau I.2 présente une comparaison entre les deux architectures CISC et RISC.

CISC	RISC
- Instructions composés (complexes) sont exécutées en plusieurs cycles	- Instructions simples sont exécutées en un seul cycle
- Instructions ont un format variable	- Instructions ont un format fixe
- Divers instructions peuvent accéder à la mémoire	- Seulement les instruction load et store peuvent accéder à la mémoire
- Décodeur complexe (séquenceur microprogrammé)	- Décodeur simple (séquenceur câblé)
- Moins de registres	- Plus de registres
- Plus de modes d'adressage	- Moins de modes d'adressage
- Compilateur simple	- Compilateur complexe

TABLE I.2 – Comparaison entr CISC et RISC.

I.4.6 Procédure de transfert des données : Architectures de Von Neumann et Harvard

La procédure de transfert des données et instructions entre le microprocesseur et les mémoires est un élément important qui conditionne directement les performances d'un microprocesseur. Il existe deux architectures d'accès à la mémoire par le microprocesseur :

— Von Neumann

— Harvard

Ces deux approches décrivent la manière dont un microprocesseur organise et gère les mémoires pour stocker les instructions et les données.

I.4.6.1 Architecture de Von Neumann : L'architecture de Von Neumann a été proposée en 1945 par le mathématicien et physicien Hongrois/Américain John Von Neumann. Son principe repose sur le concept de programme enregistré en mémoire (stored program concept). Dans cette architecture, les instructions du programme (code machine) et les données manipulées par ces instructions sont stockées à la fois dans le même espace mémoire adressé par le microprocesseur. Le microprocesseur communique donc avec cette mémoire au moyen d'un seul bus qui transporte à la fois les adresses, les données et les signaux de contrôles (bus d'adresses, bus de données et bus de contrôle partagés). Ce choix simplifie la conception matérielle et réduit les coûts, mais leur inconvénient majeur est qu'il ne permet pas au processeur d'accéder simultanément à une instruction et à une donnée, ce qui limite la vitesse d'exécution. Cette limitation est connue sous le nom de Von Neumann bottleneck. La figure 15 décrit le principe de cette architecture.

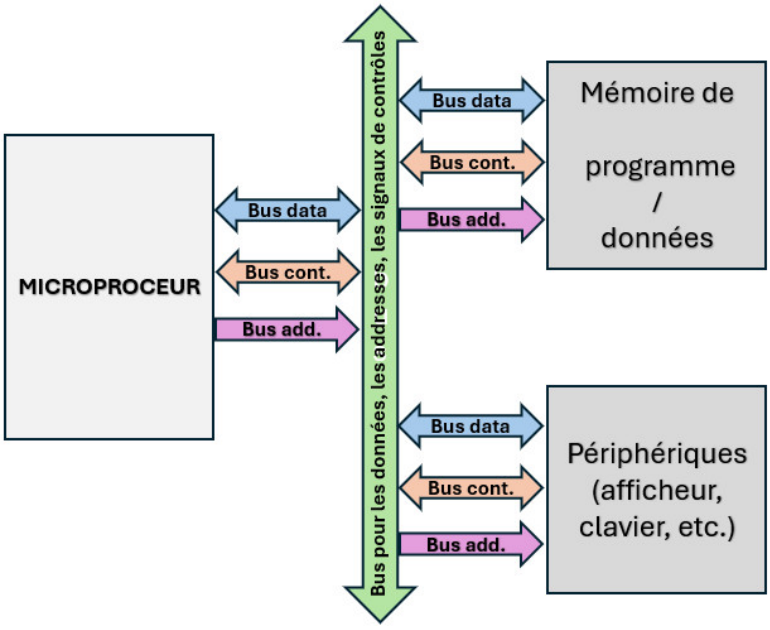


FIGURE 15 – Architecture de Von Neumann.

I.4.6.2 Architecture de Harvard : Contrairement à l'architecture de Von Neumann, l'architecture de Harvard propose de séparer entre la mémoire programme et la mémoire de données (voir la figure 16). Chaque mémoire dispose de son propre bus d'adresses et de données, ce qui permet au processeur d'accéder simultanément à une instruction et à une donnée, améliorant

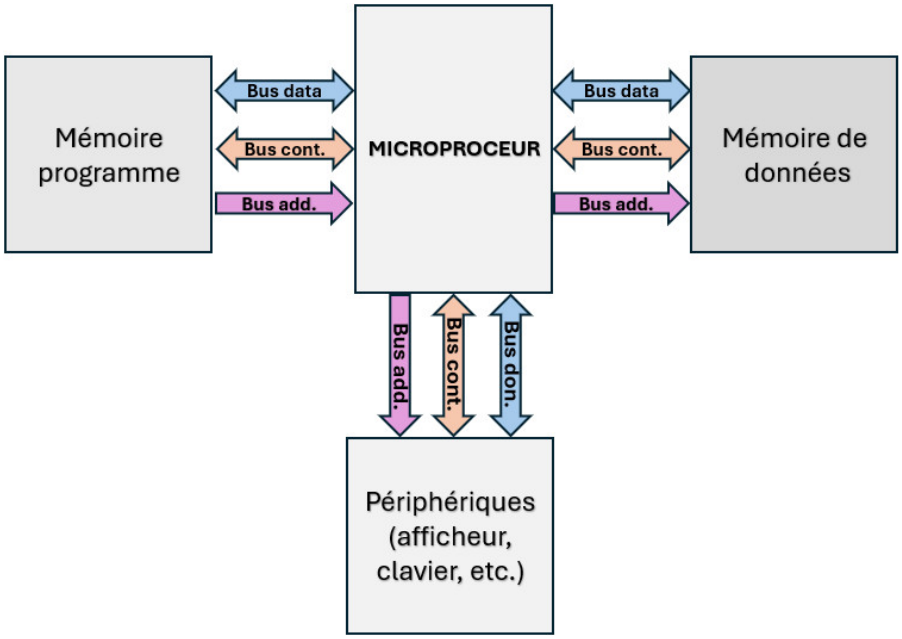


FIGURE 16 – Architecture de Harvard.

ainsi significativement la vitesse d'exécution. Cette architecture permet donc de surmonter l'inconvénient de l'architecture de Von Neumann (Von Neumann bottleneck).

Malgré que l'architecture Harvard permet d'améliorer la vitesse du microprocesseur, cependant, la séparation des mémoires programmes et de données rend la conception matérielle plus complexe et parfois plus coûteuse. De plus, la mémoire programme est généralement non-modifiable (ROM ou Flash), tandis que la mémoire de données est volatile (RAM), ce qui limite sa flexibilité par rapport à l'architecture de Von Neumann.

Le tableau I.3 résume la différence entre les architectures Harvard et Von Neumann.

Paramètres	Architecture Von Neumann	Architecture Harvard
- Mémoires	- Les données et les instructions (programme) sont stockées dans la même mémoire	- Les données et les instructions (programme) sont stockées dans des mémoires séparées
- Bus	- Bus d'adresse commun pour les données et les instructions	- Bus de données et d'instructions sont séparés
- Nombre de cycles	- Le programme est exécuté de façon séquentielle et nécessite plus de cycles	- Le programme est exécuté en parallèle avec les données, donc nécessite moins de cycles
- Transfert de données/ instruction	- Données ou instruction transférés en un seul cycle	- Données et code transférés dans le même cycle
- Exigences matérielles	- Nécessite moins de matériel que Harvard	- Nécessite plus de matériel que Von Neumann

TABLE I.3 – Architectures Von Neumann vs. Harvard.

Le Tableau I.4 montre les domaines d'applications des architectures Harvard et Von Neumann

Architecture Von Neumann	Architecture Harvard
- Ordinateurs personnels (PC) - Serveurs - Supercalculateurs et ordinateurs utilisés en recherche scientifique	- Microcontrôleurs et microprocesseurs embarqués - Systèmes temps réel - Processeurs de traitement du signal (DSP)

TABLE I.4 – Domaines d'applications des architectures Harvard et Von Neumann

I.4.7 Procédure d'exécution des instructions : Architectures classique et pipeline

Au fil du temps, plusieurs architectures ont été proposées pour améliorer le temps d'exécution des tâches par les microprocesseurs. Pour cela, certaines technologies proposent d'augmenter tout simplement la fréquence de l'horloge du microprocesseur. Mais l'inconvénient de cette technologie est que l'accélération des fréquences provoque un surcroît de consommation, ce qui entraîne le chauffage des composants électroniques. On est alors amené à équiper les microprocesseurs de systèmes de refroidissement ou à diminuer la tension d'alimentation.

D'autres technologies proposent d'augmenter la puissance de traitement du microprocesseur en diminuant le nombre moyen de cycles d'horloge nécessaire à l'exécution d'une instruction. Dans le cas d'une programmation en langage de haut niveau, cette amélioration peut se faire en optimisant le compilateur. Il faut qu'il soit capable de sélectionner les séquences d'instructions minimisant le nombre moyen de cycles par instructions. Une autre solution est d'utiliser une architecture de microprocesseur qui réduise le nombre de cycles par instruction. Parmi les différentes architectures proposées, on trouve *l'architecture pipeline* qui est la plus populaire dans les systèmes à microprocesseur.

I.4.7.1 Architecture pipeline

Définition : Le pipeline est une technologie qui consiste à augmenter la vitesse d'exécution des instructions en parallélisant les différentes phases de leur exécution. Ainsi, réduire le temps d'exécution des programmes.

Principe : Dans l'architecture classique (voir la figure 17.a), l'exécution des instructions se fait en série, c.à.d., l'exécution de l'instruction suivante ne commence qu'après la fin de l'exécution de l'instruction encore. Le fonctionnement du microprocesseur n'est donc pas efficace.

A l'inverse de l'architecture classique, l'architecture pipeline voir la figure (17.b) permet d'améliorer l'efficacité du microprocesseur. En effet, lorsque la première phase de l'exécution d'une instruction est achevée, l'instruction entre dans la seconde phase de son exécution et en parallèle la première phase de l'exécution de l'instruction suivante débute. Il peut donc y avoir une instruction en cours d'exécution dans chacune des phases et chacun des composants du microprocesseur peut être utilisé à chaque cycle d'horloge. Dans cette structure, le microprocesseur débute l'exécution d'une instruction à chaque cycle et le pipeline est pleinement occupé à partir du quatrième cycle. Le temps d'exécution d'une instruction n'est pas réduit, mais le débit d'exécution des instructions est considérablement augmenté.

Un microprocesseur à architecture pipeline se caractérise par le nombre d'étapes utilisées pour l'exécution d'une instruction. On appelle aussi ce nombre de phases le nombre d'étages

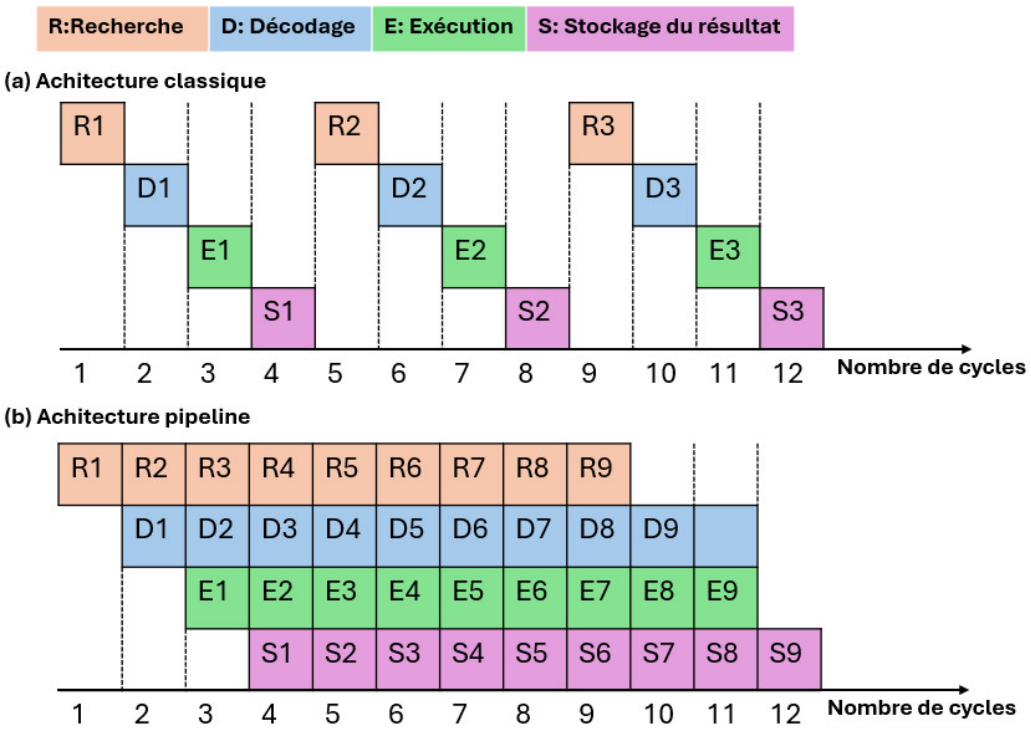


FIGURE 17 – Architecture classique vs. architecture pipeline.

du pipeline. Dans le cas de l'exemple de la figure 17 le nombre d'étages est 4. Mais, ce nombre peut différer suivant le constructeur. Par exemple :
- Les Pentiums 2, 3 et 4 d'Intel comprennent respectivement un pipeline de 12, 10 et 20 étages ;
- L'Athlon d'AMD comprend un pipeline de 11 étages.

En plus de l'architecture pipeline d'autres architectures ont été développées comme :
+ Architecture superscalaire ;
+ Architecture pipeline et superscalaire ;
+Architecture VLIW ; etc.

I.4.7.2 Gain de performance :

Le gain de l'architecture pipeline est obtenu en calculant le rapport entre le nombre de cycles nécessaires pour exécuter un ensemble d'instructions en utilisant l'architecture classique sur le nombre de cycles nécessaires pour exécuter le même nombre d'instructions en utilisant l'architecture pipeline. En effet, supposons qu'on a n instructions à exécuter et chaque instruction s'exécute en k cycles d'horloge, alors :

— Avec l'architecture classique, il faut donc $n \times k$ cycles d'horloge ;

— Avec l'architecture pipeline, il faut k cycles d'horloge pour exécuter la première instruction puis $n-1$ cycles pour les $n-1$ instructions suivantes, c.à.d., au totale, on aura besoin de $k+n-1$ cycles.

Le gain est donc donné par :

$$G = \frac{n.k}{k+n-1} \tag{1}$$

I.4.7.3 Problèmes et solutions des pipelines Problèmes rencontrés lors de l'exécution d'instructions par le pipeline sont classés en trois types :

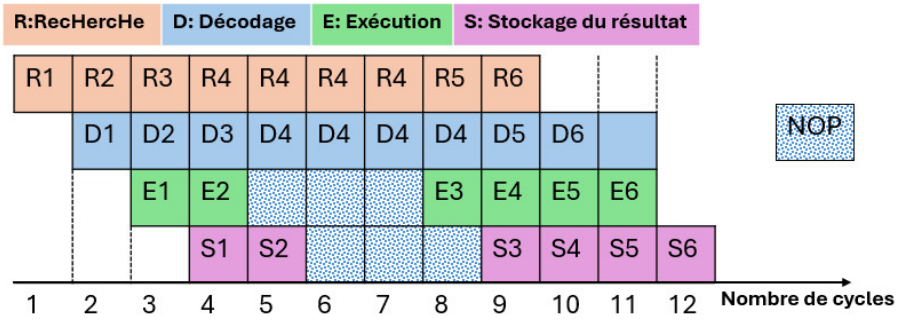


FIGURE 18 – Résoudre les aléas d'un pipeline par blocage des instructions

— *Aléas de Données (Data Hazards)* : proviennent lorsque une instruction dépend du résultat d'une instruction précédente non encore terminée.
Exemple : ADD R1, R2, R3 suivie de SUB R4, R1, R5 (SUB dépend de R1).

— *Aléas de Contrôle (Control Hazards)* : Surviennent des instructions de branchement (sauts, appels de fonctions) qui modifient le compteur de programme (PC). Dans ce cas, le processeur ne sait pas quelle instruction charger ensuite (branchement pris ou non) avant la fin du calcul, bloquant l'arrivée de nouvelles instructions.

— *Aléas Structurels (Structural Hazards)* :proviennent quant deux instructions différentes dans le pipeline tentent d'utiliser la même ressource matérielle simultanément.
Exemple : Un seul port d'accès mémoire utilisé à la fois pour chercher une instruction (Fetch) et écrire un résultat (Writeback).

Plusieurs solution ont été proposées pour résoudre les aléas d'un pipeline, y compris :

— Blocage (Stalling/Bubbles) : Arrêter l'instruction fautive et celles qui suivent jusqu'à la résolution du problème, insérant des cycles vides (NOP - No Operation), comme le montre la figure **??**.

— Prédiction de branchement : Prédire l'issue d'un saut pour continuer à alimenter le pipeline, avec une pénalité si la prédiction est fausse.

— Réacheminement (Forwarding/Bypassing) : Transférer directement le résultat produit à l'unité fonctionnelle qui en a besoin avant l'écriture dans le registre.

II Exemples de systèmes à microprocesseur

II.1 Ordinateurs

II.1.1 Définition

Un ordinateur est un système électronique capable de stocker, traiter et restituer des informations. Son rôle fondamental est de recevoir des données en entrée, les traiter, puis de fournir un résultat en sortie, que ce soit sous forme visuelle (écran), imprimée (imprimante) ou via d'autres périphériques. Il est composé de deux partie principales :

— **Partie matérielle (hardware)** : désigne l'ensemble des composants physiques et électroniques qui constituent la machine (comme la carte mère, les mémoires, etc.).

— **Partie logicielle (software)** : regroupe l'ensemble des programmes et applications qui exploitent les ressources matérielles de l'ordinateur (comme Windows, Office, etc.).

II.1.2 Évolution des ordinateurs avec l'évolution des microprocesseurs

L'histoire des ordinateurs commence au milieu du 20e siècle et elle passe par plusieurs étapes :

— 1936-1956, les premiers ordinateurs, comme l'ENIAC (1946), ont été réalisés (figure 19). Ils sont constitués de milliers de tubes à vide, occupent des salles entières et sont destinés à des calculs scientifiques ou militaires. Leur programmation se fait manuellement à l'aide de câbles et d'interrupteurs.

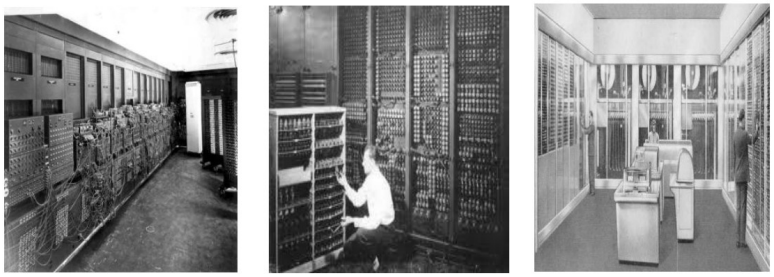


FIGURE 19 – Ordinateur ENIAC inventé en 1946.

— 1957-1965, la deuxième génération d'ordinateurs sont apparue avec l'invention du transistor en 1947 par les Américains John Bardeen, William Shockley et Walter Brattain. Dans cette génération, les tubes électroniques qui sont fragiles et de grande taille sont remplacés par les transistors qui sont plus petits et fiables. Les ordinateurs à base des transistors ont dominé l'informatique dans la fin des années 1950 et le début des années 1960. En 1959, la société IBM lance l'ordinateur IBM 1401 (Figure 20), qui utilisait des cartes perforées (support de stockage de données en carton rigide). Il fut le premier ordinateur vendu à plus de 10 000 exemplaires.

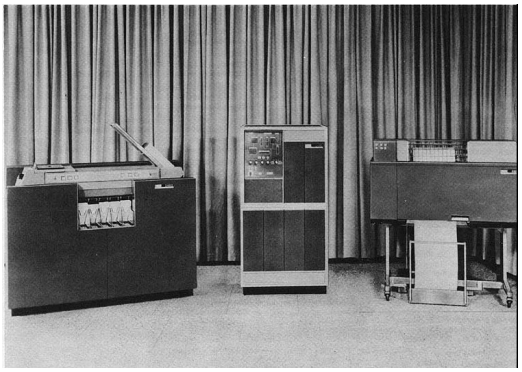


FIGURE 20 – Ordinateur IBM1401 inventé en 1959 par IBM.

— 1963-1971, cette période caractérisée par l'émergence des ordinateurs à base de circuits intégrés, troisième génération. En 1964, IBM produit la série 360 (figure 21), première gamme d'ordinateurs compatibles entre eux et première gamme aussi à couvrir l'ensemble des domaines d'applications commerciales et scientifiques. L'année 1970 se caractérise par l'apparition des mini-ordinateurs.

— 1971-1980, la quatrième génération des ordinateurs est celle des ordinateurs qui sont à base des microprocesseurs inventés.

— 1980-2000 : C'est la naissance des PC (Personal Computers, figure 22), accompagnés de systèmes d'exploitation comme MS-DOS puis Windows. En plus, l'Internet devient com-



FIGURE 21 – Ordinateur s360 inventé en 1965 par IBM (à gauche) et exemples de mini-ordinateurs (à droite).

mercialement accessible, connectant des millions d'ordinateurs dans le monde et changeant radicalement la façon dont les gens utilisent l'informatique.



FIGURE 22 – Ordinateurs personnels.

— Années 2000 et 2010 : Les ordinateurs deviennent plus petits, plus maniables, et le développement des smartphones et tablettes voit le cloud computing émerger, avec des services informatiques disponibles en ligne (Fig. 23).



FIGURE 23 – PC, smartphones et tablettes.

— Depuis 2010 à aujourd'hui, l'évolution des ordinateurs est marquée par la montée en puissance du cloud computing et des objets connectés, transformant nos usages avec le smartphone comme appareil central, l'omniprésence d'Internet (5G, fibre), la virtualisation, l'essor de l'IA (apprentissage automatique), l'évolution des processeurs (multi-cœurs, ARM) et le développement des calculateurs neuromorphiques et quantiques, rendant les services plus mobiles, intelligents et intégrés.

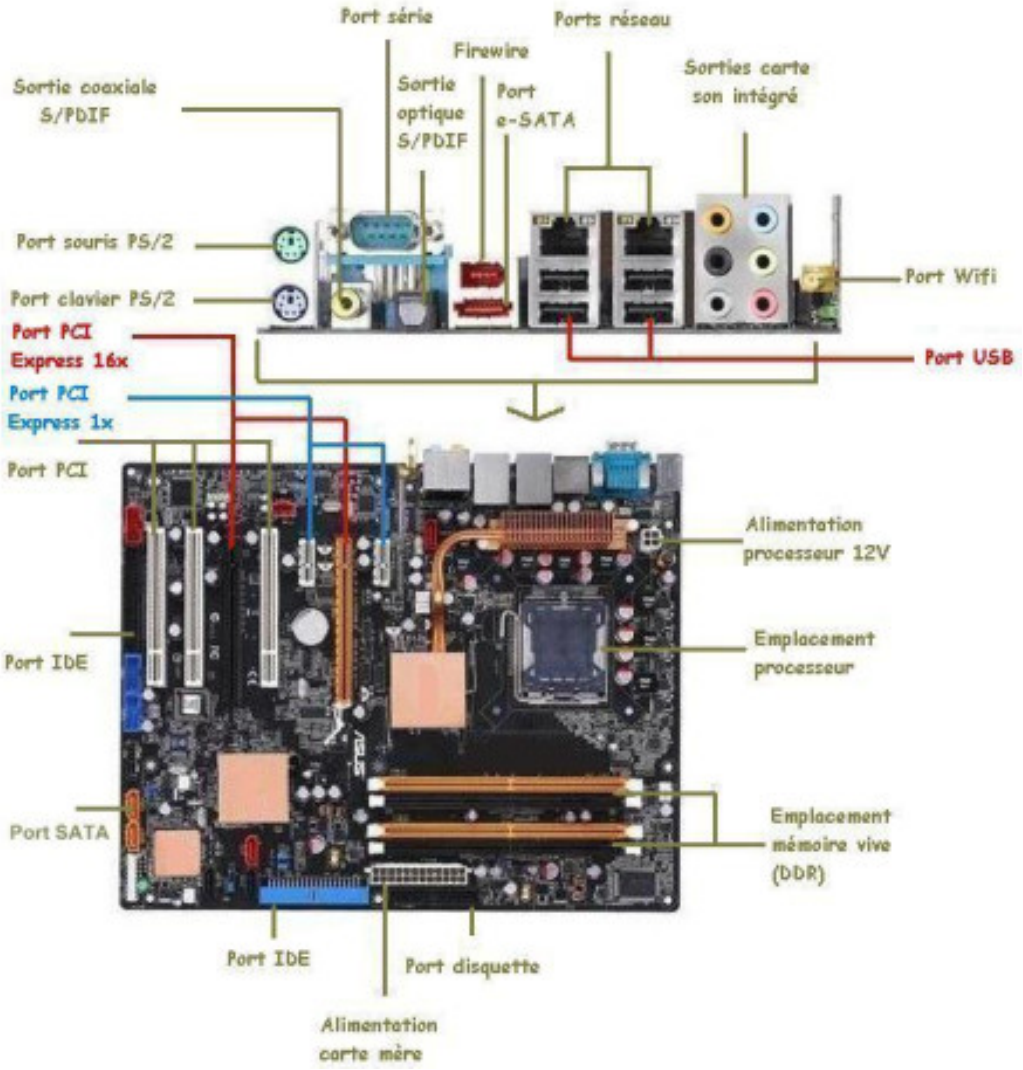


FIGURE 24 – Architecture de la carte mère.

II.1.3 Carte mère et le microprocesseur

La carte mère ou « motherboard » est un circuit imprimé qui relie et coordonne l'ensemble des éléments matériels (processeur, mémoires, périphériques). Elle est équipée de circuits électroniques (chipset, régulateurs de tension) et de connecteurs permettant l'intégration et la communication entre les différents modules. Les principaux éléments de la carte mère sont (figure 24) :

— **Mémoire vive**, ou mémoire RAM (Random Access Memory), est une mémoire de travail volatile et rapide qui stocke temporairement les données dont les applications ont besoin pour s'exécuter. Elle se présente sous forme de barrettes connectées à la carte mère, installée dans des slots DIMM, et permet au processeur d'accéder rapidement aux données, ce qui améliore considérablement les performances de l'ordinateur, surtout lorsqu'il exécute plusieurs programmes simultanément.

— **Socket processeur** (CPU socket) : emplacement où est installé le microprocesseur. Il conditionne la compatibilité avec une famille de processeurs (Intel, AMD, ARM, etc.).

— **BIOS/UEFI** : puce mémoire contenant le microprogramme de démarrage et les paramètres de configuration de la carte mère.

— **Bus d'extension et connecteurs PCIe** : permettent d'ajouter des cartes graphiques, cartes son, cartes réseau, etc.

— **Connecteurs de stockage** : ports **SATA** ou **NVMe** (M.2) pour disques durs, SSD et lecteurs optiques.

— **Connecteurs d'alimentation** : assurent l'apport en énergie depuis le bloc d'alimentation (24 broches ATX, connecteur CPU, PCIe).

— **Ports d'entrées/sorties (I/O)** : ils regroupent USB, HDMI/DisplayPort, audio, Ethernet, etc.

— **Régulateurs de tension (VRM)** : circuits assurant une alimentation stable du processeur et de la mémoire.

II.1.4 Périphériques

Les périphériques représentent les dispositifs externes reliés à un ordinateur et assurent l'interface entre l'utilisateur et la machine. Ils permettent soit d'introduire des informations vers l'ordinateur, soit de restituer des résultats sous différentes formes (texte, image, son, etc.). On distingue généralement trois catégories de périphériques :

— **Périphériques d'entrées** : clavier, souris, scanner, webcam, Microphone, manette de jeux ;

— **Périphériques de sorties** : écran, imprimante, haut-parleur, vidéo projecteur ;

— **Périphériques d'entrées/sorties** : Les supports de stockage d'informations (FLASH DISQUE, FLASH USB, CD/DVD, Disque dur), modem, routeur.

II.2 Microcontrôleur

Un microcontrôleur est un circuit intégré regroupant tous les éléments d'un système à microprocesseur. Il contient généralement :

— Un microprocesseur ;

— les mémoires de données et programmes (RAM, ROM, etc.) ;

— Des interfaces (synchrones ou asynchrones) pour le dialogue avec d'autres unités ;

— Des convertisseurs analogiques/numériques pour la conversion des signaux ;

— Des temporisateurs pour générer ou mesurer la durée des signaux ; etc.

III Conclusion

Dans ce chapitre, nous avons mis l'accent sur l'architecture générale des systèmes à microprocesseur, ainsi que le fonctionnement et le rôle de chacun de ses composants. Dans le chapitre suivant, nous allons nous intéresser de plus près à l'étude d'un microprocesseur. À des fins pédagogiques et pour simplifier la compréhension, nous avons choisi d'étudier le microprocesseur Intel 8086.

Chapitre II

Architecture et fonctionnement du microprocesseur Intel 8086

A travers ce chapitre, nous proposons de découvrir l'architecture interne d'un microprocesseur, leurs composants, leurs rôles et leurs interactions pour traiter les programmes et les données. Parmi les différents microprocesseurs existants, nous choisissons de traiter le microprocesseur Intel 8086 qui est un microprocesseur classique souvent utilisé dans les milieux universitaires pour enseigner les concepts fondamentaux de la programmation en langage d'assemblage et de l'architecture des microprocesseurs. Il a été introduit en 1978 par Intel comme une extension 16 bits du microprocesseur 8 bits 8080 d'Intel, utilisant la technique de segmentation de la mémoire pour gérer efficacement la zone mémoire utilisable. C'est le premier microprocesseur de la famille Intel x86 (8086, 80186, 80286, 80386, 80486, Pentium, ...). Plusieurs successeurs de ce processeur se sont, en effet, développés petit à petit en ajoutant à chaque processeur des instructions et des fonctionnalités supplémentaires, mais tout en se basant à chaque fois sur les spécificités du processeur 8086.

I Architecture externe du µp 8086

I.1 Brochage

Le µp 8086 est un circuit intégré présenté sous forme d'un boîtier DIP à 40 broches comme le montre la figure 1. Il fonctionne soit en mode minimal (pour un système à un seul processeur), soit en mode maximal (pour un système à plusieurs processeurs). Le mode minimal est plus simple : le 8086 contrôle directement les bus. En revanche, le mode maximal est plus complexe : dans ce cas, un contrôleur de bus externe est utilisé pour gérer les signaux de contrôle et prendre en charge les différents microprocesseurs.

La sélection d'un mode se fait à l'aide de la broche $MN/\overline{MX}$ (broche 33). Parmi les 40 broches : le rôle de 32 broches (1-23 et 32-40) est le même pour les deux modes de fonctionnement du µp 8086, par contre le rôle des autres 8 broches (24-31) change suivant le mode de fonctionnement.

I.2 Fonctions des broches

Les 40 broches du µp 8086 peuvent être regroupées suivant leur fonction en 10 catégories, comme le montre le schéma fonctionnel de la figure 2.

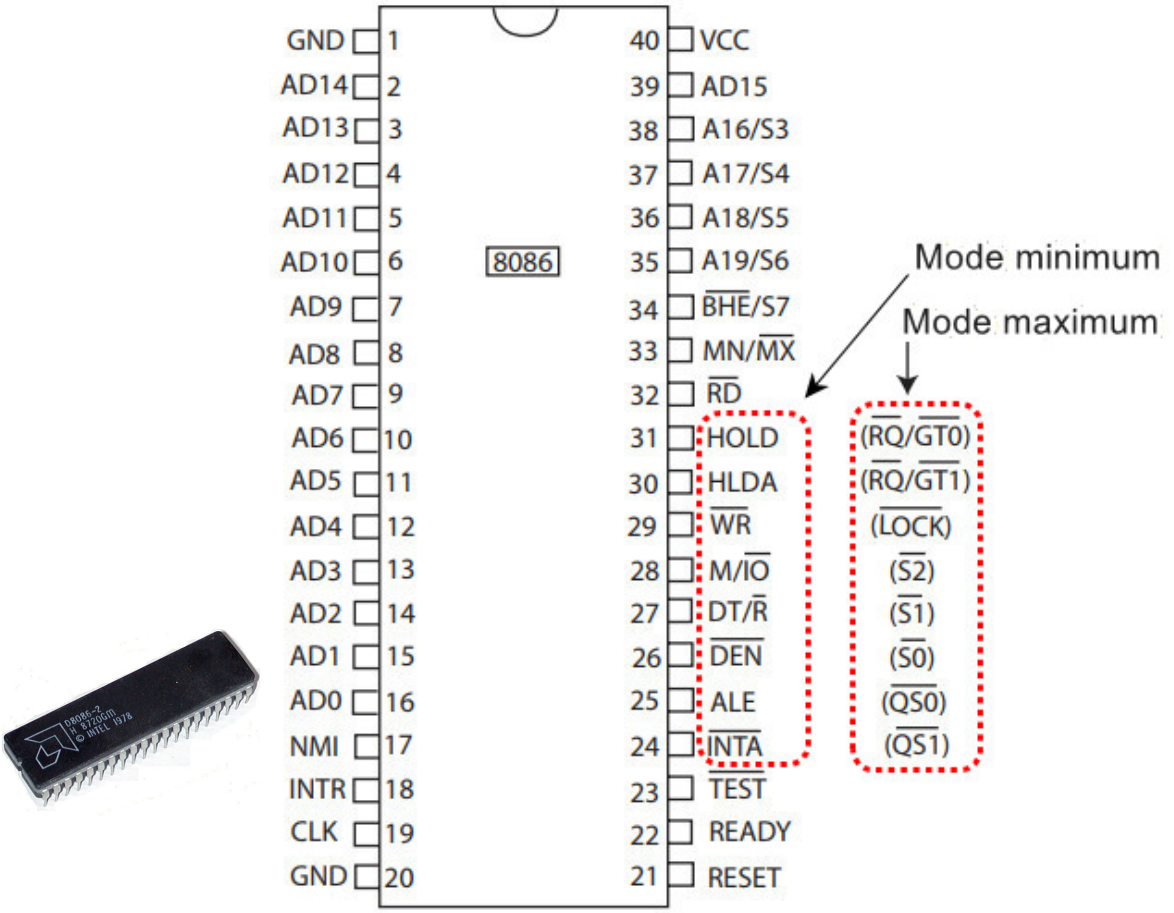


FIGURE 1 – Circuit intégré 8086 et affectation des broches du 8086 en mode min et max

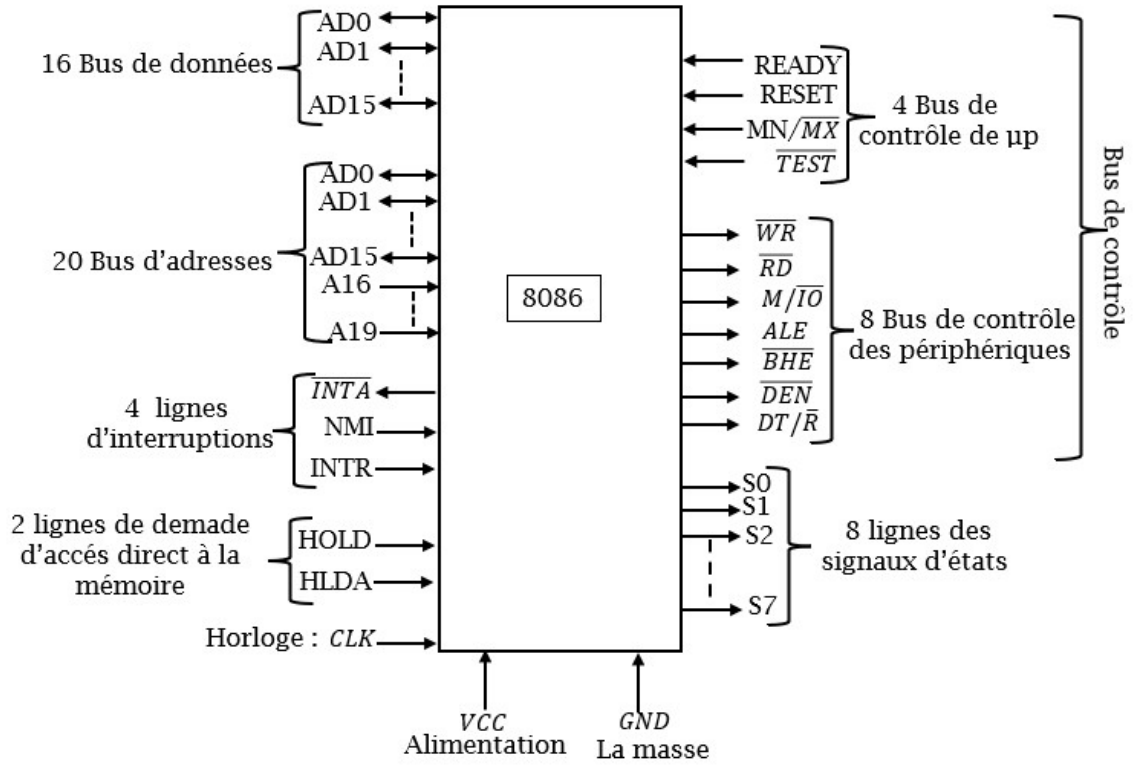


FIGURE 2 – Schéma fonctionnel du µp 8086.

I.2.1 Signal d'horloge : CLK

La broche CLK représente l'entrée du signal d'horloge qui cadence le fonctionnement du microprocesseur. Il est généré par le générateur d'horloge 8284 (ou 8084A). Ce générateur (Figure 3) a été développé par Intel pour générer le signal :

— d'horloge principale (CLK) à partir d'une source externe ou d'un cristal interne ;

— de prêt (Ready) qui synchronise le up avec les mémoires et les périphériques ;

— de réinitialisation (RESET) utilisé pour la réinitialisation du système ;

— d'horloge périphérique (PCLK) qui est un tiers de la fréquence de l'horloge principale.

Il est constitué d'un oscillateur à quartz contrôlé par un cristal, d'un compteur divisant la fréquence par trois et d'une logique de synchronisation pour le MULTIBUS.

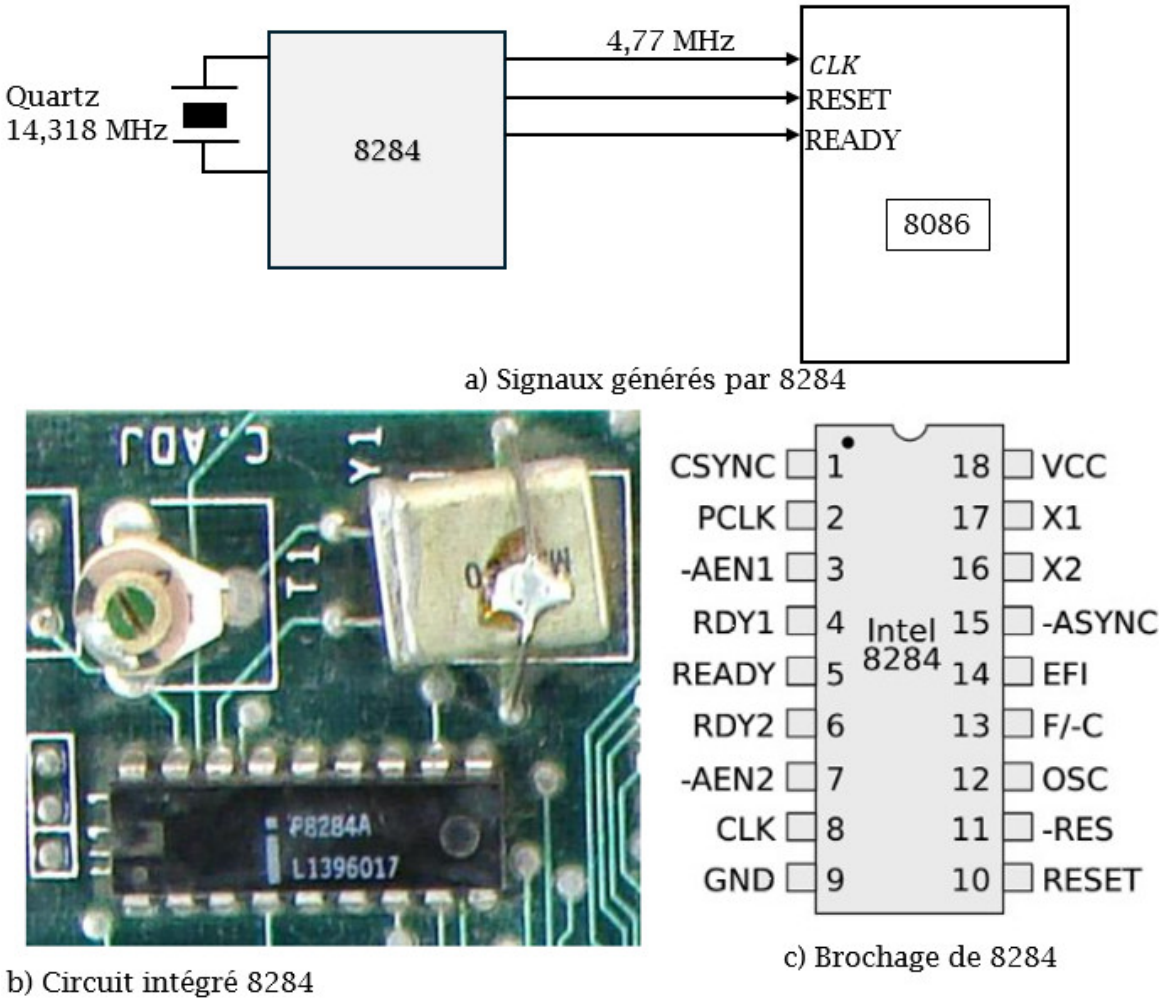


FIGURE 3 – Générateur d'horloge 8284.

I.2.2 Bus d'adresses

Le µp 8086 dispose d'un bus de données de 20 lignes (20 bits), il peut donc adresser 2^{20} = 1 Mo d'emplacement de mémoire. Ces lignes sont :

— A16/S3 à A19/S6 : 4 bits de poids fort du bus d'adresses (A16-A19), multiplexés avec 4 bits d'état (S3-S6),

— AD0 à AD15 : 16 bits de poids faible du bus d'adresses, multiplexés avec 16 bits de données. Ils contiennent les :

  — Adresses si le signal ALE (Address Latch Enable) est au niveau logique « 1 »

  — Données si le signal ALE est au niveau logique « 0 »

I.2.3 Bus de données

Le bus de données du µp 8086 contient 16 lignes, AD0 à AD15, multiplexés avec 16 bits de poids faible du bus d'adresses. Le bus A/D est multiplexé (multiplexage temporel) d'où la nécessité d'un démultiplexeur pour obtenir séparément les données et les adresses.

Le circuit de démultiplexage A/D est constitué d'un verrou (latch), c.à.d., un ensemble de bascules D comme le montre la figure 4.

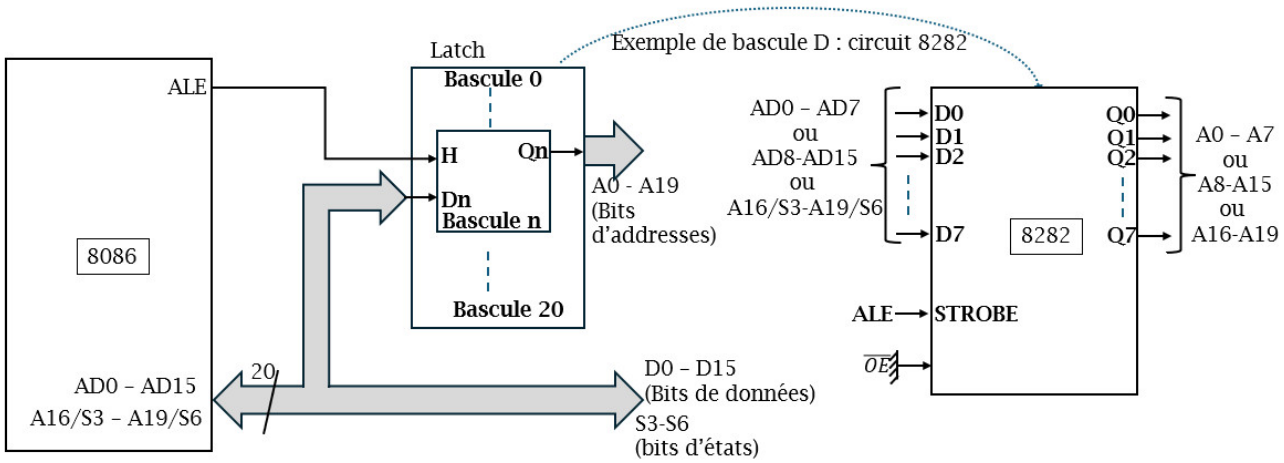


FIGURE 4 – Circuit de démultiplexage A/D.

Le démultiplexage des signaux AD0 à AD15 (ou A16/S3 à A19/S6) se fait en mémorisant l'adresse lorsque celle-ci est présente sur le bus A/D. La commande de mémorisation de l'adresse est générée par le microprocesseur : c'est le signal ALE :

— Si ALE = 1, le latch est transparent (Q = D) ;

— Si ALE = 0, mémorisation de la dernière valeur de D sur les sorties Q ;

— Le microprocesseur génère les signaux de lecture $\overline{RD}$ ou d'écriture $\overline{WR}$ seulement lorsque qu'une donnée est présentée sur le bus A/D. Le chronogramme du bus A/D est présenté sur la figure 5

I.2.4 Bus de contrôle

Les bus de contrôle peuvent être classés en bus de contrôle du microprocesseur et bus de contrôle des périphériques (figure 2).

I.2.4.1 Bus de contrôle du microprocesseur : Transporte des signaux de contrôle émis par les périphériques indiquant une tâche à effectuer par le microprocesseur. Ces signaux sont :

— **READY** : est le signal de synchronisation entre le µp et la mémoire. Il provient du générateur d'horloge et est utilisé par exemple par la mémoire pour signaler au µp qu'elle n'est pas prêt à transférer les données, forçant le µp à attendre (WAIT).

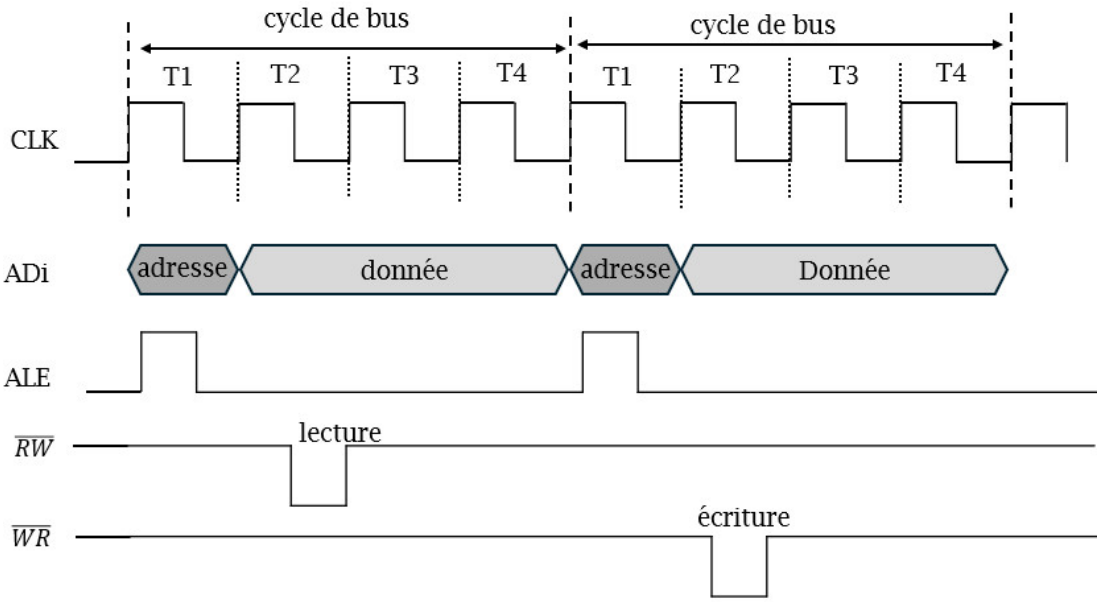


FIGURE 5 – Chronogramme de lecture et écriture sur le bus AD.

— **RESET** : est le signal de remise à zéro du microprocesseur. Il est fourni par le générateur d'horloge. Lorsqu'il est à l'état haut pendant au moins 4 périodes d'horloge, le microprocesseur est réinitialisé : il va exécuter l'instruction se trouvant à l'adresse FFFF0H (adresse de bootstrap).

— $\overline{\textbf{TEST}}$: est le signal de mis en attente du microprocesseur d'un événement extérieur.

— **MN/$\overline{\textbf{MX}}$** : est le signal de choix du mode de fonctionnement du microprocesseur :

  — **Mode minimum** (**MN/$\overline{\textbf{MX}}$ = 1**) : ce mode est activé dans le cas des systèmes à un seul µp. Dans ce cas, le µp 8086 fonctionne d'une façon autonome et génère lui-même les signaux de commande (comme $\overline{RD}$, $\overline{WR}$) ;

  — **Mode maximum** (**MN/$\overline{\textbf{MX}}$ = 0**) : ce mode est employé dans des systèmes à multiprocesseurs. Dans ce cas, les signaux de commande sont produits par un contrôleur de bus, comme le 8288.

I.2.4.2 Bus de contrôle des périphériques : Transporte des signaux de contrôle émis par le microprocesseur. Ces signaux incluent :

— $\overline{\textbf{WR}}$ (Write) : est le signal d'écriture de données en mémoire ;

— $\overline{\textbf{RD}}$ (Read) : est le signal de lecture de données en mémoire ;

— **M/$\overline{\textbf{IO}}$** (Memory/Input-Output) : ce signal permet d'indiquer si le µp s'adresse à la mémoire ou aux Entrées/Sorties :

  — Le µp s'adresse à la mémoire si $M/\overline{IO} = 1$;

  — Le µp s'adresse aux entrées/Sorties si $M/\overline{IO} = 0$.

— **ALE** (Adress latch enable) : ce signal indique le type de l'information (adresse ou donnée) qui circule dans le bus AD :

  — Le bus contient les adresses si $ALE = 1$ (on parle de verrouillage d'adresse) ;

  — Le bus contient des données si $ALE = 0$.

— $\overline{\textbf{DEN}}$ (Data Enable) : est le signal qui indique que les données sont disponibles sur le bus AD (équivalent d'ALE pour les données).

— **DT/$\overline{\mathbf{R}}$** (Data Transmit/Receive) : est le signal qui indique le sens de transfert des données :
  — si $DT/\overline{R} = 1$ alors les données sont émises par le microprocesseur (écriture) ;
  — si $DT/\overline{R} = 0$ alors les données sont reçues par le microprocesseur (lecture).

La figure 6 montre le chronogramme des signaux $\overline{DEN}$ et $DT/\overline{R}$. Ces deux signaux sont aussi employés pour commander le tampon de bus (buffers) utilisé pour amplifier le courant fourni par le microprocesseur sur le bus de données. En effet, si un nombre important de composants (généralement plus de 10) sont attachés aux broches de bus, le courant se divise plusieurs fois, alors l'ensemble du système 8086 doit être amplifié. Les signaux démultiplexés (bus d'adresses A0-A19 et $\overline{BHE}$) sont déjà amplifiés par les verrous (latch). Il reste donc les bus de données (D0-D15) à amplifier. Pour cela, on utilise des tampons (buffers) bidirectionnels. Parmi les tampons de bus existants, nous pouvons citer les circuits transmetteurs bidirectionnels 8286 (voir la figure 7) ou 74245.

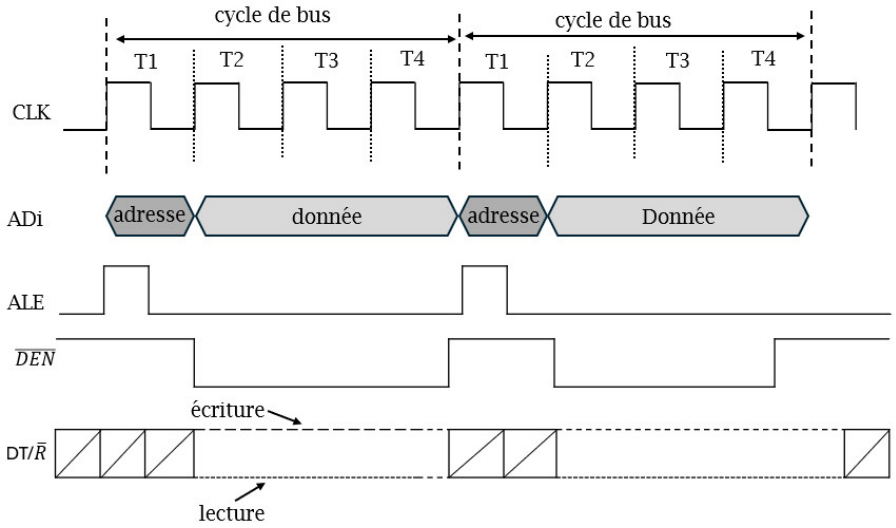


FIGURE 6 – Chronogramme des signaux $\overline{DEN}$ et $DT/\overline{R}$.

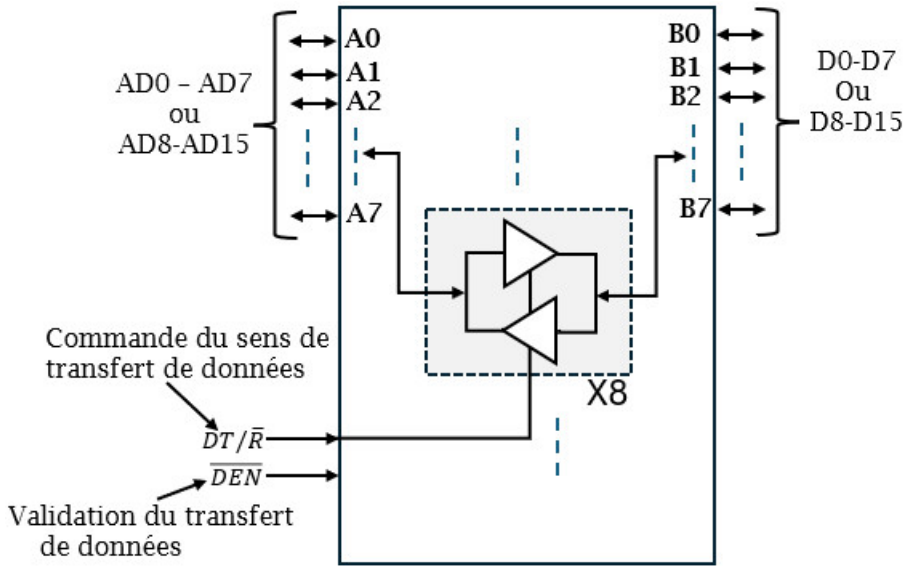


FIGURE 7 – Circuit transmetteur bidirectionnel 8286.

Remarque : les signaux de contrôle $\overline{RD}$, $\overline{WR}$ et $M/\overline{IO}$ sont amplifier à l'aide des tampons

unidirectionnels tels que 74LS244.

— **$\overline{\text{BHE}}$** (Bus High Enable) : ce signal indique le transfert de l'octet de poids fort de données (D8-D15) sur le bus AD ;

Remarque : On note que l'espace mémoire du 8086 est divisé en deux parties (banques) de 512 ko chacune (c.à.d., la capacité d'adressage de 1 Mo, 20 bits d'adresses) : la banque paire ou inférieure dont les adresses sont paires et la banque impaire ou supérieure dont les adresses sont impaires. Le µp 8086 utilise le signal $\overline{BHE}$ conjointement avec le bit d'adresse A_0 pour sélectionner l'une de ces deux banques (voir la figure 8)

— A0 pour la banque paire qui contient les octets de poids faible

— $\overline{BHE}$ pour la banque impaire qui contient les octets de poids fort.

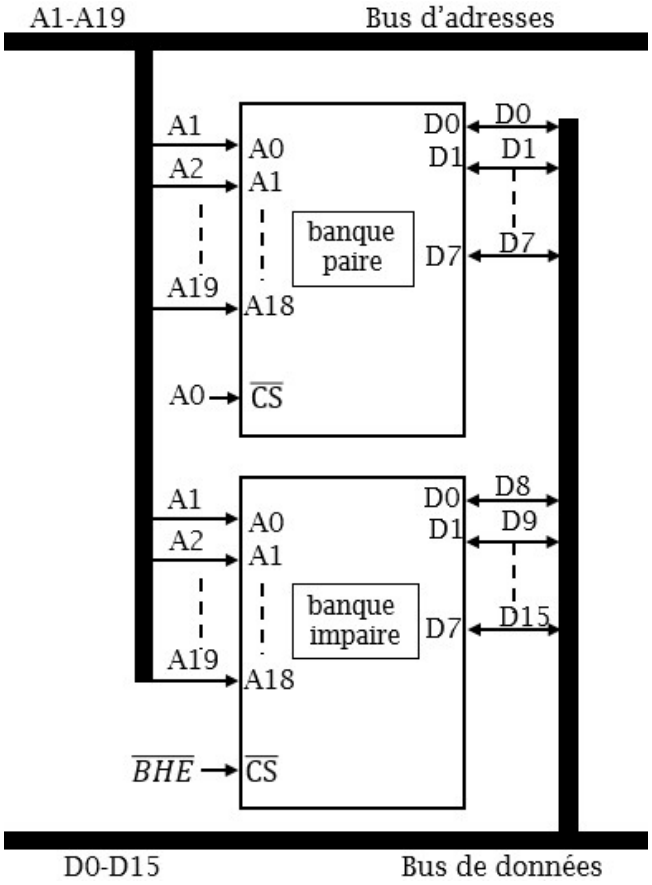


FIGURE 8 – Circuit de sélection des banques par le µp 8086.

Suivant les différentes combinaisons de $\overline{BHE}$ et $A0$, le microprocesseur peut lire et écrire les données sur 8 ou 16 bits (voir table II.1). Les autres bits A1 à A19 sont donc utilisés pour désigner une case mémoire des deux banques.

$\overline{BHE}$	$A0$	octets transférés
0	0	octets fort et faible (mot complet)
0	1	octet fort (adresse impaire)
1	0	octet faible (adresse paire)
1	1	aucun octet

TABLE II.1 – Octet transféré suivant les valeurs de $\overline{BHE}$ et $A0$

Remarque : Le tableau II.1 montre que le µp 8086 ne peut lire une donnée sur 16 bits en une seule fois que si l'octet de poids fort de cette donnée est rangé à une adresse impaire et l'octet de poids faible à une adresse paire (alignement sur les adresses paires), sinon la lecture de cette donnée doit se faire en deux opérations successives, ce qui conduit à une augmentation du temps du transfert due à un mauvais alignement de données.

En résumé le bus système de µp 8086 peut donc être développé comme le montre la figure 9.

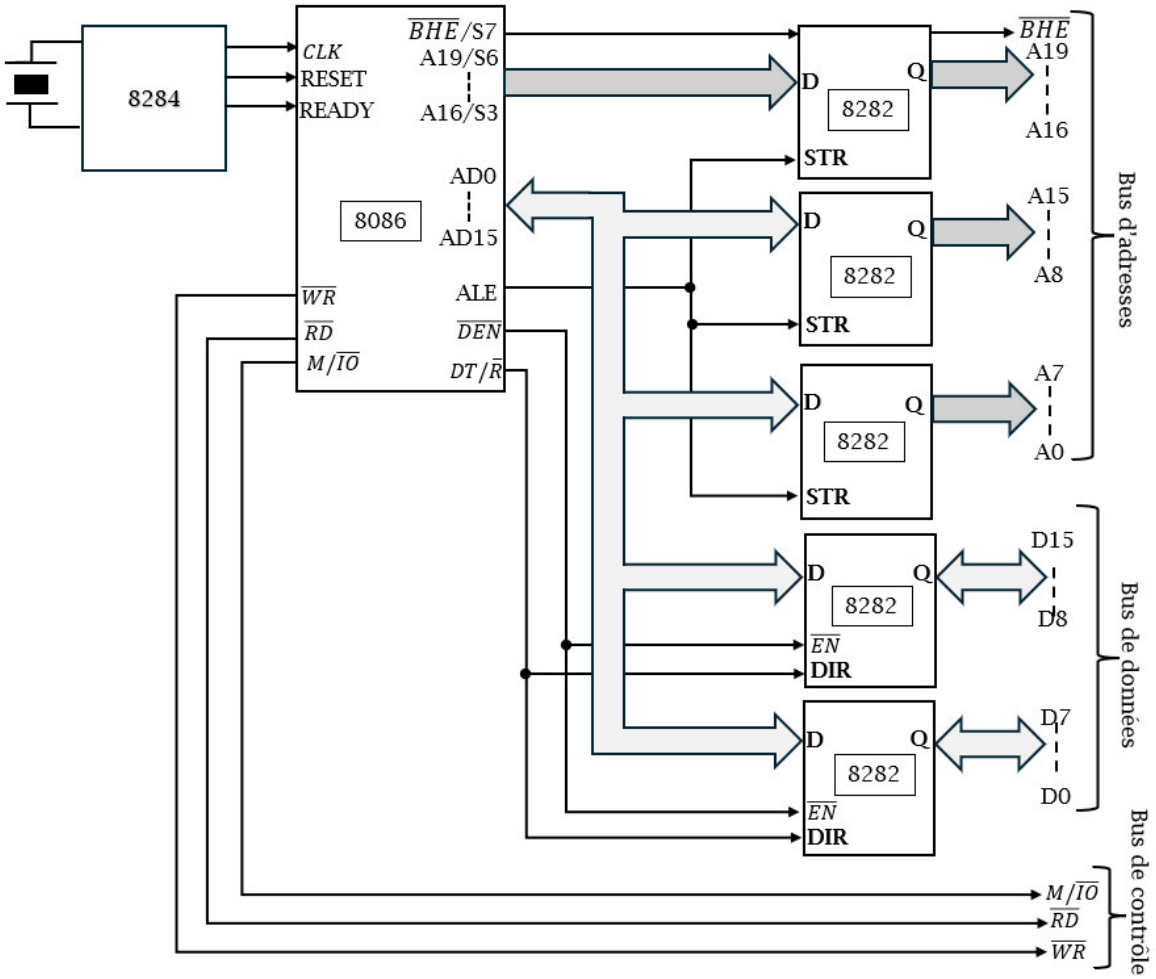


FIGURE 9 – Bus système de µp 8086.

I.2.5 Interruptions 8086

Les interruptions d'un µp sont des signaux qui suspendent le programme en cours pour exécuter un sous- programme, et elles peuvent être matérielles, logicielles ou générées par des exceptions. Les interruptions matérielles sont causées par des périphériques (comme les pins NMI et INTR), les interruptions logicielles sont déclenchées par l'instruction INT n, et les exceptions surviennent lors d'erreurs comme la division par zéro. Nous allons traiter ces interruptions en détail dans le chapitre 4. En bref, les interruptions de 8086 sont :

- **$\overline{\text{INTA}}$** (Interrupt Acknowledge) : est un signal qui indique que le microprocesseur accepte une interruption.
- **NMI** (Non-Maskable Interrup) : est une interruption non-masquable, cela signifie qu'il n'est pas possible de l'ignorer ou de l'arrêter.

— **INTR** (Interrupt Request or IRQ) : est un signal matériel qui indique au processeur de suspendre sa tâche en cours et de s'occuper d'un autre événement, tel qu'un mouvement de souris, une pression sur une touche, ou des données provenant d'une carte réseau.

I.2.6 Signaux de demande d'accès à la mémoire

Les signaux de demande d'accès à la mémoire sont de deux types :

— **HOLD** : est un signal de demande d'accès direct à la mémoire. En effet, lorsqu'un périphérique externe doit effectuer des transferts de données directement avec la mémoire sans passer par le microprocesseur, il active la ligne HOLD (en la plaçant à un niveau de tension haut). Cela demande au microprocesseur de suspendre son fonctionnement normal et de libérer le contrôle des bus.

— **HLDA** (Hold Acknowledgement ou en français : Accusé de réception de maintien) : est un signal de sortie qui indique que le microprocesseur a bien reçu une requête de maintien (signal HOLD) et est prêt à libérer le bus. Cela permet à un périphérique, comme un contrôleur de DMA (Accès direct à la mémoire), de prendre le contrôle du bus pour transférer des données.

I.2.7 Signaux d'états

Les signaux d'état (S0-S7) du microprocesseur 8086 indiquent l'état d'une opération de bus et peuvent être divisés en deux groupes :

— **S0, S1, et S2** : sont des signaux de contrôle qui indiquent le type de cycle de bus en cours (par exemple, lecture, écriture ou une autre action spécifique est en train de se dérouler).

— **S3-S7** : ces signaux, multiplexés avec les bits d'adresse de poids fort (A16-A19) et le signal BHE (Bus High Enable), sont utilisés pour fournir des informations supplémentaires sur le type d'opération.

I.2.8 Broches d'alimentation et de la masse

Les broches d'alimentation et de la masse du microprocesseur 8086 sont :

— **VCC** (Voltage Common Collector) : est la broche d'alimentation qui fournit la tension positive nécessaire au fonctionnement du microprocesseur. Elle est souvent fixée à une valeur spécifique par exemple +5 V.

— **GND** (Ground ou masse en français), est le point de référence de tension pour un microprocesseur. Il sert de chemin de retour commun pour le courant électrique et garantit que tous les composants du microprocesseur partagent le même potentiel de référence, souvent considéré comme zéro volt.

II Architecture interne du µp 8086

L'architecture interne de µp 8086 est constituée de deux unités indépendantes fonctionnant en parallèle (voir la figure 10) :

— EU (Execution Unit) ou l'unité d'exécution ;

— BIU (Bus Interface Unit) ou l'unité d'interface de bus.

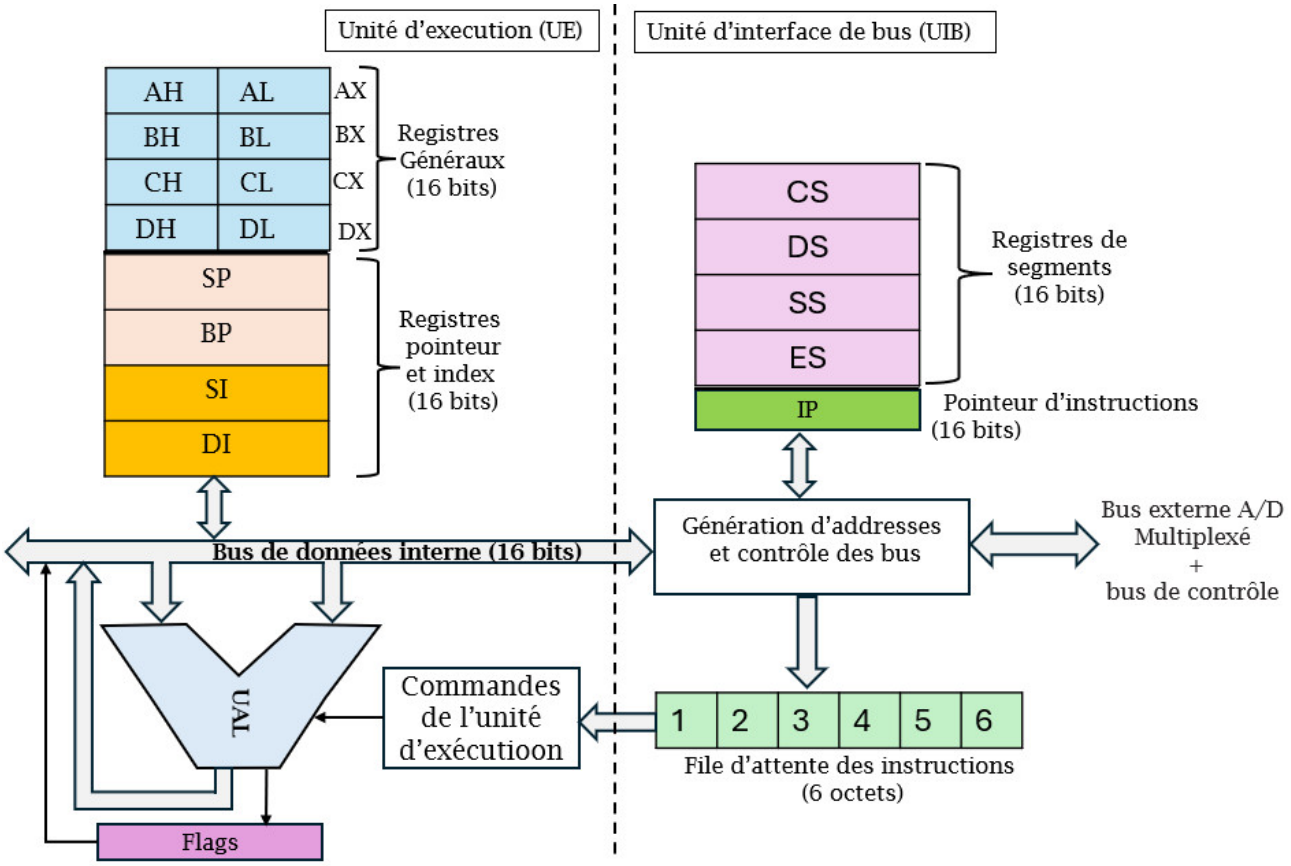


FIGURE 10 – Architecture interne de µp 8086.

Unité	Rôle
Unité d'interface de bus	Elle permet de : + Calculer les adresses physiques (adresses réelles) ; + Récupérer (Fetch en anglais) les instructions de la mémoire ; + Ranger les instructions dans une file d'attente ; + Établir les communications avec les périphériques du microprocesseur (Mémoires, Interfaces, d'E/S).
Unité d'exécution	Elle permet de : + Décoder les instructions contenues dans la file d'attente ; + Exécuter les instructions décodées.

TABLE II.2 – Fonctions des unités EU et BIU du 8086.

Le tableau II.2 présente brièvement le rôle de ces deux unités.

Remarque : La façon d'organisation interne du microprocesseur 8086 a pour objectif de réaliser un traitement en parallèle, d'où une accélération du processus d'exécution des instructions (fonctionnement selon le principe du pipeline). En effet, quand le microprocesseur exécute des instructions dans UE, en même temps, l'UIB récupère les instructions de la mémoire principale. Par conséquent, le µp prend en charge le pipeline en 2 étapes : Etape1 : Fetch , Etape2 : Exécute.

II.1 Unité d'interface de bus

Comme le montre la figure 10, les éléments principaux de l'unité d'interface de bus sont :

— Registres de segments ;

— Pointeur d'instructions ;

— Bloc logique de génération d'adresses et de contrôle de bus ;

— File d'attente d'instructions.

II.1.1 Registres de segments

Dans le contexte du microprocesseur, un segment est une partie de l'espace mémoire, constituant donc une plage d'adresses dans la mémoire principale.

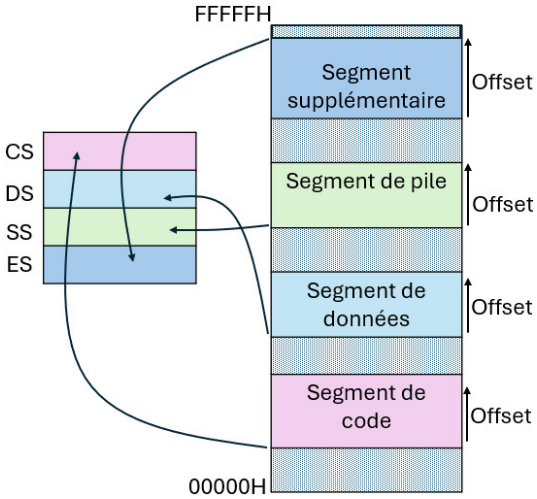
Le µp 8086 adresse une mémoire segmentée en quatre segments (divisée en quatre segments) : le *segment de code* inclut les instructions du programme, le *segment de données* inclut les données manipulées par le programme, le *segment de pile* inclut la pile de sauvegarde et le *segment supplémentaire* peut aussi inclure des données. Cette segmentation a permis au microprocesseur de ne pouvoir accéder en tout qu'à 4 segments de 64 Ko chacun dans sa plage de mémoire de 1 Mo. Les adresses de ces quatre segments sont stockées dans les registres de segments suivants :

— **CS** (Code Segment) : Contient l'adresse du segment du code ;

— **DS** (Data Segment) : Contient l'adresse du segment de données ;

— **SS** (Stack Segment) : Contient l'adresse du segment de la pile ;

— **ES** (Extra Segment) : Contient l'adresse du segment supplémentaire.



Ces registres sont des registres de 16 bits.

II.1.2 Pointeur d'instruction

Le pointeur d'instruction (en anglais IP : Instruction Pointer) est un registre de 16 bits contenant l'adresse de la prochaine instruction à exécuter. L'adresse physique totale est calculée en combinant le contenu du registre CS) et le contenu du pointeur IP.

II.1.3 Bloc logique de génération d'adresses et de contrôle des bus

Le rôle de ce bloc est la création des adresses pour accéder à la mémoire et à la gestion des signaux de contrôle pour coordonner les opérations des bus. On peut résumer leur principe de fonctionnement comme suit :

- *Sélection de l'adresse* : pour exécuter une instruction, le microprocesseur doit d'abord localiser les données correspondantes en mémoire. Pour cela, l'unité de génération d'adresses calcule ou récupère l'adresse de la donnée nécessaire. Le calcul de l'adresse se fait à l'aide d'un additionneur.
- *Envoie de l'adresse* : l'adresse sélectionnée est ensuite envoyée par le microprocesseur sur le bus d'adresses.

- *Accès aux données* : à l'aide de cette adresse le microprocesseur accède à l'emplacement exact (une "case") où se trouve une donnée.
- *Transfert des données* : une fois, l'adresse est envoyée, les données peuvent être transférées entre le microprocesseur et la mémoire via le bus de données.
- *Contrôle des opérations* : le bus de contrôle gère le timing et coordonne les opérations entre le microprocesseur et le reste du système en envoyant des signaux de contrôle. Il signale au microprocesseur l'opération à réaliser, par exemple lire des données, écrire des données ou effectuer une autre opération.

II.1.4 File d'attente d'instructions

La file d'attente des instructions de 6 octets permet de précharger l'instruction suivante dans la file pour améliorer les performances du microprocesseur. En effet, pour accélérer le processus de l'exécution, l'unité d'interface de bus du microprocesseur 8086 charge à l'avance les instructions suivantes dans cette file dont la taille est de 6 octets, ce qui correspond à la longueur maximale d'une instruction en langage machine pour cette architecture. Ce qui signifie que même l'instruction la plus longue peut être complètement chargée et stockée dans la file avant que le µp ne l'exécute, évitant ainsi des délais d'attente.

II.2 Unité d'exécution

Les composants principaux de cette unité sont :

— UAL (unité arithmétique et logique) ;

— Registres généraux ;

— Registre d'état ;

— Pointeurs de pile et index.

II.2.1 Unité arithmétique et logique

Comme son nom le suggère, cette unité peut effectuer deux types d'opérations :

- Les *opérations arithmétiques* telles que la sommation, la soustraction, la multiplication et la division ;
- Les *opérations logiques* à savoir OU, ET, OU _EXCLUSIF, de même les opérations de décalage et de rotation (arithmétique et logique).

La figure 11 montre le schéma général de l'unité arithmétique et logique (U.A.L), avec « n » est le nombre de bits. Pour le microprocesseur 8086, « n » peut être « 8 » ou « 16 ».

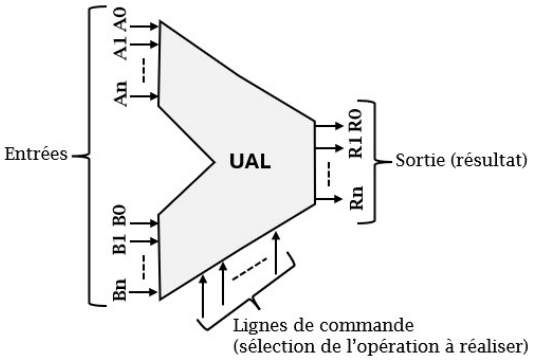


FIGURE 11 – Unité arithmétique et logique.

II.2.2 Registres généraux

Le µp 8086 contient 4 registres généraux sur 16 bits ou 8 registres sur 8 bits :

— **AX = (AH,AL)** (Accumulateur) ;

— **BX = (BH,BL)** (Base) ;

— **CX = (CH,CL)** (Compteur) ;

— **DX = (DH,DL)** (Données) .

Les alphabets X, H et L désignent respectivement mot (16 bits), octet du poids fort (8 bits) et octet du poids faible (8 bits).
Exemple : AX est le registre sur 16 bit (mot), AL est le registre d'octet de poids faible, AH est le registre d'octet de poids fort.

Ces registres sont employés pour stocker temporairement des données pendant l'exécution des programmes, pour les calculs et le stockage d'adresses, et pour la gestion du flux d'exécution des instructions. Malgré que ces registres sont généraux, ils peuvent être employés pour réaliser des tâches particulières en dehors des opérations arithmétiques et logiques classiques. En effet :

- AX, AH, AL : sont employés lors des opérations arithmétiques et logiques (accumulateurs) ;
- BX : est utilisé pour le calcul des adresses ;
- CX et CL : sont utilisés comme des compteurs de boucles ;
- DX : est utilisé pour stocker les adresses des ports d'entrée/sortie et utilisé également d'extension à AX, dans le cas des opérations arithmétiques (opération sur 32 bits).

II.2.3 Registres de pointeurs et d'index

Ces registres sont aussi connus sous le nom des registres d'adresse. Ils sont sur 16 bits et employés pour gérer les adresses mémoire, permettant au microprocesseur de localiser et d'accéder aux instructions et aux données en mémoire.

— *Pointeurs* : ils sont utilisés pour la gestion de la mémoire, en particulier de la pile. On distingue entre :
  — **SP** (Stack Pointer en anglais ou pointeur de pile) : contient l'adresse de l'élément supérieur de la pile, permettant de sauvegarder des données en cours d'exécution d'un programme ;
  — **BP** (Base Pointer en anglais ou pointeur de base en français) : utilisé pour accéder aux données dans la pile.

— *Index* : ces registres sont utilisés fréquemment lors des transferts de chaînes de caractères et de tableaux entre deux zones mémoire, car ils permettent de localiser respectivement la source et la destination des données à déplacer ou à traiter. On distingue entre :
  — **SI** (Source Index) : sert à pointer vers la source des données lors d'une opération, par exemple pour lire des données depuis une adresse mémoire.
  — **DI** (Destination Index) : sert à pointer vers la destination des données, c'est-à-dire l'adresse où les données doivent être écrites.

II.2.4 Registre d'état

Le *registre d'état* (aussi connu sous le nom de registre des drapeaux, ou flags en anglais) est un registre sur 16 bits qui indique l'état du microprocesseur suite à l'exécution d'une opération. Chacun des 16 bits (drapeaux) peut être mis à 0 ou 1 pour signaler une condition particulière. Ils sont utilisés par le microprocesseur pour prendre des décisions, comme dans le cas des sauts conditionnels et les boucles. Les drapeaux de ce registre peuvent être divisés en *drapeaux conditionnels* et *drapeaux de contrôle* (figure 12).

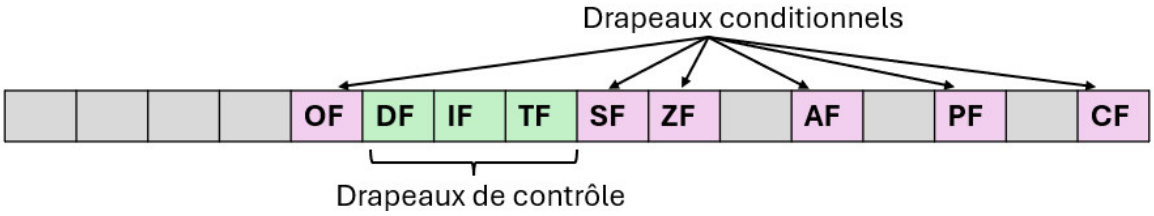
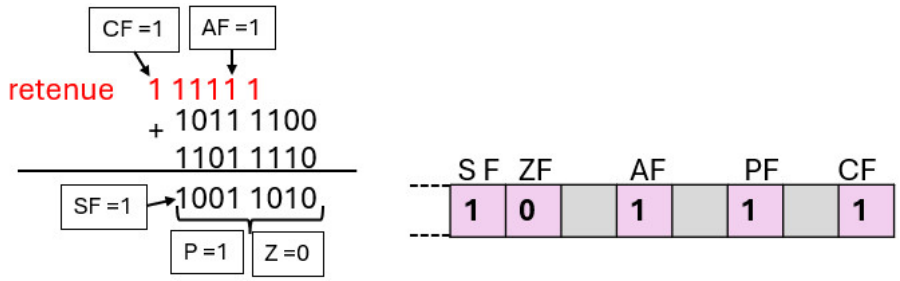


FIGURE 12 – Registre d'état.

- **Drapeaux conditionnels :** indiquent le résultat d'une opération arithmétique ou logique. Cette catégorie de drapeaux ainsi que leur rôle sont comme suit :
  - **CF** : c'est le bit de retenue (Carry en anglais). $CF = 1$, si une retenue est produite lors d'une addition des nombres non-signés ou si un emprunt est nécessaire lors d'une soustraction.
  - **PF** : c'est le bit de parité (Parity en anglais). $PF = 1$, si le résultat d'une opération contient un nombre pair de bits à 1 (parité paire), et à 0 si le nombre de bits à 1 est impair (parité impaire).
  - **AF** : représente le bit de la retenue auxiliaire (Auxillary Carry). $AC = 1$, lorsqu'une opération arithmétique génère une retenue ou un emprunt entre le demi-octet (nibble) de poids faible et le demi-octet de poids fort (les 4 bits de droite et les 4 bits de gauche). Cet indicateur est utilisé lors des opérations 8 bits.
  - **ZF** : c'est un indicateur de la nullité (Zero). $ZF = 1$, si le résultat obtenu est égal à 0.
  - **SF** : c'est un bit de signe (Sign). Ce drapeau est employé lors de manipulation des nombres signés. $SF = 1$, si le résultat est négatif (c.à.d., le bit le plus significatif est 1), sinon $SF = 0$.
  - **OF** : c'est le bit de débordement (Overflow). $OF = 1$, si le résultat d'une opération sur les nombres signés dépasse la capacité du registre (par exemple, si un grand nombre positif est additionné à un autre grand nombre positif, ce qui donne un résultat négatif).

Exemple : Quelles sont les valeurs des drapeaux conditionnels lors de l'addition de deux nombres suivants 10111100 et 11011110 ?



- **Drapeaux de contrôle :** sont utilisés pour contrôler le fonctionnement du microprocesseur. On distingue trois principaux drapeaux de contrôle :
  - **TF** : représente le bit de mode pas-à-pas (Trap en anglais). Si $TF = 1$, le microprocesseur passe en mode pas-à-pas, exécutant une instruction à la fois, puis déclenche une interruption. Il est très utile dans le débogage des programmes (c.à.d., le processus de

recherche et de correction des erreurs (« bugs ») dans un programme ou un matériel informatique).

— **IF** : ce bit indique si le microprocesseur va répondre aux interruptions ou non (Interrupt). Si $IF = 1$, le drapeau est activé, le processeur répond aux interruptions du périphérique d'E/S et les programmes en cours d'exécution seront interrompus momentanément ; sinon, il les ignore.

— **DF** : c'est un indicateur de direction (Direction). Il indique la direction de l'incrémentation ou de la décrémentation des pointeurs de chaîne lors des opérations de chaînes (comme dans le cas des instructions MOVSB ou LODSB). Si $DF = 0$, la chaîne sera lue dans l'ordre croissant. Si $DF = 1$, la chaîne sera lue dans l'ordre décroissant.

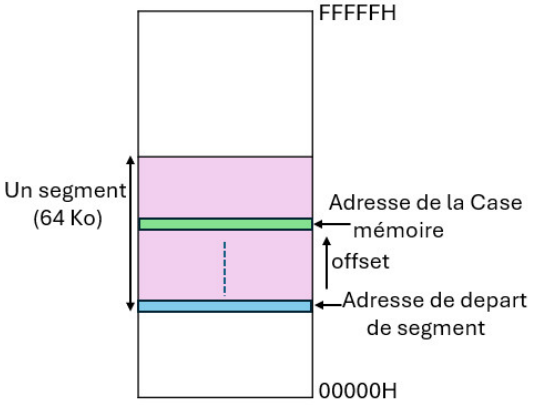
III Organisation de la mémoire par le 8086

Le microprocesseur dispose de 20 bits d'adresses, alors il peut donc adresser un espace mémoire de 1 Mo ($= 2^{20} = 1\,048\,576$ octets $= 1$ Mo). Cet espace est divisé en quatre segments de 64 Ko (65 536 octets) chacun. Le début de chaque segment est désigné par son adresse qui est forcément un multiple de 16 et codé sur 20 bits. On note que, dans une telle adresse, les 4 bits de poids faible sont à zéro. On peut donc définir l'adresse d'un segment uniquement avec ses 16 bits de poids fort, les 4 bits de poids faible étant implicitement à 0. Cela signifiait que n'importe quelle case mémoire parmi les $= 2^{16} = 65\,536$ cases contenus dans un segment peut être désigné par une adresse sur 16 bits.

Le microprocesseur 8086 localise une case mémoire dans un segment au moyen de son adresse qui peut être calculée à l'aide de :

— l'adresse de départ du segment ;

— et un *offset* (appelé aussi déplacement ou adresse effective) qui définit la position de cette case dans ce segment.



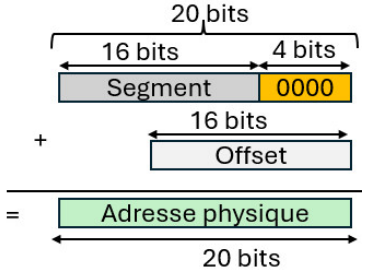
Remarque : La donnée d'un couple (segment,offset) est connue sous le nome de l'**adresse logique**, notée comme suit **segment : offset**. Par contre, l'adresse d'une case mémoire définie sur 20 bits (5 digits hexa) est appelée **adresse physique** (ou aussi effective ou réelle), car elle correspond à la valeur transmise réellement sur le bus d'adresses A0-A19.

Relation entre adresse physique et l'adresse logique

Pour calculer l'adresse physique à partir de l'adresse logique, le microprocesseur 8086 utilise la formule suivante

$$AdressePhysique = (Segment \times 16) + Offset \quad (1)$$

On a multiplié l'adresse de départ du segment par 16 pour obtenir une adresse sur 20 bits au lieu de 16 bits. En effet, la multiplication d'un nombre binaire par 16 (c.à.d., 10000b) permet de le décaler vers la gauche par quatre bits.

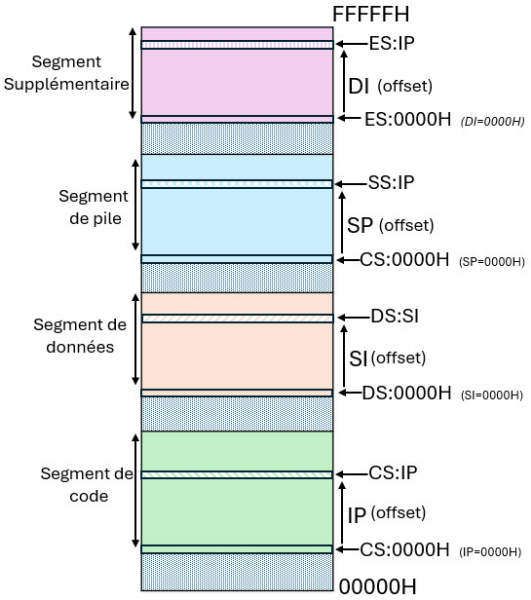


Exemple : accès à l'adresse d'une instruction

Comme nous avons vu, les programmes sont stockés dans le segment de code dont l'adresse se trouve dans le registre CS. Ainsi, le Pointeur d'instruction (IP) contient l'adresse de la prochaine instruction à exécuter par le µp. Alors, l'adresse logique et l'adresse physique peuvent être obtenues en combinant les contenus de registre CS et IP, ainsi la prochaine instruction à exécuter se trouve à :

+ l'adresse logique CS :IP ;
+ l'adresse physique $CS \times 16 + IP$.

De la même façon, les registres de segments DS, ES et SS peuvent être respectivement combinés avec les registres d'index (SI et DI) et pointeurs (SP) pour accéder aux segments mémoires DS, ES et SS. Ainsi, les adresses logiques sont *DS : SI, ES : DI, SS : SP.* La figure ci-contre résume la combinaison entre les différents registres pour accéder aux différentes zones mémoires.



Remarque : D'une manière générale, les segments ne sont pas obligatoirement distincts les uns des autres, ils peuvent se chevaucher partiellement ou complètement.

Après un RESET du microprocesseur, le contenu des registres est comme suit : $IP = 0000H$; $CS = FFFFH$; $DS = 0000H$; $ES = 0000H$; $SS = 0000H$, dans ce cas la première instruction à exécuter par le microprocesseur se trouve à l'adresse logique FFFFh :IP qui correspond à une adresse physique de FFFF0h. D'une manière générale, cette instruction est un saut vers sous-programme (programme principal) qui initialise ensuite les autres registres de segment.

Chapitre III

Programmation en assembleur du µp 8086

Dans le chapitre précédent, nous avons étudié l'architecture (interne/externe) et le fonctionnement du µp 8086. Dans ce chapitre, nous allons nous focaliser sur la programmation en assembleur de ce microprocesseur.

Le *langage assembleur* (ou langage d'assemblage) est un langage de programmation de bas niveau, directement traduisible en langage machine binaire compris par les microprocesseurs, et permet de communiquer directement avec le microprocesseur. Il utilise des mots simples (appelés *mnémoniques)* pour représenter des instructions machine. Contrairement aux langages de haut niveau (C, C^{++}, java, Python), il est beaucoup plus difficile à lire et à écrire, car il utilise directement des éléments tels que les registres du processeur et les adresses mémoire, sans structures de contrôle de haut niveau comme les boucles *"for"* ou les conditions *"if-else".* Il sert à écrire des programmes très performants ou à effectuer des tâches spécifiques nécessitant un contrôle précis du matériel, comme l'écriture de pilotes (programmation d'un logiciel qui facilite la communication entre un appareil et son système d'exploitation) ou la rétro-ingénierie (analyse d'un produit ou système existant pour comprendre sa conception, sa fabrication et son fonctionnement, généralement sans avoir accès à sa documentation d'origine).

Chaque microprocesseur est développé pour exécuter une série bien définie d'instructions appelée jeu d'instruction (Instruction Set). Ce jeu est déterminé par le constructeur et constitue le langage natif du processeur.

I Jeu d'instruction de µp 8086

Chaque instruction est définie par un *code opération (ou opcode)* codé en binaire. Comme il est difficile pour l'utilisateur de manipuler directement ces codes, chaque instruction (code) est associée à une *mnémonique*. Ainsi, un programme écrit sous forme de mnémoniques constitue un programme en langage assembleur. Le nombre d'instructions dépend de l'architecture du microprocesseur. Il varie entre 75 et 150 pour les microprocesseurs de type CISC et entre 10 et 30 pour les microprocesseurs de type RISC.

Les différentes instructions peuvent être classées en quatre catégories principales :

— Instructions de transfert de données (chargement, déplacement, copie, ...) ;

— Instructions arithmétiques (addition, soustraction, multiplication, ...) ;

— Instructions logiques (OU, ET, NON, décalages, ...) ;

— Instructions de branchement et de contrôle (boucles, sauts conditionnels, appels de sous-programmes).

I.1 Instructions de transfert de données et modes d'adressage

I.1.1 Instructions de transfert de données

Ces instructions permettent de transférer les données d'une source vers une destination. Le µp 8086 permet de faire les transferts suivants :

— registre vers mémoire ;

— mémoire vers registre ;

— registre vers registre.

Noter bien que ce µp n'autorise pas les transferts de mémoire vers mémoire. Mais on peut le faire en passant par un registre intérimaire.

Il existe plusieurs instructions de transfert telles que *MOV, XCHG, LAHF, SAHF,* etc.

• **MOV**.

Syntaxe : *MOV destination,source* ; l'instruction MOV est l'abréviation du verbe « to move » : permet de déplacer la donnée de la source vers la destination.

Exemples :

+ MOV R1,R2 ; déplacer le contenu du registre R2 dans le registre R1
+ MOV R,M ; déplacer le contenu d'une case mémoire dans un registre
+ MOV M,R ; déplacer le contenu d'un registre dans une case mémoire
+ MOV R,val ; charger le registre R avec la valeur val (constante)
+ MOV taille M,val ; copier la valeur val dans une case mémoire (taille = BYTE ou WORD)

•**XCHG**.

Syntaxe : *XCHG destination,source ;* cette instruction permet d'échanger l'opérande source avec l'opérande destination.

•**LAHF** (Load AH into Flags) ; cette instruction permet de transférer les 8 bits les plus à droite du registre d'état dans le registre AH.

•**SAHF** (Store AH into Flags) ; cette instruction fait l'opération inverse de LAHF, transférer le contenu registre AH dans l'octet de poids faible du registre d'état.

Exemple :

MOV AX, E910 ; (AX) =E901H ; (AL) =10H et (AH) =E9H
SAHF ; (AH) =E9H ; SF=1 ; ZF=1 ; AF=0 ; PF=0 ; CF=1

I.1.2 Modes d'adressage

Modes d'adressage définissent la façon dont le microprocesseur accède aux données, que ce soit dans les registres, la mémoire ou directement dans l'instruction. On distingue donc l'*adressage par registre*, a*adressage immédiat de registre*, *adressage direct*, a*adressage immédiat de la mémoire*, *adressage basé* et *adressage indexé*.

▷ Adressage par registre :

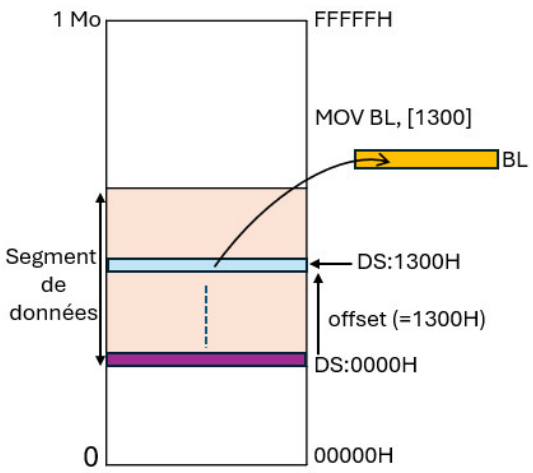
Dans ce mode, la source et la destination les deux sont des registres (opérandes), le transfert se fait de registre à registre :
Syntaxe : *MOV AX,BX* ; charge le contenu du registre BX dans le registre AX.

▷ Adressage immédiat de registre :

Dans ce mode, le premier opérande est un registre par contre le deuxième est une valeur constante, la donnée est fournie immédiatement avec l'instruction :
Syntaxe : *MOV AL,20H* ; charge le registre AL avec la valeur 20H.

▷ Adressage direct :

Ce mode d'adressage est similaire au précédent, mais cette fois-ci le deuxième opérande est l'adresse effective (offset) de la case mémoire où la donnée est stockée :
Syntaxe : *MOV BL,[1300H]* ; transfère le contenu de la case mémoire d'adresse effective 1300H vers le registre BL.

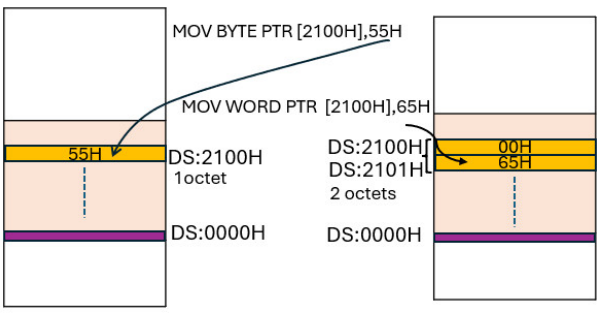


Remarque : dans ce mode, l'adresse effective représente l'offset de la case mémoire dans le segment de données par défaut (segment dont l'adresse est contenue dans le registre DS).

Dans le cas où la donnée se trouverait dans un autre segment autre que le segment de donnée, on ajoute un préfixe de segment, on parle de forçage de segment. Par exemple :
MOV BL,ES :[1500H]; transfère le contenu de la case mémoire de segment supplémentaire (ES) d'adresse effective 1500H vers le registre BL.

▷ Adressage immédiat de la mémoire :

Dans ce mode d'adressage, l'instruction inclut le mnémonique de l'opération, l'opérande 1 (adresse mémoire), l'opérande 2 (la donnée) et le format de la donnée : octet (8 bits) ou mot (16 bits). En effet, le microprocesseur 8086 peut manipuler des données sur 8 bits ou sur 16 bits.
Syntaxe

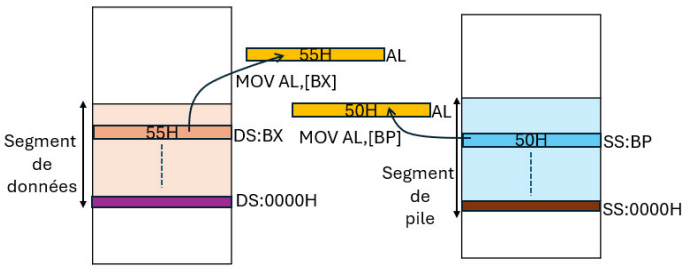


+ *MOV BYTE PTR [2100H], 55H* ; charger la donnée 55H (sur 1 octet) dans la case mémoire d'offset 2100H ;
+ *MOV WORD PTR [2100H], 65H* ; transfère la donnée 0065H (sur 2 octets) dans les cases mémoire d'offset 2100H et 2101H.

▷ Adressage basé

Comme son nom indique, ce mode utilise les registres de base. En effet, l'adresse effective de le second opérande se trouve dans un registre de base (BX ou BP) et le segment associé dépend du registre (DS pour BX, SS pour BP).

Exemples :



— *MOV AL, [BX]* ; charger la donnée dont l'offset est contenu dans le registre de base BX vers le registre AL. Remarquer que le segment associé par défaut au registre BX est le segment de données (DS). On dit alors que l'adressage est basé sur DS.

— *MOV AL, [BP]* ; transfère la donnée dont l'offset est contenu dans le registre de base BP vers le registre AL. De même remarquer que le segment par défaut associé au registre BP est le segment de pile (SS). Dans ce cas, l'adressage est basé sur SS.

▷ Adressage indexé

Comme son nom indique, ce mode d'adressage fait appel aux registres index, utilisé souvent pour parcourir des tableaux ou chaînes de caractères. L'offset est contenu dans un registre index (SI ou DI).

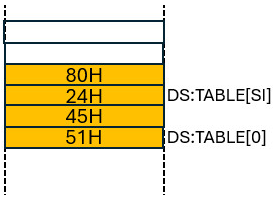
Exemple 1 :

— *MOV AL, [SI];* charge le registre AL avec la donnée de la case mémoire dont l'offset est contenu dans SI.

— *MOV [DI], BX;* charge les cases mémoires d'offset DI et DI + 1 avec le la donnée du registre BX, la donnée est sur 16 bits (2 octets).

Exemple 2 : Manipulation d'un tableau

```
MOV SI, 0                          ; Initialisation du registre d'index, SI joue le rôle d'indice du tableau
MOV WORD PTR TABLE[SI], 4551H      ; Déclaration du tableau, table représente l'offset du premier élément du tableau, charge le TABLE[0] et TABLE[1] par le mot 4551H
MOV SI, 2                          ; réinitialisation du registre d'index
MOV WORD PTR TABLE[SI], 8024H      ; charge le TABLE[2] et TABLE[3] par le mot 8024H ;
```



I.2 Instructions arithmétiques

Il existe quatre instructions arithmétiques de base : l'addition, la soustraction, la multiplication et la division ; qui incluent diverses va-

riantes. A l'aide de ces quatre instructions, plusieurs modes d'adressage sont envisageables.

▷ Addition

On distingue deux types d'instructions d'addition :

• **ADD** : Addition sans retenue

Syntaxe : *ADD opérande1, opérande2 ;* l'opération a effectuée est *opérande1*←*opérande1 + opérande2*

• **ADC** : Addition avec retenue

Syntaxe : *ADC opérande1, opérande2 ;* l'opération a effectuée est *opérande1*←*opérande1 + opérande2+CF*

Exemples :

— ADD AH, [2425H] ; ajoute le contenu de la case mémoire d'offset 2425H à l'accumulateur AH *(adressage direct).*

— ADD AH, [BX] ; ajoute le contenu de la case mémoire pointée par BX à l'accumulateur AH *(adressage basé).*

— ADD BYTE PTR [2521H], 45H ; ajoute la valeur 45H au contenu de la case mémoire d'offset 2521H *(adressage immédiat).*

— ADC AH, [2425H] ; ajoute le contenu de la case mémoire d'offset 2425H à l'accumulateur AH puis ajoute la retenue de cette dernière opération au nouveau contenu de AH l'accumulateur AH *(adressage direct).*

▷ Soustraction

On distingue deux types d'instructions de soustraction :

• **SUB** Soustraction sans retenue

Syntaxe : *SUB opérande1, opérande2* ; l'opération à effectuer est *opérande1*←*opérande1 - opérande2*

• **SBB** Soustraction avec retenue

Syntaxe : *SUB opérande1, opérande2* ; l'opération à effectuer est *opérande1*←*opérande1 - opérande2-CF*

Exemples :

— SUB AX,BX ; Soustraire le contenu de registre BX du contenu de registre AX

— SBB AX,BX ; Soustraire le contenu de registre BX du contenu de registre AX puis soustraire la retenue de cette dernière opération du nouveau contenu du registre AX.

▷ Multiplication

Cette instruction, notée **MUL**, effectue la multiplication du contenu de registre AL par un opérande sur 1 octet ou du contenu de registre AX par un opérande sur 2 octets. Le résultat est placé dans AX si les données à multiplier sont sur 1 octet (résultat sur 16 bits), dans (DX, AX) si elles sont sur 2 octets (résultat sur 32 bits).

Syntaxe : *MUL opérande* ; ici l'opérande est un registre ou une case mémoire.

— **Exemple 1 :** opération sur 1 octet, registre-registre
MOV AL,51
MOV BL,32
MUL BL ; AX ←51 × 32

— **Exemple 2 :** opération sur 2 octets, registre-registre
MOV AX,4253
MOV BX,1689
MUL BX ; (DX, AX) ←4253 × 1689

— **Exemple 3 :** opération sur 1 octet, registre-mémoire
MOV AL,43
MOV BYTE PTR [1200H],28
MUL BYTE PTR [1200H] ; AX ←43 × 28

— **Exemple 4 :** opération sur 2 octets, registre-mémoire
MOV AX,1234
MOV WORD PTR [1200H],5678
MUL WORD PTR [1200H] ; (DX, AX) ←1234 × 5678

▷ Division

Cette instruction, notée **DIV**, effectue la division du contenu de AX par un opérande sur 1 octet ou le contenu de (DX,AX) par un opérande sur 2 octets. Le résultat (quotient et le reste) est stocké comme suit :
Si l'opérande est sur 1 octet : Alors AL ←quotient et AH ←reste ;
Si l'opérande est sur 2 octets : alors AX ←quotient et DX ←reste

— **Exemple 1 : opération sur 1 octet**
MOV AX,35
MOV BL,10
DIV BL ; AL ←3 (quotient) ET AH ←5 (reste)

— **Exemple 2 : opération sur 2 octets**
MOV DX,0
MOV AX,1234
MOV BX,10
DIV BX ; AX ←123 (quotient) et DX ←4 (reste)

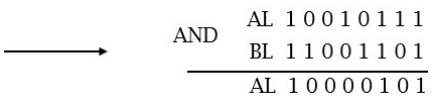
▷Autres instructions arithmétiques :

- INC : incrémentation d'une unité le contenu d'un opérande (Exemple INC AX) ;
- NEG : cette instruction (Négation) inverse le contenu d'un opérande (Exemple, NEG AX)
- DEC : décrémentation d'une unité le contenu d'un opérande (DEC AX) ;
- CMP Opérande1, Opérande2 ; soustrait les deux opérandes (Exemple : CMP AX, 5). Le résultat de la soustraction n'est pas sauvé, seuls les indicateurs sont modifiés (Flags modifiés : AF - CF - OF - PF - SF - ZF)
- IMUL : multiplication signée ;
- IDIV : division signée.

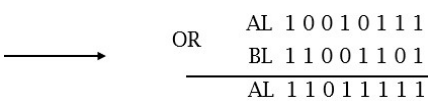
I.3 Instructions logiques

Les instructions logiques manipulent les données directement au niveau des bits. Les principales opérations logiques sont : ET (AND), OU (OR), XOR (OU_exclusif), complément à 1, complément à 2.

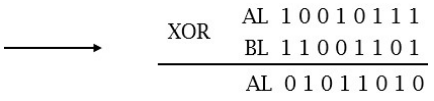
▷**AND** : Cette instruction effectue l'opération ET-logique entre deux opérandes.
Syntaxe : *AND opérande1, opérande2* ; opérande1←opérande1 ET opérande2 ;
Exemple :
MOV AL,10010111B ; AL ←10010111
MOV BL,11001101B ; BL ←110011011
AND AL, BL ; AL ←10000101



▷**OR** : Cette instruction, notée OR, effectue l'opération OU-logique entre deux opérandes.
Syntaxe : *OR opérande1, opérande2;* opérande1←opérande1 OU opérande2 ;
Exemple :
MOV AL,10010111B ; AL ←10010111
MOV BL,11001101B ; BL ←11001101
AND AL, BL ; AL ←1101111



▷**XOR** : Cette instruction effectue l'opération OU_EXCLUSIF entre deux opérandes.
Syntaxe : *XOR opérande1, opérande2;* opérande1←opérande1 $\oplus$ opérande2
Exemple :
MOV AL,10010111B ; AL ←10010111
MOV BL,11001101B ; BL ←11001101
XOR AL, BL ; AL ←01011010



▷ **NOT** : Cette instruction effectue le complément à 1 d'un opérande.
Syntaxe : *NOT opérande1* ; opérande1←$\overline{oprande1}$
Exemple :
MOV AL,10010111B ; AL ←10010111
NOT AL ; AL ←01101000

▷ **NEG** : Cette instruction effectue le complément à 2 d'un opérande.
Syntaxe : *NEG opérande1* ; opérande1←$\overline{oprande1}$+1
Exemple :
MOV AL,10010111B ; AL ←10010111
NEG AL ; AL ←01101001

I.4 Instructions de décalage et rotation

Ces instructions permettent de déplacer d'un certain nombre de positions les bits d'un opérande vers la gauche ou vers la droite.

I.4.1 Instructions de décalage

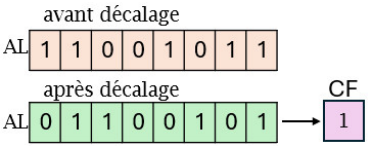
Dans le cas des instructions de décalage, les bits qui sont déplacés sont remplacés par des zéros et on distingue entre :

— Les décalages logiques utilisés pour des données non signées ;

— Les décalages arithmétiques utilisés pour des données signées.

▷**SHR (Shift Right)** : Cette instruction effectue un décalage logique vers la droite.
Syntaxe : *SHR opérande, n* ; Décale tous les bits vers la droite de n positions. Les bits qui sortent par la droite sont placés dans le drapeau de retenue (CF – Carry Flag), en revanche, les nouvelles positions à gauche sont remplies par des zéros ;

Exemple 1 : décalage par une position MOV AL, 11001011B ; AL ←11001011B
SHR AL,1 ; Décalage logique d'une position à droite : AL ←01100101 et CF ←1



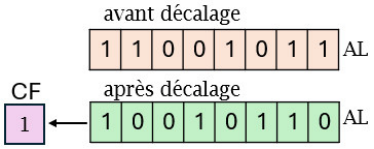
Remarques :

- Si le nombre n est supérieur à 1, ce nombre doit être chargé dans le registre CL (ou CX) avant l'exécution de l'instruction ;
- Dans le cas où $n > 1$ seul le dernier bit qui sort est conservé dans le drapeau CF.

Exemple 2 : décalage par 3 positions ($n = 3$)
MOV AL, 11001011B ; AL ←11001011B
MOV CL,3 ;
SHR AL,CL ; Décalage logique de 3 positions à droite : AL ←00011001 et CF ←0

▷**SHL (Shift Left)** : A l'inverse de SHR, cette instruction effectue un décalage logique vers la gauche.
Syntaxe : *SHL opérande, n* ; Décale tous les bits vers la gauche de n positions. Les bits qui sortent par la gauche sont placés dans le drapeau de retenue (CF – Carry Flag), en revanche, les nouvelles positions à droit sont remplies par des zéros ;

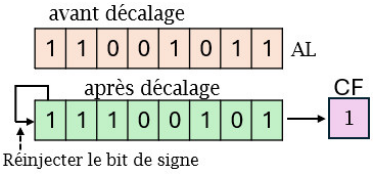
Exemple : décalage par une position MOV AL, 11001011B ; AL ←11001011B
SHL AL,1 ; Décalage logique d'une position à droite : AL ←10010110 et CF ←1



Remarque : Les mêmes remarques que précédemment s'appliquent quand $n > 1$.

▷**SAR (Shift Arithmetic Right)** : Cette instruction décale les bits d'un opérande vers la droite, tout en préservant le bit de signe (bit de poids fort) inchangé. A l'inverse de décalage logique (SHR), où les nouvelles positions à gauche sont remplies par des zéros, SAR reproduit le bit de signe, ce qui permet de garder la valeur signée de l'opérande.

Exemple 1 : décalage par une position
MOV AL, 11001011B ; AL ←11001011B
SAR AL,1 ; Décalage logique d'une position à droite : AL ←11100101 et CF ←1
Remarques : Les mêmes remarques que précédemment s'appliquent quand $n > 1$.



▷**SAL (Shift Arithmetic Left)** : Cette instruction est utilisée pour les nombres signés, mais elle a le même fonctionnement que SHL.

Exemples d'applications des instructions de décalage •**Exemple 1** : test l'état d'un bit dans un octet ou un mot

Supposons qu'on veut tester l'état du bit 5 du registre AL. Il y a deux possibilités pour le faire. En effet, on peut faire un décalage de 6 positions vers la droite ou 3 vers la gauche, le bit 5 de AL est transféré dans le drapeau de flag CF. Il suffit donc de tester ce drapeau.

```
MOV AL, 11001011B ;
MOV CL,6
SHR AL,CL
```

ou

```
MOV AL, 11001011B ;
MOV CL,3
SHL AL,CL
```

•**Exemple 2** : cadrage d'un ensemble de bits

Par exemple, pour cadrer à droite le digit de poids fort du registre AL, il suffit de faire un décalage à droite par quatre positions.

```
MOV AL,11001011B
MOV CL,4
SHR AL,Cl ; AL←0000 1100
```

•**Exemple 3** : Réaliser une multiplication ou division par 2

Un décalage à gauche revient à effectuer une multiplication par 2 et un décalage à droite, une division par 2.

```
MOV AL,16
MOV CL,2
SHR AL,CL ; AL ←16/2^2 = 4
```

ou

```
MOV AL,16
MOV CL,2
SHL AL,CL ; AL ←16 × 2^2 = 64
```

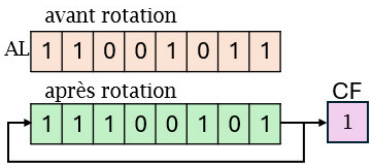
I.4.2 Instructions de rotations

▷ **ROR (Rotate Right)** : Cette instruction réalise une rotation des bits d'un opérande vers la droite. Contrairement aux instructions de décalage, aucun bit n'est perdu : le bit qui sort à droite est réinjecté du côté gauche. Seul le dernier bit déplacé est copié dans le drapeau de retenue CF.

Syntaxe : *ROR opérande, n ;* décale l'opérande de n positions vers la droite et réinsère par la gauche les bits sortants.

Exemple : décalage par une position

```
MOV AL, 11001011B ; AL ←11001011B
ROR AL,1 ; Rotation d'une position à droite : AL ←11100101 et CF ←1
```

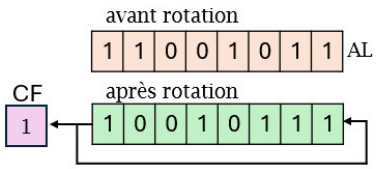


▷ **ROL (Rotate Left)** : Cette instruction réalise une rotation des bits d'un opérande vers la gauche. Le bit qui sort à gauche est réinjecté du côté droit. Seul le dernier bit déplacé est copié dans le drapeau de retenue CF.

Syntaxe : *ROL opérande, n* ; décale l'opérande de n positions vers la gauche et réinsère par la droite les bits sortant.

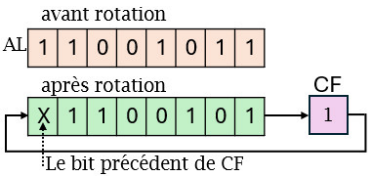
Exemple : décalage par une position

```
MOV AL, 11001011B ; AL ←11001011B
ROL AL,1 ; Rotation d'une position à gauche : AL ←10010111 et CF ←1
```



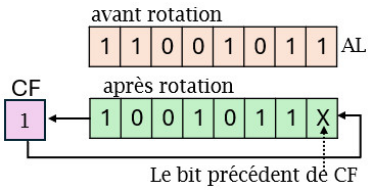
▷ **RCR (Rotate Right through Carry)** : Cette instruction décale l'opérande de n positions vers la droite en passant par drapeau de retenue CF. Le bit sortant par la droite est copié dans le drapeau de retenue CF et la valeur précédente de CF est réinséré par la gauche. **Syntaxe :** *RCR opérande, n ;*

Exemple : décalage par une position
MOV AL, 11001011B ; AL ←11001011B
RCR AL,1 ; Rotation d'une position à droite : AL ←X1100101 et CF ←1



▷ **RCL (Rotate Left through Carry)** : Cette instruction décale l'opérande de n positions vers la gauche en passant par drapeau de retenue CF. Le bit sortant par la gauche est copié dans le drapeau de retenue CF et la valeur précédente de CF est réinséré par la droite.
Syntaxe : *RCL opérande, n ;*

Exemple : décalage par une position
MOV AL, 11001011B ; AL ←11001011B
RCL AL,1 ; Rotation d'une position à gauche : AL ←1001011X et CF ←1



I.5 Instructions de branchement

Les instructions de branchement, appelées aussi de saut, modifient l'ordre d'exécution normal d'un programme, en forçant le processeur de passer à une autre adresse mémoire, plutôt que d'exécuter l'instruction suivante séquentiellement. Elles sont utilisées pour prendre des décisions, répéter des séquences d'instructions (boucles) ou exécuter des sous-programmes. On distingue généralement trois catégories de branchements :

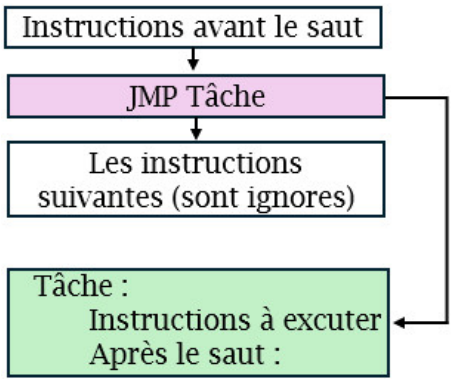
— Branchement inconditionnel ;

— Branchements conditionnels ;

— Appels de sous-programmes.

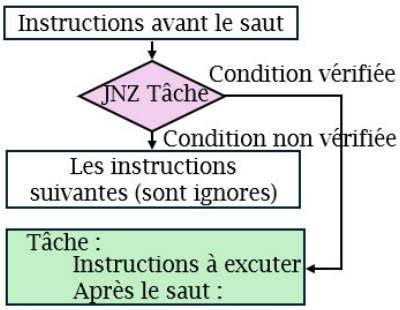
I.5.1 Instruction de branchement inconditionnel

Instruction de branchement inconditionnel, notée **JMP** (jump) est une instruction qui force le microprocesseur à sauter à une autre ligne de programme (une autre adresse mémoire ou un "label") sans vérifier aucune condition.
Syntaxe *JMP label* ; effectuer un saut vers le label (voir la figure ci-contre). Un label (ou étiquette) est une représentation symbolique d'une instruction en mémoire.



Remarque : l'instruction JMP ajoute au contenu du registre IP (pointeur d'instruction) le nombre d'octets qui définit la distance entre l'instruction de saut et sa destination. Cette distance est négative (codée en complément à 2) dans le cas des sauts en arrière.

I.5.2 Instructions de branchements conditionnels



Ces instructions ne s'exécutent que si certaines conditions sont vérifiées, sinon l'exécution se poursuit séquentiellement à l'instruction suivante. Contrairement au langage de haut niveau, dans le langage assembleur, il n'existe pas de tests qui permettent d'évaluer des conditions, comme == (égal), != (différent), < (inférieur), > (supérieur), <=, >=. Cependant, il est toutefois bien sûr possible de réaliser des tests. Pour cela, on utilise les bits du registre flags comme condition de test et une instruction de branchement conditionnelle (saut si certains bits du registre FLAGS valent 0 ou 1) pour déclencher la partie *alors* ou la partie *sinon* du test. Les drapeaux de flags sont positionnés en fonction du résultat de la dernière opération.

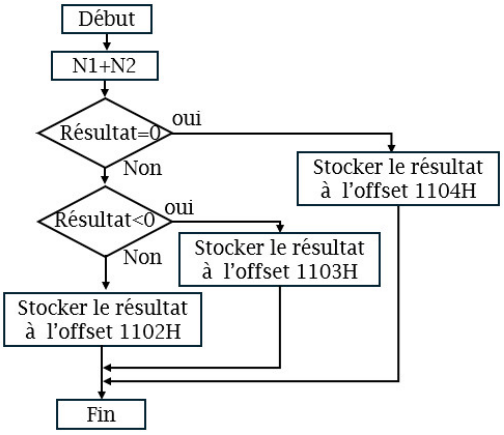
Syntaxe *Jcondition label* ; effectuer un saut vers le label si certaines conditions sont vérifiées (voir le tableau III.1).

Syntaxe	Nom	Condition
JZ label	Jump if Zero	saut si ZF = 1
JNZ label	Jump if Not Zero	saut si ZF = 0
JE label	Jump if Equal	Saut si égal, $ZF = 1$
JNE label	Jump if Not Equal	Saut si non égal, $ZF = 0$
JS label	Jump if Sign	Saut si signe, $SF = 1$
JNS label	Jump if Not Sign	Saut si pas de signe, $SF = 0$
JO label	Jump if Overflow	Saut si débordement, $OF = 1$
JNO label	Jump if Not Overflow	Saut si pas de débordement, $O = 0$
JP label	Jump if Parity	Saut si parité, $PF = 1$
JNP label	Jump if Not Parity	Saut si pas de parité, $P = 0$
JC label	Jump if Carry	Saut si retenue, $CF = 1$
JNC label	Jump if Not Carry	Saut si pas de retenue, $CF = 0$

TABLE III.1 – Instructions de branchements conditionnels

Exemple d'application : Donner l'organigramme et le code en assembleur qui permet de faire la somme de deux nombres signés N1 et N2 stockés respectivement aux offsets 1100H et 1101H et de rangé le résultat à l'offset 1102H s'il est positif, à l'offset 1103H s'il est négatif et à l'offset 1104H s'il est nul.

Organigramme



Programme

```
MOV AL,[1100H]
ADD AL,[1101H]
JZ nul
JS négatif
MOV [1102H],AL
JMP fin
nul : MOV [1104H],AL
JMP fin
negatif : MOV [1103H],AL
fin : HLT
```

Remarque Il existe d'autres instructions de saut conditionnel, appelées instructions des sauts

arithmétiques (voir le tableau III.2) . Ces instructions sont généralement associées à l'instruction de comparaison (CMP) : *CMP opérande 1, opérande2.*

Exemple :
MOV AL, 23
CMP AL, 34
à l'issue de cette dernière instruction :

- le registre AL reste inchangé, contient toujours la valeur 23 ;
- l'indicateur de retenue est positionné à 1 (CF=1), car le deuxième opérande (34) est supérieur au premier (23) ;
- l'indicateur ZF est positionné à 0, car les deux opérandes sont différents ;
- le bit de signe S est positionné à 1, car la soustraction implicite, 23-34, donne un nombre négatif.

Nombre non-signé			Nombre signé		
Syntaxe	Nom	Condition	Syntaxe	Nom	Condition
JE label	Jump if Equal	Saut si égal, $ZF = 1$	JE label	Jump if Equal	Saut si égal, $ZF = 1$
JA label	Jump if Above	Saut si supérieur, $CF = 0$ et $ZF = 0$	JG label	Jump if Greater	Saut si supérieur, $SF = OF$ et $ZF = 0$
JB label	Jump if Below	Saut si inférieur, $CF = 1$	JL label	Jump if Less	Saut si inférieur, $SF \neq OF$
JBE label	Jump if Below or Equal	Saut si inférieur ou égal, $CF = 1$ et $ZF = 1$	JLE label	Jump if Lower or Equal	Saut si inférieur ou égal, $SF \neq OF$
JAE label	Jump if Above or Equal	Saut si supérieur ou égal, $CF = 0$	JGE label	Jump if Greater or Equal	Saut si supérieur ou égal, $SF = OF$
JNE label	Jump if Not Equal	Saut si non égal, $ZF = 0$	JNE label	Jump if Not Equal	Saut si non égal, $ZF = 0$

TABLE III.2 – Instructions de branchements conditionnels

I.5.3 Appel de sous-programmes

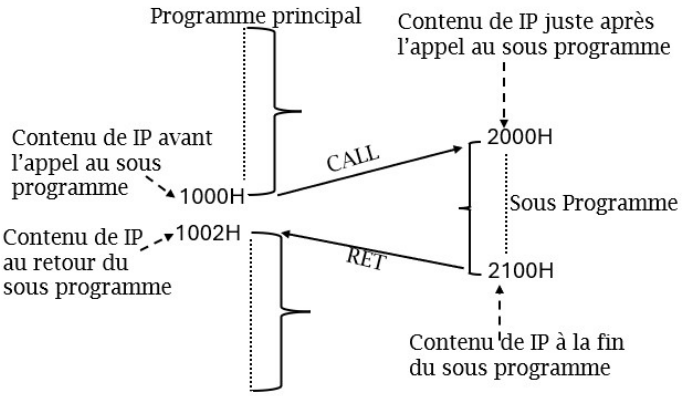
Un sous-programme est une séquence d'instructions réutilisable, aussi appelé fonction ou procédure, qui réalise une tâche spécifique, permettant d'éviter la répétition d'une même séquence d'instructions à plusieurs endroits d'un programme, de diviser un programme complexe en blocs plus petits et organisés, et d'améliorer la lisibilité, on écrit cette séquence une seule fois et on lui attribue un nom unique (identificateur). Quand nous en avons besoin, nous l'appelons dans le programme principal avec l'instruction *CALL*, à la fin de son exécution, l'instruction *RET* permet de revenir au programme principal.

Syntaxe : Écriture d'un sous-programme

```
non_sp  PROC
          ⋮}      ; instructions du sous-programme
non_sp  RET       ; instruction de retour au programme principal
```

Syntaxe : Appel d'un sous-programme

```
          ⋮}      ; instructions précédant l'appel au sous programme
CALL  non_sp      ; instruction de l'appel au sous programme
          ⋮}      ; instructions à exécutées après le retour au programme principal
```



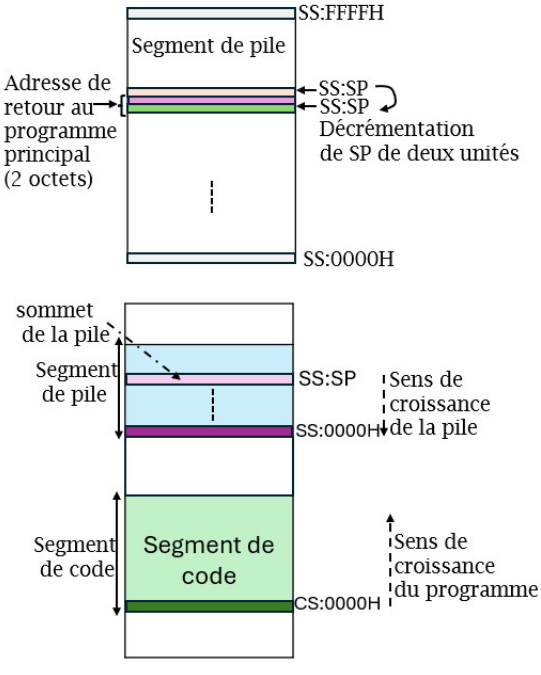
Une fois, l'instruction *CALL non_sp* est rencontrée durant l'exécution d'un programme principal, le pointeur d'instruction (IP) est chargé avec l'adresse de la première instruction du sous-programme *non_sp*. Avant de charger IP avec l'adresse du sous-programme, le contenu du registre IP, qui correspond à l'adresse de retour au programme principal (adresse de l'instruction située juste après le CALL), est rangée dans une zone mémoire particulière appelée pile. Cette adresse est récupérée à partir de la pile et rechargée dans IP lors de l'exécution de l'instruction RET, ce qui permet de poursuivre l'exécution normale du programme principal.

La pile fonctionnant en mode LIFO (Last In, First Out : dernier entré, premier sorti). On distingue deux opérations

— Empiler une donnée (push) : placer une donnée au sommet de la pile ;

— Dépiler une donnée (pop) : retirer la donnée se trouvant au sommet de la pile.

Le sommet de la pile est repéré par le pointeur de pile SP (Stack Pointer), qui contient l'adresse (l'offset) de la dernière donnée empilée. La pile est définie dans le segment de pile, dont l'adresse de départ est contenue dans le registre SS (Stack Segment). Une fois, la donnée est empilée, le pointeur de pile SP est automatiquement *décrémenté*. Au retour du sous-programme, le pointeur d'instruction IP est rechargé avec la valeur contenue sommet de la pile et SP est *incrémenté*.



Remarques :

- Le programme et la pile croissent en sens inverse, dont le but est d'éviter le risque de collision entre le code et la pile dans le cas où celle-ci est placée dans le même segment que le code (SS = CS).
- La pile peut également servir à sauvegarder le contenu de registres qui ne sont pas automati-

quement sauvegardés lors de l'appel à un sous-
programme. Pour cela, on utilise les instructions suivantes :

— PUSH Opérande ; placer l'opérande au sommet de la pile ;

— POP Opérande ; retirer la donnée se trouvant au sommet de la pile.

Où opérande est un registre ou une donnée sur 16 bits (on ne peut empiler que des données sur 2 octets).

• **Exemple 1 :** Instruction push-pop

```
PUSH AX ; empilage du registre AX
PUSH BX ; empilage du registre BX
PUSH [1100H] ;empilage du contenu des cases mémoires 1100H–1101H
POP [1100H] ; dépilage dans l'ordre inverse de l'empilage
POP BX
POP AX
```

• **Exemple 2 :** Utilisation de la pile pour le transfert de données

La pile peut être aussi utilisée pour transmettre les données à une procédure (sous-programme qui fait un calcul) ; les données utilisées par la procédure sont placées sur la pile avant l'appel de la procédure, puis celle-ci les récupère en effectuant un adressage basé sur la pile en utilisant le registre BP. Par exemple, soit une procédure effectuant la somme de deux nombres et retournant le résultat dans le registre AX :

Programme principal :

```
            MOV AX,200
            PUSH AX ; empilage du première donnée
            MOV AX,300
            PUSH AX ; empilage du deuxième donnée
            CALL somme ; appel de la procédure somme
```

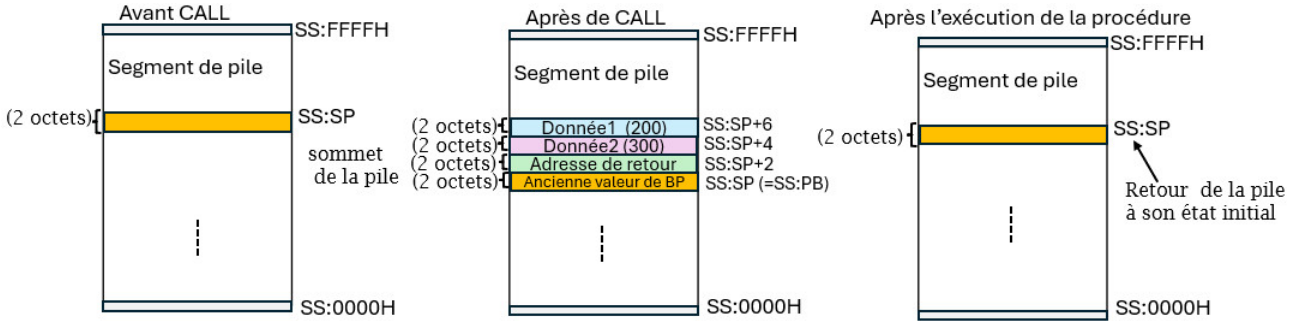
Procédure somme :

```
somme       PROC
            PUSH BP ; sauvegarde de BP
            MOV BP,SP ; faire pointer BP sur le sommet de la pile
            MOV AX,[BP+4] ; récupération du deuxième donnée
            ADD AX,[BP+6] ; addition au première donnée
            POP BP ; restauration de l'ancienne valeur de BP
            RET 4 ; retour et dépilage des données
somme       ENDP
```

Le rôle de l'instruction RET 4 est de retourner au programme principal et d'incrémenter le pointeur de pile de 4 unités pour dépiler les données afin de remettre la pile dans son état initial. La figure suivante montre l'état de la pile avant et après la procédure Somme.

II Directives de langage assembleur

Les directives sont utilisées dans les programmes assembleur pour définir des constantes, contrôler le flux du programme et gérer la mémoire (allouer de la mémoire pour des variables). Par opposition aux instructions (comme MOV, ADD) qui sont exécutées par le microprocesseur, les directives ne sont pas exécutées par le microprocesseur, mais elles aident à structurer le code et à gérer les ressources avant l'exécution du programme. Parmi les directives plus utilisées, nous citons :



▷ **Identificateurs** : Les identificateurs sont utilisés pour repérer une donnée, une ligne dans un programme (Label ou étiquette), un sous-programme, une case mémoire, etc.
Remarques : Les identificateurs :

- peuvent être des lettres majuscules ou minuscules, des chiffres, et quelques caractères spéciaux (@, ?, _,), , mais jamais d'espaces ;
- ne doivent pas commencer par un chiffre ;
- ne doivent pas dépasser 80 caractères.

Exemples : Table_1 ; AFF1 ; Somme ;...

▷**Constantes numériques** : Dans un programme assembleur, les constantes peuvent être représentées en binaire (ex. 01001010B), décimal (200 ou 200D), hexadécimal (45H, n'oublier pas de mettre un 0 devant une lettre ex 0F3H) et octal (22O or 22Q). En plus, une constante peut être identifiée par un nom à l'aide de la directive **EQU** (EQUate en anglais). Par exemples : VAR1 EQU 0A1H ; SOM1 EQU 00H.

▷ **Chaines de caractères** : Une chaîne de caractères est une séquence ordonnée de caractères, comme des lettres, des chiffres et des symboles. En assembleur, elle est définie entre apostrophes. Durant l'exécution, chaque caractère sera traduit en code ASCII (un octet par caractère).
Exemples : 'A', 'AGA 51', 'ABDE'

▷ **Variables** : On distingue différents types de variables selon les directives suivantes :

- DB (Define byte) : la variable est codée sur un octet, appelée de TYPE 1 ;
- DW (Define word) : la variable est codée sur deux octets (1 mot), appelée de TYPE 2 ;
- DD (Define double) : la variable est codée sur quatre octets (2 mots), appelée de TYPE 4 ;

Une variable se caractérise par une longueur (length) et une dimension (size), avec la longueur c'est le nombre de termes de la variable (octets, mots, double mots), par contre la dimension représente le nombre d'octets (SIZE = LENGTH × TYPE).
Exemples :
- CHIFFRES DB 0,1,2,3,4,5,6,7,8 : a une longueur 9 et une dimension 9
- YEARS DW 2000, 2010, 2025, 2026 : a une longueur 4 et une dimension 8
- MSG DB 'Bonjour ASM 8086' : a une longueur 14 et une dimension 14

▷ **Segmentation** : La définition d'un segment, zone mémoire, commence par son nom, un identificateur suivi de SEGMENT, et termine par ENDS précédée du nom du segment.

Exemples :

```
CODE SEGMENT                TABLE SEGMENT
⋮                           ⋮
CODE ENDS                   TABLE ENDS
```

▷ **ORG** : La directive ORG (ORiGin) sert à définir l'adresse de départ d'un segment de code ou de données en mémoire.

Synaxe : ORG offset ; Origine du programme en mémoire

Exemples :

```
ORG 1000H ; le code commence à l'adresse 1000H
MOV AX, 0 ; Instruction commençant à l'adresse 1000H
```

▷ **ASSUME** : La directive ASSUME permet d'indiquer quel segment mémoire (Code, Données, Pile) doit être associé à quels registres de segment (CS, DS, SS, ES) pour le reste du programme.

Syntaxe : ASSUME CS :CODE, DS :DATA, SS :DATA ; Associe les segments aux registres

▷ **PTR** : La directive PTR (pointeur) permet de spécifier la taille (en octets) d'un opérande lorsqu'elle n'est pas évidente ou qu'elle doit être forcée.

Syntaxe : type PTR [adresse] ou type PTR [registre indexé]

Exemples :

```
MOV AL, BYTE PTR [BX] ; Lit 1 octet à l'adresse pointée par BX
MOV AX, WORD PTR [BX] ; Lit 2 octets (un mot) à l'adresse pointée par BX
MOV AX, WORD PTR VAR1 ; Charge la valeur de VAR1 dans AX
```

▷ **Fonctions DOS** : Les fonctions DOS sont employées pour réaliser les opérations standards (affichage, saisie, ...). Le système d'exploitation DOS fournit des fonctions pré-définies :

— Affichage d'un caractère :

```
MOV DL,'A' ; Le caractère à afficher doit être placé dans le registre DL
MOV AH, 2 ; Code de la fonction dos de l'affichage
INT 21H ; Afficher le registre DL
```

— Saisie d'un caractère avec écho

```
MOV AH, 1 ; Code de la fonction dos de saisie avec écho
INT 21H ; Le caractère saisi est maintenant dans AL
```

— Saisie d'un caractère sans écho

```
MOV AH,7 ; Code de la fonction dos de saisie sans écho
INT 21H ; Le caractère saisi est maintenant dans AL
```

— Arrêt du programme

```
MOV AH, 4CH ; Code de la fonction dos de terminaison de programme
INT 21H ; Mettre à la fin du programme
```

Chapitre IV

Interfaces d'entrées/sorties et interruptions de µp 8086

Après avoir présenté l'architecture interne et externe ainsi que la programmation en assembleur de µp 8086, dans ce chapitre, nous allons discuter comment ce µp peut communiquer avec ses périphériques (clavier, écran, souris,...), c.à.d., transmettre et recevoir des données/commandes. Pour cela, il utilise des interfaces d'entrées/sorties et interruptions.

I Généralités sur les interfaces d'entrées/sorties

I.1 Définitions

Les interfaces d'entrées/sorties (E/S) sont des circuits intégrés qui permettent au microprocesseur de communiquer avec ses périphériques comme l'écran, clavier, imprimante, disques, etc. ; Elles utilisent les bus (d'adresses, de données et de commande) pour échanger des données via des ports d'E/S et des registres. Les ports d'E/S sont les points d'accès aux interfaces (figure 1). Ils contiennent des tampons (pour l'entrée) et des bascules (pour la sortie). Registres, mémoires internes des ports, sont utilisés pour stocker les données à envoyer ou reçues (registres de données), ainsi que les bits de contrôle et de statut (registres de configuration).

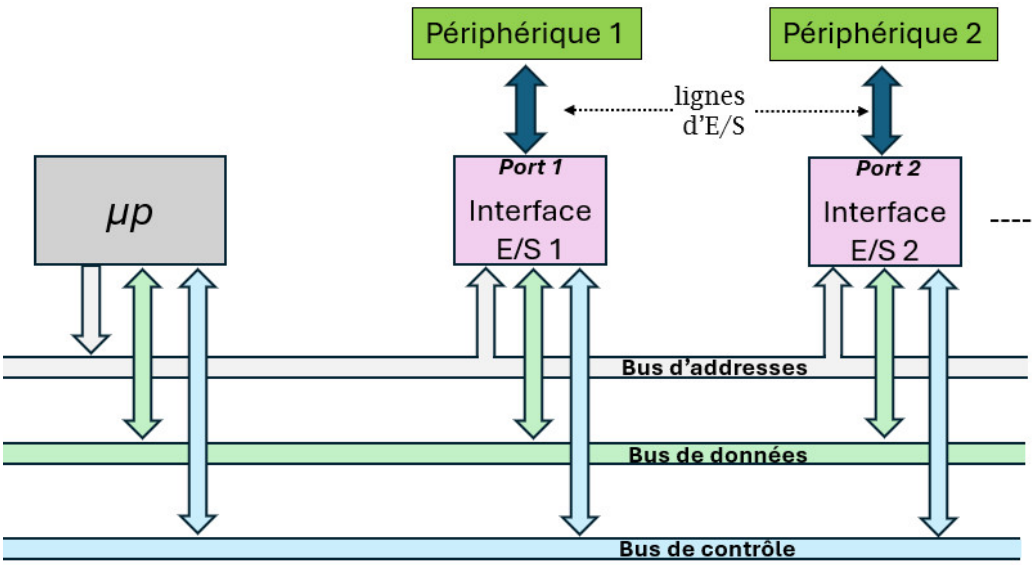


Figure 1 – Interfaces d'entrées /sorties d'un microprocesseur.

On distingue deux types d'interfaces E/S :

— **Interface parallèle** qui permet de transférer des données simultanément sur plusieurs lignes (bits à la fois), utilisant des ports programmables comme le circuit Intel 8255, qui offre des ports configurables (Mode 0, 1, 2) pour des transferts rapides, spécifiquement pour des imprimantes ou des afficheurs, via des signaux de contrôle (lecture, écriture, sélection) et des bus de données.

— **Interface série** qui permet la communication bit par bit, sur une seule ligne ou quelques lignes de données, entre le microprocesseur et des périphériques externes (comme les routeurs, modems, etc.), utilisant des ports programmables comme le circuit Intel 8251, aussi appelée USART (Universal Synchronous Asynchronous Receiver Transmitter). Elle convertit les données parallèles en flux série pour l'émission, et les données séries reçues en données parallèles pour la réception, gérant ainsi la communication série synchrone et asynchrone.

La figure 2 représente l'architecture externe d'une interface E/S.

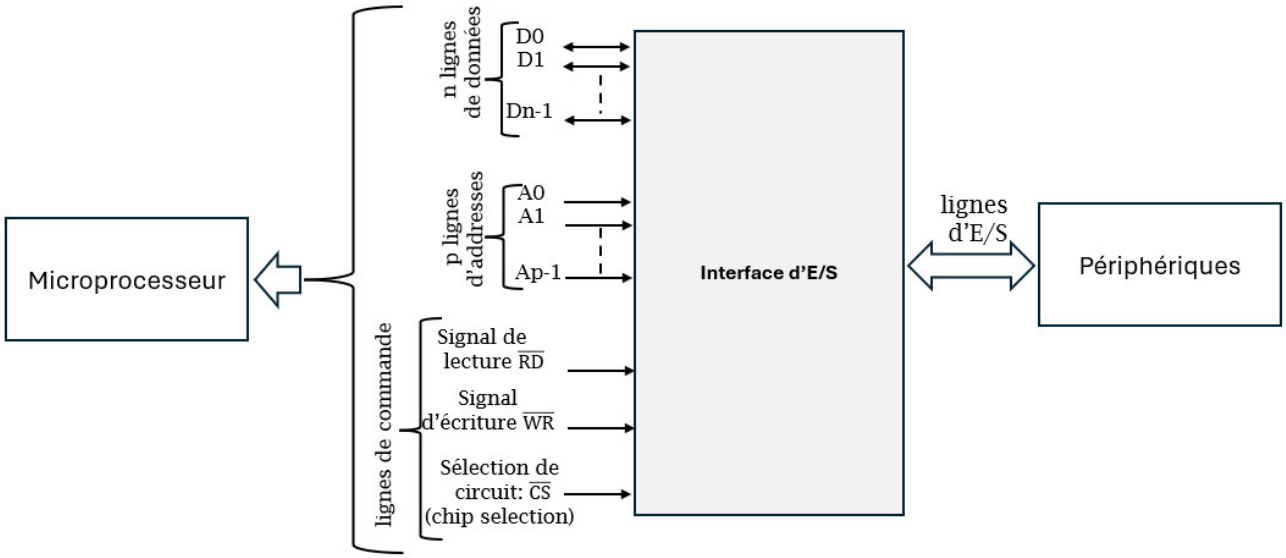


FIGURE 2 – Architecture externe d'une interface E/S.

I.2 Adressage du port d'une interface d'E/S

Les registres de données et de configuration d'une interface d'E/S sont assignés par leurs adresses. Le microprocesseur utilise l'adresse de l'un de ces registres pour accéder au port d'une interface d'E/S, c.à.d., l'adresse du port n'est autre que l'adresse de l'un de ces registres. Suivant l'emplacement de ces adresses, on distingue deux types d'adressages des ports E/S par le microprocesseur :

- **Adressage cartographique** : dans ce type d'adressage, les adresses des ports d'E/S se trouvent au même espace mémoire que les autres segments mémoires (DS, CS, etc.), on dit que les E/S sont mappées en mémoire. Les conséquences de cet adressage sont :
  — l'espace d'adressage des autres segments mémoires (DS, CS, SS, ES) diminue ;
  — les adresses des ports E/S ont la même longueur que les cases mémoires (8 bits) ;
  — les instructions employées pour manipuler les cases mémoires peuvent être appliquées aux ports d'E/S, ce qui signifie que les mêmes instructions permettant de lire et écrire dans la mémoire et les ports d'E/S, ainsi tous les modes d'adressage étant valables pour les E/S.

- **Adressage indépendant :** dans ce type d'adressage, l'espace mémoire et l'espace d'E/S sont séparés, c'est le cas du microprocesseur 8086. Les conséquences de cet adressage sont :
  - à l'inverse de l'adressage cartographique, l'espace mémoire total adressable n'est pas diminué ;
  - l'adressage des ports d'E/S peut se faire avec des adresses plus courtes (moins de bits) que pour les cases mémoire ;
  - contrairement à l'adressage cartographique, le microprocesseur utilise des instructions spécifiques pour accéder aux ports E/S ;
  - le microprocesseur doit fournir des signaux de commande permettant de différencier l'adressage de la mémoire de l'adressage des ports d'E/S.

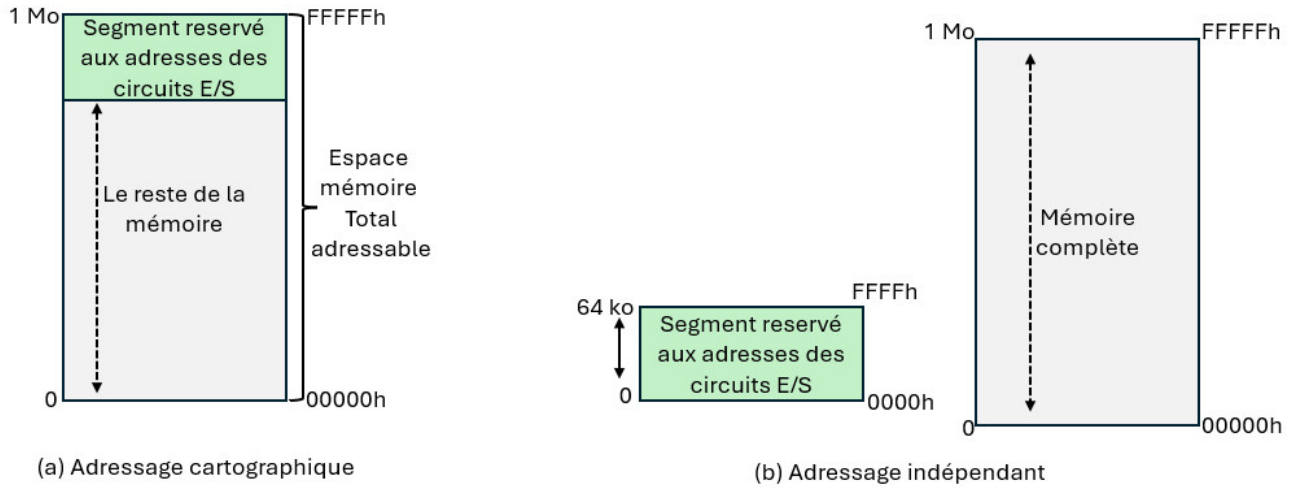


FIGURE 3 – Types d'adressages des ports E/S par le microprocesseur.

Remarque : Contrairement à l'adressage cartographique qui est applicable pour tous les microprocesseurs, l'adressage indépendant n'est applicable que pour les microprocesseurs qui disposent d'un jeu d'instructions spécifiques pour l'accès aux ports d'E/S ainsi qu'un signal de commande qui sert de différencier l'adressage mémoire de l'adressage des ports E/S.

II Interfaces E/S de µp 8086

II.1 Gestion des interfaces d'E/S par le µp 8086

Le µp 8086 utilise l'adressage indépendant pour gérer les ports E/S. Il dispose donc de deux espaces d'adressage séparés : un espace mémoire de 1 Mo (adresse sur 20 bits) utilisé pour le fonctionnement général de µp et un espace d'E/S de 64 Ko (adresse sur 16 bits) pour la gestion des ports E/S. Le microprocesseur utilise le signal $M/\overline{IO}$ (Memory/Input-Output) pour différencier l'adressage de la mémoire de l'adressage des ports.

- $M/\overline{IO} = 1$: accès à la mémoire
- $M/\overline{IO} = 0$: accès aux ports

La figure 4 montre un exemple de commande de circuit de décodage d'adresse dans les deux espaces par le signal $M/\overline{IO}$.

Les instructions de lecture et d'écriture sur les ports E/S sont respectivement **IN** (input) et **OUT** (OUTPUT).

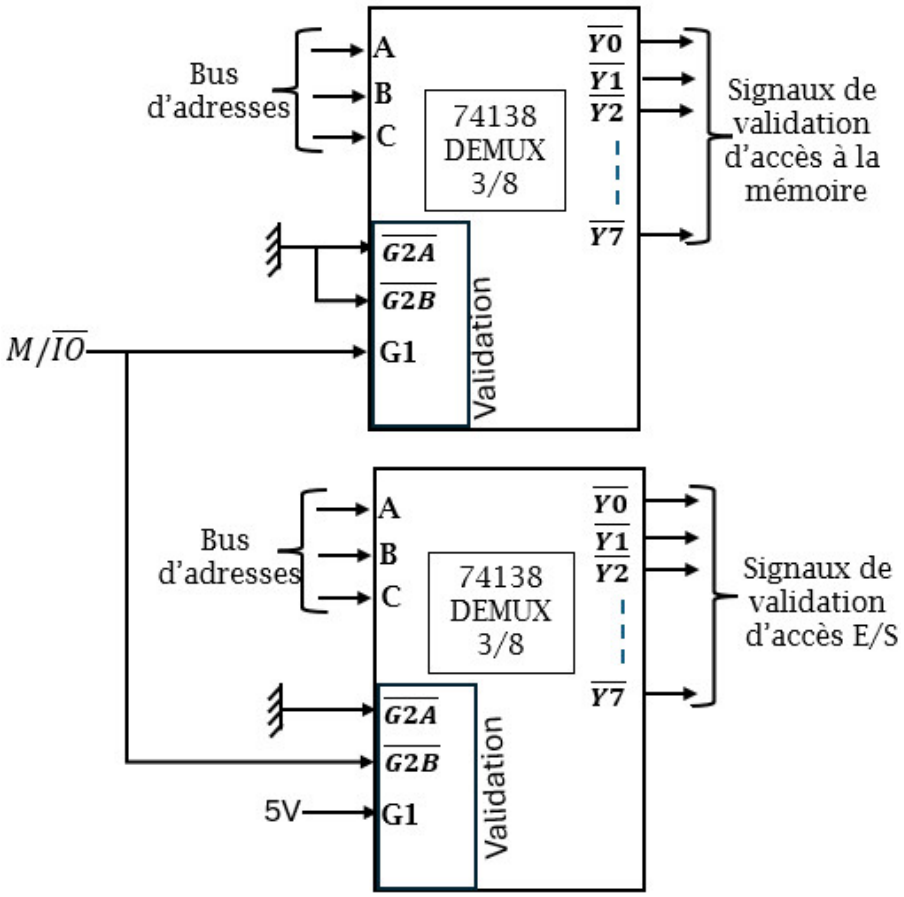


FIGURE 4 – Circuit de validation de décodage d'adresse dans les deux espaces de 8086.

▷ **Syntaxe de lecture sur un port d'E/S :**

- Cas 1 : adresse du port est sur 8 bits :
  - IN AL, adresse ; lecture d'un port sur 8 bits ;
  - IN AX, adresse ; lecture d'un port sur 16 bits.
- Cas 2 : adresse du port est sur 16 bits. Dans ce cas, avant d'accéder au port à lire, il faut charger son adresse dans le registre DX (MOV DX, adresse).
  - IN AL, DX ; lecture d'un port sur 8 bits ;
  - IN, AX, DX ; lecture d'un port sur 16 bits.

Exemple :
MOV DX,250H ; Charger l'adresse du port d'E/S qui est 250H dans DX, adresse est sur 16 bits
IN AL, DX ; Lecture du port d'E/S sur 8 bits à l'adresse 250H

▷ **Syntaxe de l'écriture sur un port d'E/S :**

- Cas 1 : adresse du port est sur 8 bits :
  - OUT adresse, AL ; écriture d'un port sur 8 bits ;
  - OUT adresse, AX ; écriture d'un port sur 16 bits.
- Cas 2 : adresse du port est sur 16 bits. Dans ce cas, avant d'accéder au port à écrire, il faut charger son adresse dans le registre DX (MOV DX, adresse).
  - OUT DX,AL ; écriture d'un port sur 8 bits ;

— OUT DX,AX ; écriture d'un port sur 16 bits.

Exemple :
MOV AX,1300H ; Charger la donnée 1300H dans AX
OUT 85H,AX ; Écriture de la donnée 1300H dans le port d'E/S sur 16 bits à l'adresse 1300H

II.2 Interface parallèle 8255

II.2.1 Brochage du 8255

L'interface 8255 est une interface parallèle programmable de 40 broches (voir figure 5) permettant de transférer des données en parallèle du microprocesseur vers des périphériques et inversement. Elle peut être configurée en entrée et/ou en sortie par programme.

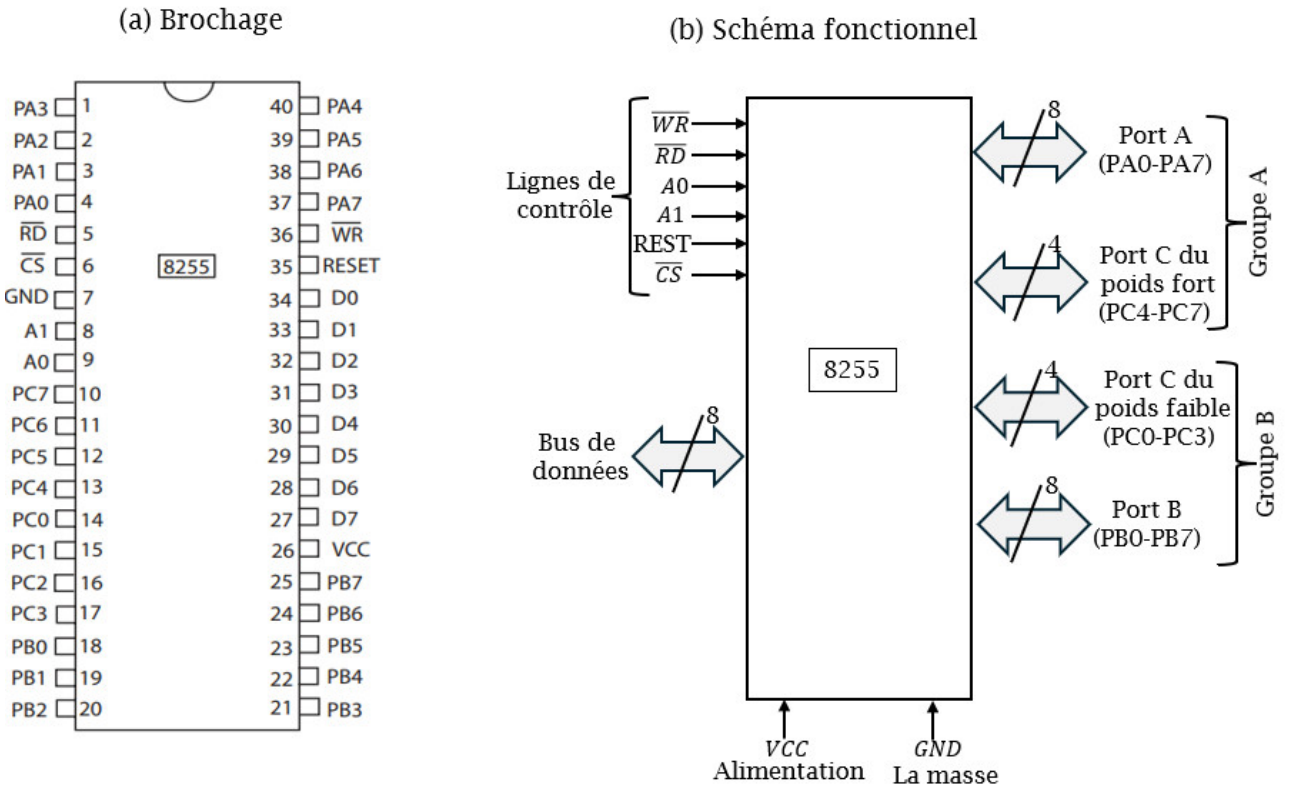


FIGURE 5 – Architecture externe de l'interface parallèle 8255.

Les différentes broches peuvent être regroupées comme suit :

- Les ports A, B et C. Ces ports sont regroupés en deux groupes : groupe A constitué par le port A et le port C haut et groupe B constitué par le port B et le port C bas ;
- Les lignes de contrôle : $\overline{RD}$ et $\overline{WR}$ sont les signaux de lecture/écriture ; A_0 et A_1 sont les bus d'adresse ; $\overline{CS}$ c'est le signal de sélection de l'interface ; $RESET$ c'est le signal de réinitialisation ;
- Le bus de données ;
- Les entrées d'alimentation Vcc et GND.

II.2.2 Accès aux ports du 8255

L'interface 8255 dispose de quatre registres :

- *Trois registres de données* servant de recevoir les données présentées sur les ports A, B et C lors de la lecture ou de l'écriture. Chaque port est associé à un registre. L'accès à un port donné se fait en accédant à son registre.
- *Un registre de contrôle* permettant de configurer des ports A, B et C en entrées et/ou en sorties. Ce registre est accessible uniquement en écriture, c.à.d., sa lecture n'est pas autorisée.

La sélection de l'un de ces registres (ports) se fait à l'aide de deux bits d'adresse $A0$ et $A1$ selon la table de vérité IV.1. Plus en détail, l'adressage des différents ports en entrée et/ou sortie se fait comme le montre le tableau IV.2.

A_0	A_1	Registre (port)
0	0	Port A
0	1	Port B
0	0	Port C
1	1	Registre de contrôle

TABLE IV.1 – Sélection d'un port.

A_0	A_1	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	Opération de lecture (Read))
0	0	0	1	0	Lecture du port A
0	1	0	1	0	Lecture du Port B
1	0	0	1	0	Lecture du Port C
					Opération de l'écriture (Write)
0	0	0	1	0	Écriture du port A
0	1	0	1	0	Écriture du Port B
1	0	0	1	0	Écriture du Port C
1	1	1	0	0	Écriture du registre de commande
					L'interface 8255 hors fonctionnement
X	X	X	X	1	Bus de données troisième état
X	X	1	1	0	Bus de données troisième état
1	1	0	1	0	Impossible

TABLE IV.2 – Sélection et manipulation des ports.

II.2.3 Modes de fonctionnement de l'interface 8255

Suivant le contenu de registre de commande, les ports peuvent être configurés en entrées ou en sorties. De plus, on distingue trois modes de fonctionnement de 8255 :

— **Mode 0** : c'est le mode le plus simple. Dans ce mode, les ports sont configurés en entrée/sortie de base. Les données écrites dans les registres correspondants sont mémorisées sur les lignes de sorties ; l'état des lignes d'entrées est recopié dans les registres correspondants et n'est pas mémorisé. Il n'utilise pas les signaux de contrôle de flux (handshaking). Il est idéal pour des périphériques simples (comme un afficheur simple).

— **Mode 1** : appelé aussi mode d'entrée/sortie échantillonnée (ou strobée). Dans ce mode, le microprocesseur contrôle directement le timing des transferts de données avec les périphériques, utilisant les signaux de contrôle (handshaking) pour gérer le flux de données, comme Strobe (STROB) et Input Buffer Full (IBF). Il permet donc de synchroniser les transferts entre le microprocesseur et le périphérique.

— **Mode 2** : appelé aussi mode de bus bidirectionnel. Il permet une communication bidirectionnelle sur le Port A avec des signaux de contrôle provenant du Port C (PC3-PC7). Il est utilisé pour l'échange de données entre périphériques (comme un contrôleur de disquette) ou avec un autre microprocesseur, supportant les interruptions et utilisant des bascules pour mémoriser les données entrantes et sortantes.

La figure 6 montre la configuration des ports ainsi que le choix des modes selon le contenu du registre de commande. Le rôle des bits D0-D6 change suivant le contenu du bit D7. D7=1 : Mode E/S, les bits D6-D0 sont utilisés pour configurer les Ports A, B et C en entrée/sortie. D7=0 : Mode BSR (Bit Set/Reset) les bits D6-D0 sont utilisés pour manipuler individuellement les bits du Port C.

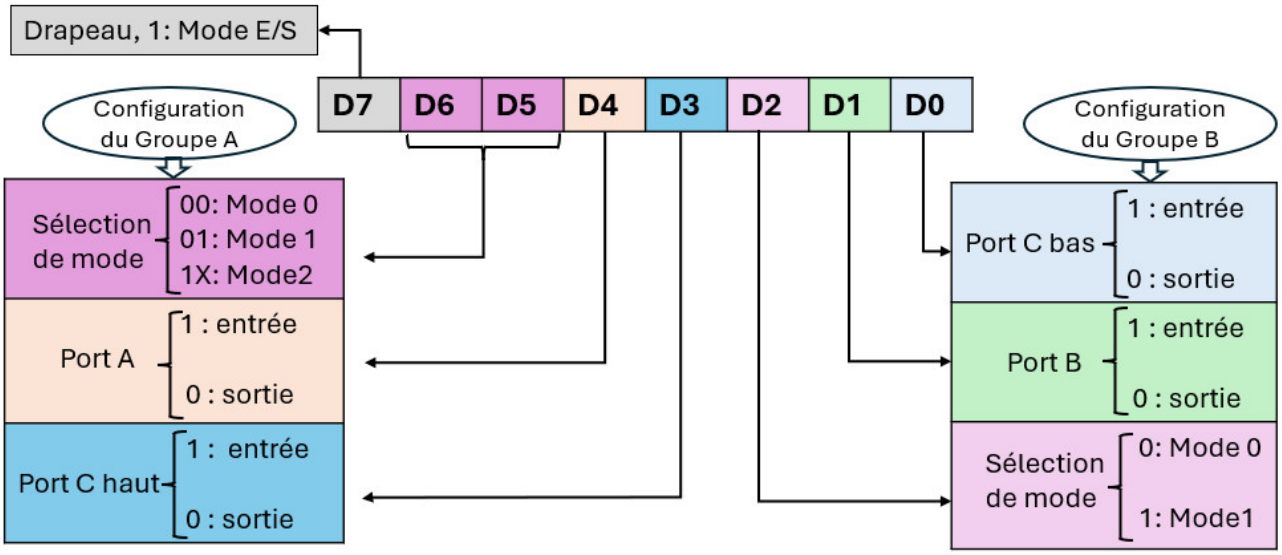


FIGURE 6 – Configuration des ports ainsi que le choix des modes selon le contenu du registre de commande.

II.2.4 Connections du 8255 avec le microprocesseur 8086

Comme nous avons vu, le bus de données du microprocesseur 8086 est sur 16 bits, cependant le bus de données de l'interface 8255 est sur 8 bits. On peut donc connecter le bus de données du 8255 sur les lignes de données de poids fort du 8086 (D8-D15) ou sur celles de poids faible (D0-D7).

Comme nous avons vu dans le chapitre 2, une donnée de 8 bits est envoyée ou reçue par le microprocesseur sur les lignes :

— D0-D7 si l'adresse de la donnée est paire, validation par A0 =0 ;

— D8-D15 si l'adresse de la donnée est impaire, validation par $\overline{BHE} = 0$ (bus high enable) ;

L'un de ces deux signaux peut don être utilisé conjointement avec la sortie d'un décodeur d'adresses pour sélectionner l'interface 8255 (voir la figure 7). On utilise le signal $A0$ dans le cas où les adresses de 8255 sont dans la banque pair (des adresses paires) et $\overline{BHE}$ s'ils sont dans la banque impaire (des adresses impaires). Les bits $A1$ et $A2$ sont utilisés pour sélectionner un registre interne de 8255. A3-A15 sont employés par le décodeur pour sélectionner l'adresse de base de 8255.

Exemple : Connexion du 8255 sur D0-D7 du 8086

La figure 8 montre un exemple de connexion de l'interface 8255 sur les lignes de poids faible du bus de données du 8086, avec décodage d'adresses par les lignes A8-A9-A10.

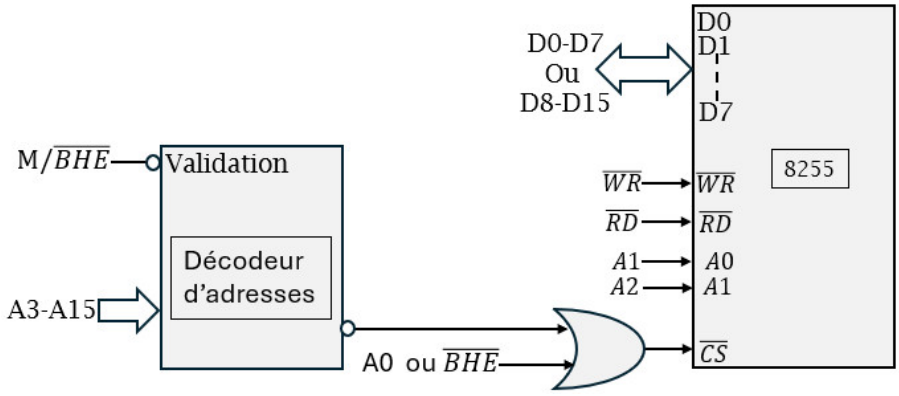


FIGURE 7 – Circuit de sélection de l'interface 8255 par le µp 8086.

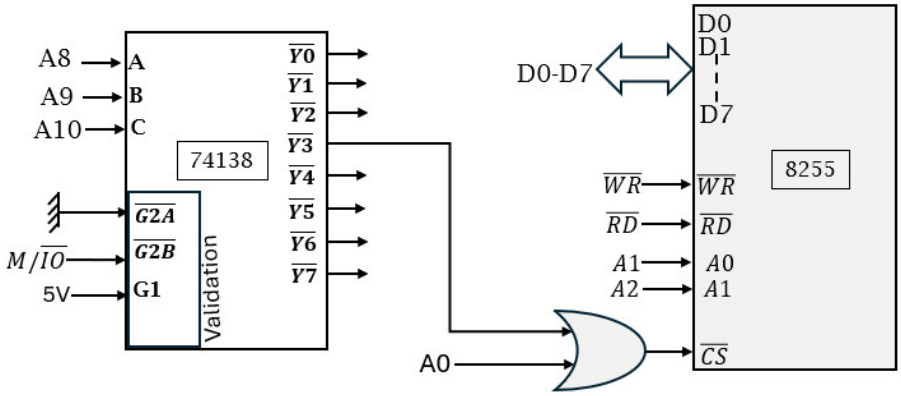


FIGURE 8 – Connexion du 8255 sur 8 bits de poids faible du bus de données du 8086.

A partir de ce schéma, on voit que l'interface 8255 est sélectionnée par le microprocesseur 8086 si $A10A9A8 = 011 = 3$ et $A0 = 0$. Alors donc l'adresse de base du 8255 est 300H. Cette adresse représente l'adresse du Port A du 8255 (A2A1=00), avec les autres combinaisons des bits A2A1 le microprocesseur peut accéder aux autres Ports.

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0
X	X	X	X	X	0	1	1	X	X	X	X	X	X	X	0
Adresse de base =300H													Sélection de registres		$\overline{CS}$=0

Les adresses des différents ports sont déterminées par l'intermédiaire des bits $A2A1$ et les autres comme suit :

A2	A1	Registre sélectionné	Adresse du Registre
0	0	Registre du Port A	300H+0 =300H
0	1	Registre du Port B	300H+2 =302H
1	0	Registre du Port C	300H+4 =304H
1	1	Registre de commande	300H+6 =306H

Remarques :

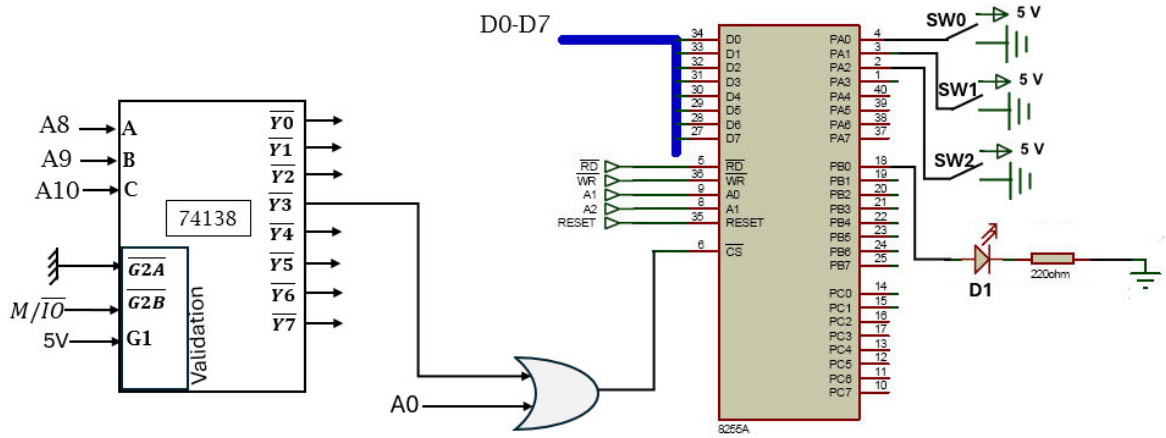
— Puisque dans cet exemple, l'adressage est incomplet, alors le 8255 est disponible dans plusieurs plages d'adresses selon les valeurs des bits non-décodés A15-A11 et A7-A1. L'adresse 300H représente la première adresse possible pour accéder à 8255 (tous les bits non-codés sont mis à 0).

— Ici les adresses de 8255 sont paires, alors pour que le 8255 possède des adresses consécutives (paires et impaires) (c.à.d., 300H,301H, 302H et 303H), il suffit d'exploiter tout le bus de données (D0-D15) du 8086.

II.2.5 Exemple d'application en mode 0

On considère le montage de la figure suivant qui nous permet d'allumer la LED D1 suivant l'état des interrupteurs SW0, SW1 et SW2.

Écrire un programme en assembleur qui permet d'allumer D1 quand les interrupteurs SW0 et SW2 sont fermés sur 5 V et l'interrupteur SW1 est fermé sur 0 V.




```
PortA     EQU 300H          ; adresse des registres du 8255
PortB     EQU 302H
PortC     EQU 304H
Controle  EQU 306H
          MOV DX ,Controle  ; initialisation du port A en entrée
          MOV AL,90H        ; et du port B en sortie (mode 0) :
          OUT DX,AL         ; Controle = 10010000B = 90H
boucle :  MOV DX,PortA      ; lecture du port A
          IN AL,DX
          AND AL,00000111B  ; masquage PA0, PA1 et PA2
          CMP AL,00000101B  ; test PA0 = 1, PA1 = 0 et PA2 = 1
          JNE FAUX          ; non -> aller au label "faux" ...
          MOV AL,00000001B  ; oui -> mettre PB0 ' a 1
          JMP suite         ; et continuer au label "suite"
faux :    MOV AL,00000000B  ; ... mettre PB0 à 0
suite :   MOV DX,PortB      ; écriture du port B
          OUT DX,AL
          JMP boucle        ; retourner lire le port A
```

II.3 Interface série 8250

II.3.1 Généralités sur les interfaces séries

Malgré sa rapidité, les interfaces parallèles ne sont pas adaptées à la communication entre le microprocesseur et certains de ses périphériques (comme le clavier, la souris, etc.). En plus, elles sont très coûteuses lors d'une communication longue distance. Dans ces conditions, les liaisons séries sont une meilleure alternative. Elle permet de réduire le nombre de fils de connexions, simplifiant le câblage. L'inconvénient majeur de ces liaisons est leur faible vitesse de transmission par rapport aux interfaces parallèles. La figure 9 présente l'architecture générale des interfaces séries.

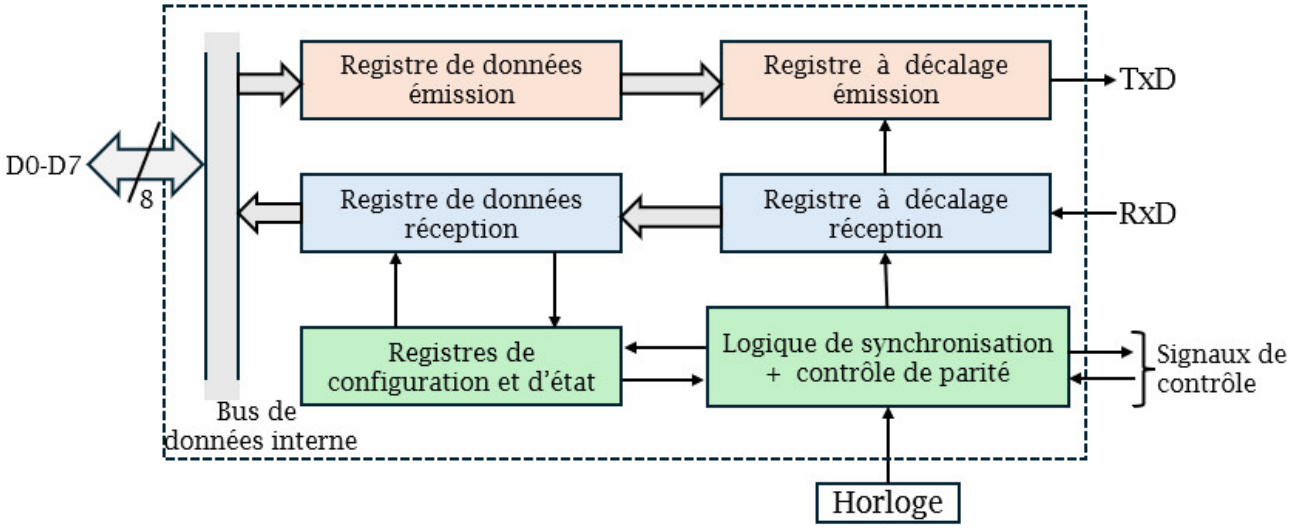


FIGURE 9 – Structure générale d'une interface série.

Modes de transmission série

Il existe trois modes de transmission série :

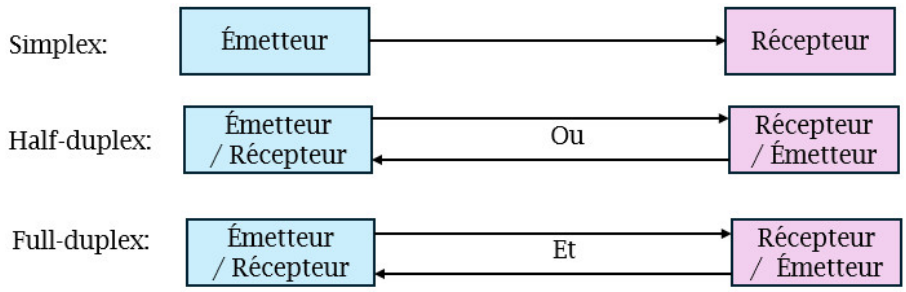


FIGURE 10 – Modes de transmission série.

— *Simplex est unidirectionnel* : dans ce mode les données circulent toujours dans le même sens, de l'émetteur vers le récepteur, et jamais dans l'autre sens (comme la télévision et la radio).

— *Half-duplex* (semi-duplex) est bidirectionnel mais alterné, la communication se fait dans les deux sens, mais pas simultanément. Les deux appareils émettent et reçoivent à tour de rôle (comme les talkies-walkies).

— *Full-duplex* (duplex intégral) est bidirectionnel et simultané, la communication se fait dans les deux sens en même temps (comme le téléphone).

Types de transmission série

On distingue entre deux types de transmission séries :

— *synchrone* : dans ce type, le microprocesseur et le périphérique partagent un signal d'horloge commun pour cadencer les échanges, les données successives sont transmises par blocs séparés par une durée déterminée.

— *asynchrone* : le microprocesseur et le périphérique n'utilisent pas d'horloge commune, mais des bits de start/stop ou des signaux de contrôle pour signaler le début et la fin des données, permettant des transferts plus flexibles mais potentiellement plus lents.

Remarques :
- Le microprocesseur initie le transfert, l'interface s'occupe de la synchronisation avec le périphérique, puis notifie le processeur une fois le travail terminé, libérant ce dernier pour d'autres tâches. Mais il utilise le bus système (synchrone) pour communiquer rapidement avec la mémoire et les contrôleurs.
- La vitesse de transmission est mesurée en bps (nombre de bits par seconde) et aussi par BAUD (nombre des changements du signal par seconde).
- Il existe deux types de circuits intégrés pour la communication série : interface série asynchrone s'appelle UART (Universal Asynchrenous Receiver-Transmitter, exemple : interface 8250 de Intel) et interface série synchrone/asynchrone USART (Universal Synchroneous- Asynchroneous Receiver-Transmitter, exemple : interface 8251 de Intel).

II.3.2 Brochage du 8250

L'interface 8250 est une interface série asynchrone programmable de 40 broches (voir la figure 11) assurant un transfert de données en série entre le microprocesseur et ses périphériques. Le tableau IV.3 montre le nom complet et le rôle de chaque broche.

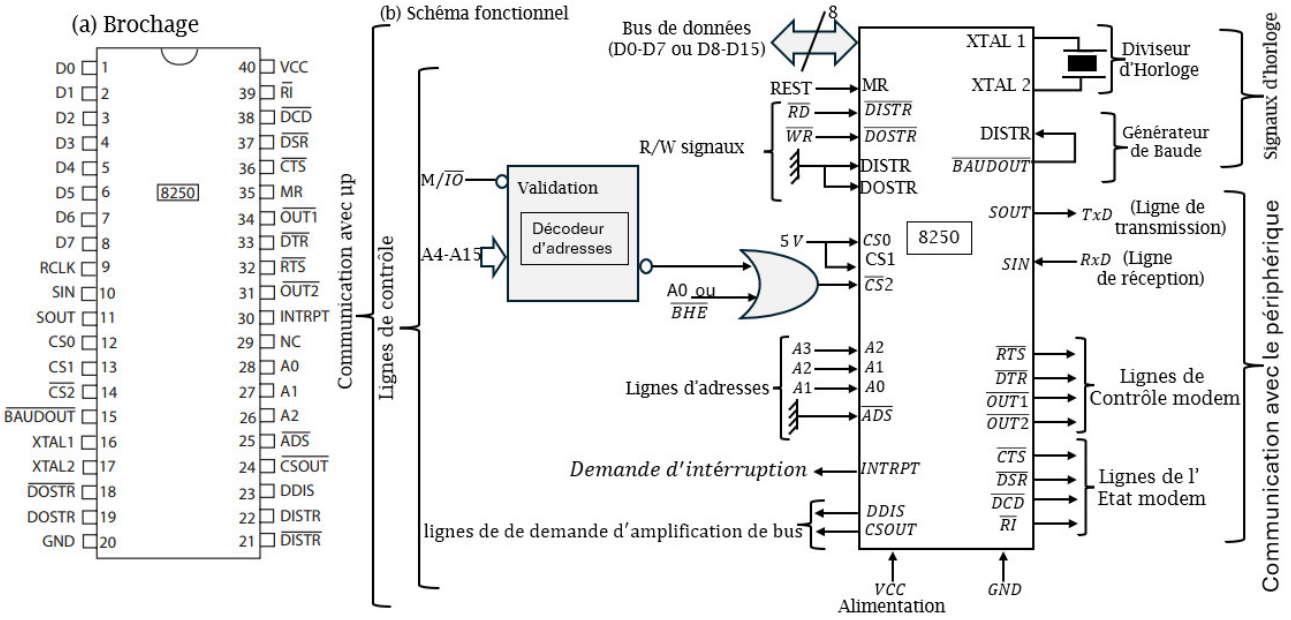


FIGURE 11 – Architecture externe de l'interface série 8250.

Pin	Nom	Rôle
1-8	D0-D7 ou D8-D15	Bus de données
9	RCLK	(Receiver Clock) Entrée d'horloge de réception (16 fois le débit en bauds)
10	SIN	(Serial Input) Entrée de données série
11	SOUT	(Serial Output) Sortie de données série
12-13-14	CS0-CS1-$\overline{CS2}$	(Chip Select) : Sélectionne le circuit
15	$\overline{BAUDOUT}$	Sortie d'horloge du générateur de débit en bauds interne
16-17	XTAL1-XTAL2 (ou XIN-XOUT)	(informal abbreviation for crystal) Connexions pour le cristal de quartz entrée-sortie
18-19	$\overline{DOSTER}$-DOSTR (ou $\overline{WR}$/WR)	(Data Output Strobe) Commandes d'écriture : Niveau-bas.
20	GND (Ground)	masse (0v)
21-22	$\overline{DISTER}$-DISTR (ou $\overline{RD}$/RD)	(Data Input Strobe) Commandes de lecture : Niveau-bas.
23	DDIS	(Driver Disable) Désactivation du pilote de bus
24	CSOUT	(Chip select out) Sortie confirmant la sélection du circuit
26-27-28	A2-A1-A0	Bus adresses des registres internes
25	$\overline{ADS}$	(Address Strobe) Échantillonnage d'adresse
29	NC	Non connecté
30	INTRPT	Sortie d'interruption vers le processeur
31	$\overline{OUT2}$	Sortie utilisateur 2 (souvent utilisée pour activer les interruptions)
32	$\overline{RTS}$	(Request to Send) Demande d'émission)
33	$\overline{DTR}$	(Data Terminal Ready) Terminal de données prêt
34	$\overline{OUT1}$	Sortie utilisateur 1
35	MR	(Master Reset) Réinitialisation générale
36	$\overline{CTS}$	(Clear to Send) Prêt à émettre
37	$\overline{DSR}$	(Data Set Ready) Équipement de données prêt
38	$\overline{DCD}$	(Data Carrier Detect) Détection de porteuse
39	$\overline{RI}$	(Ring Indicator) Indicateur d'appel
40	VCC	Alimentation positive (+5V)

TABLE IV.3 – Description détaillée des broches du 8250.

II.3.3 Registres du 8250

Le 8250 dispose de 11 registres. La sélection de l'un de ces registres se fait par l'intermédiaire de trois bits d'adresses (A0, A1 et A2) et le bit DLAB (Divisor Latch Access Bit) de registre de contrôle de linge (LCR). Puisque il n'y a que 3 bits d'adresses, plusieurs registres doivent se partager la même adresse. Ces registres peuvent être regroupés en quatre groupes :

— Les registres de contrôle de protocole incluent LCR, DLR (DLL/DLM) ;

— Les registres de contrôle de la ligne incluent LSR, MSR, MCR

— Les registres de contrôle d'interruptions incluent IER et IIR ;

— Les registres de transfert de données incluent RBR et THR ;

— Le registre à usage général (SCR).

DLAB	A2	A1	A0	Registre sélectionné
0	0	0	0	**RBR** (Receiver Buffer Register) : registre de réception (accessible seulement en lecture)
0	0	0	0	**THR** (Transmitter Holding Register) : registre d'émission (accessible seulement en écriture)
1	0	0	0	**DLL** (Divisor Latch LSB) : octet de poids faible du diviseur d'horloge
1	0	0	1	**DLM** (Divisor Latch MSB) : octet de poids fort du diviseur d'horloge
0	0	0	1	**IER** (Interrupt Enable Register) : registre d'autorisation des interruptions
X	0	1	0	**IIR** (Interrupt Identification Register) : registre d'identification des interruptions
X	0	1	1	**LCR** (Line Control Register) : registre de contrôle de ligne
X	1	0	0	**MCR** (Modem Control Register) : registre de contrôle modem
X	1	0	1	**LSR** (Line Status Register) : registre d'état de la ligne
X	1	1	0	**MSR** (Modem Status Register) : registre d'état du modem
X	1	1	1	**SCR** (Scratch Register) : registre à usage général

TABLE IV.4 – Sélection de différents registres du 8250.

II.3.3.1 Registres de contrôle de protocole : Ces registres ont pour rôle de construire un protocole de communication (réglages des bits de parités et de stop, sélection de la vitesse de transmission).

▷ **LCR** (registre de contrôle de la ligne ou Line Control Register en anglais) : La structure de ce registre est définie comme suit :

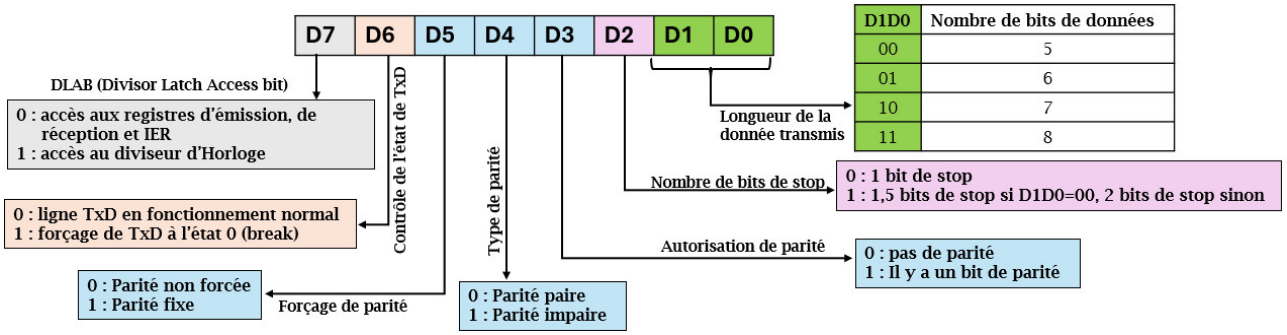


FIGURE 12 – Structure et rôle des bits de registre LCR.

▷ **DLR** (registre de sélection de la vitesse de transfert ou Divisor Latch Register en anglais) : ce registre est codé sur 16 bits (LSB dans DLL et MSB dans DLM). Il permet de sélectionner facilement une vitesse de transfert de données comme suit :

$$\text{vitesse (bits/s)} = \frac{\text{Fréquence d'horloge}}{16 \times (DLM, DLL)} \tag{1}$$

Exemple : Déterminer le contenu du registre DLR si la vitesse de transmisssion est 1200 bits/s et la fréquence d'horloge égale à 1.8432 MHz.

D'après la relation (1), on a

$$(DLM, DLL) = \frac{\text{Fréquence d'horloge}}{16 \times \text{vitesse}} = \frac{1.843210^6}{16 \times 1200} = 96 \tag{2}$$

D'où DLM=00 et DLL=96.
Remarque : Pour accéder à ce registre, il faut mettre le bit DLAB du registre LCR à l'état 1.

II.3.3.2 Registres de contrôle de la ligne : Ces registres (LSR, MSR et MCR) permettent de connaître l'état de la ligne ou du modem, et de les réinitialiser.

▷ **LSR** (registre d'état de la ligne ou Line Status Register en anglais) : La structure de ce registre est définie de la façon suivante :

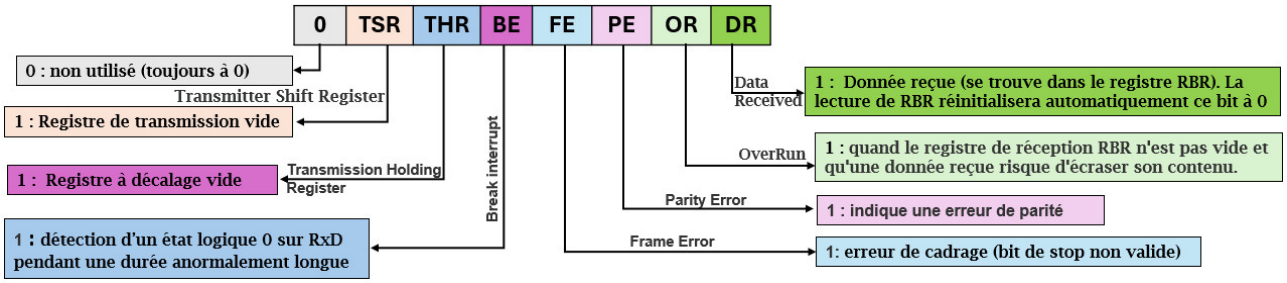


FIGURE 13 – Structure et rôle des bits du registre LCR.

▷ **MSR** (Registre d'état du modem ou Modem Status Register en anglais) : La structure de ce registre est comme suit :

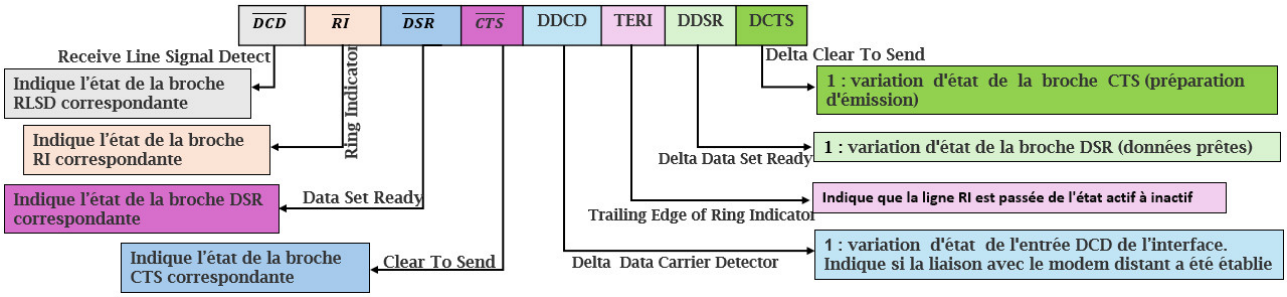


FIGURE 14 – Structure et rôle des bits du registre MSR.

▷ **MCR** (Registre de contrôle du modem ou Modem Control Register en anglais) : La structure de ce registre est définie de la manière suivante :

II.3.3.3 Registres de contrôle d'interruptions : A l'aide de ces registres, on peut identifier quels événements déclencheront une interruption. Une fois, l'interruption est détectée, on peut identifier sa cause, et y répondre.

▷ **IER** (Registre d'autorisations d'interruptions ou Interrupt Enable Register en anglais) : Pour pouvoir accéder à ce registre, il faut en premier temps forcer le bit DLAB du registre LCR à 0. Les quatre bits de poids faible (bits 0 à 3) permettent, lorsqu'ils sont forcés à 1, de

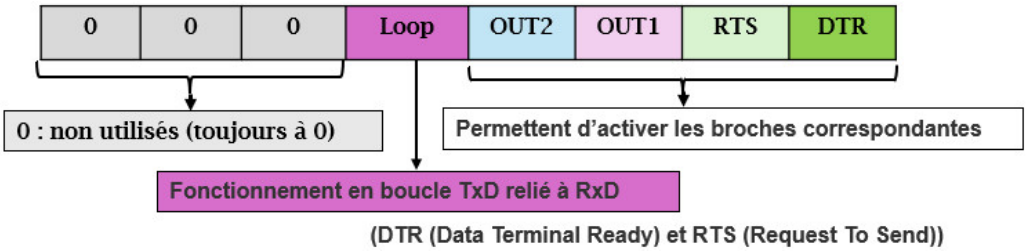


FIGURE 15 – Structure et rôle des bits du registre MCR.

générer une interruption lors de l'apparition de l'événement associé. Le registre IIR (Interrupt Identification Register) permet d'identifier la cause de cette interruption. La structure de ce registre ainsi que les événements associés avec les bits de ce registre sont présentés sur la figure suivante.

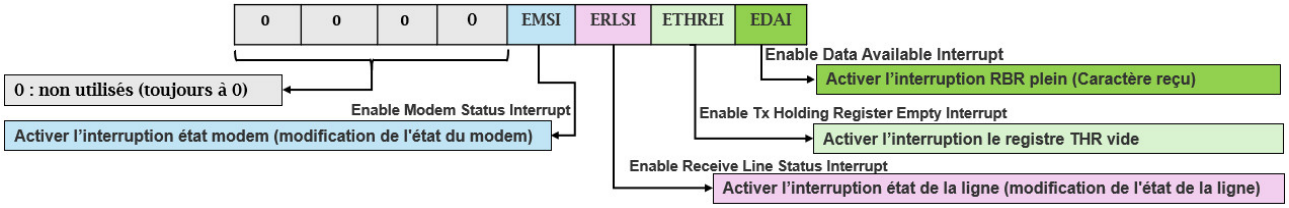


FIGURE 16 – Structure et rôle des bits du registre IER.

▷ **IIR** (Registre d'identification d'interruptions ou Interrupt Identiffication Register en anglais) : Le bit IID0 (Interrupt IDtification zero) de ce registre permet de montrer s'il y a une interruption en cours ($IID0 = 0$) ou non ($IID0 = 1$). En revanche, les bits IID1 et IID2 permettent de déterminer la cause de l'interruption, sa priorité, et le moyen de la désactiver. Le tableau IV.5 fournit les différentes interruptions possibles suivant les valeurs de ces deux bits.

II.3.3.4 Registres de transfert

▷ **THR** (registre d'attente de l'émetteur ou Transmitter Holding Register en anglais) : Lorsqu'une donnée doit être transmise, elle doit d'abord être transférée dans le registre THR qui est un registre de 8 bits. Il y restera tant que la donnée précédente ne sera pas reçue par le destinataire. Une fois, la donnée précédente est reçue, le contenu de THR sera transféré dans un autre registre : TSR (Transmission Shift Register = Registre de décalage de l'émetteur). L'interface série se chargera alors de transmettre la donnée bit à bit sur la ligne, et il pourra y insérer, selon les réglages effectués dans LCR (contrôle de ligne), un bit de parité et un nombre fixé de bits de stop.

Remarques :

— Ce registre est à écriture seule.

— Le bit DLAB du registre contrôle de la ligne LCR doit être mis à 0 pour pouvoir écrire dans ce registre.

— Suivant les réglages du registre de contrôle de ligne LCR, on pourra y envoyer de 5 à 8 bits avec le registre THR. On note que le bit 0 (poids le plus faible) qui sera le premier transmis.

0	0	0	0	0	IID2	IID1	0 (IID0)

IID2	IID1	Niveau de Priorité	Type d'interruption	Source de l'interruption	Méthode de réinitialisation
1	1	Maximale	Modification de LSR (Etat de la ligne), bit ERLSI de IER égale à 1	- Surcharge (Une nouvelle donnée arrive alors qu'on n'a pas encore lu la dernière donnée reçue) ou - Erreur de parité ou - Erreur de cadrage (bit de stop) ou - Forçage de TxD à 0 (break)	Lecture de LSR
1	0	2ème ordre	Réception d'un caractère ; bit EDAI du IER égale à 1	Données reçues	Lecture de registre de réception (RBR)
0	1	3ème ordre	Registre de transmission vide (THR) ; bit ETHREI du IER égale à 1	Registre THR vide	Écriture dans THR ou lecture de IIR
0	0	4ème ordre	Modification de l'état du modem ; bit EMSI de IER égale à 1	- Modification de la broche CTS (clear to send) ou - RI (ring indicator) ou - DCD (Data carrier detect)	Lecture de registre de l'état de modem (MSR)

TABLE IV.5 – Identification des interruptions suivant le contenu du registre IIR.

— Il est sage d'effectuer une lecture de registre d'état de la ligne LSR pour savoir quand le registre THR devient libre (bit 5), et quand la donnée présente dans TSR a été complètement transmise sur la ligne (bit 6).

▷ **RBR** (Registre de réception ou Receiver Buffer Register en anglais) : Ce registre contient la donnée reçue (sans ses bits de parité et de stop). De la même façon qu'avec les registres de transmission, les données reçues sont en premier temps réceptionnées par un registre de transit - nommé RSR (Reception Shift Register = Registre de décalage de récepteur)) - avant d'être débarrassées de ses bits de contrôle.

Remarques :

— Le registre RBR est à lecture seule. Il possède la même adresse que THR. Ainsi un ordre d'écriture à l'adresse de THR renverra donc le contenu de RBR, mais n'aura aucun effet sur RBR.

— Le bit DLAB du registre LCR (contrôle de la ligne) doit être mis à 0 pour pouvoir lire ce

registre.

— RBR est sur 8 bits : y seront reçus les 5 à 8 bits de données, sans bits de contrôle. Le bit de poids le plus faible (bit 0) est le premier à avoir été reçu.

— Il est recommandé d'effectuer une lecture de registre d'état de la ligne LSR pour détecter les éventuelles erreurs de transmission (bits 1 à 4), ou pour savoir si une donnée a été reçue par la partie traitante (bit 0).

II.3.3.5 Registre à usage général : Le registre à usage général, SCR (Scratch Register), contient des données temporaires.

II.3.4 Exemples de programmation du 8250

On distingue deux types de programmation des interfaces séries : direct ou avec les interruptions de BIOS/DOS.

II.3.4.1 Programmation directe : Dans ce cas, la programmation de l'interface se fait directement en utilisant leur jeu d'instructions.

Exemple 1 : Lecture d'un caractère du port COM1

```
            LSR EQU 3FDH      ; Adresse du registre LSR
            TXRX EQU 3F8H     ; Adresse du registre THR et RBR
            RDA EQU 01H       ; Masque pour tester la réception d'une donnée (registre RBR)
DEBUT :     IN AL, LSR        ; Lecture de LSR
            TEST AL, RDA      ; Tester RDA
            JZ DEBUT          ; If not RDA
            IN AL, TXRX       ; Lecture de la donnée
```

Exemple 2 : Écriture d'un caractère 'a' sur le port COM1

```
            LSR EQU 3FDH      ; Adresse du registre LSR
            TXRX EQU 3F8H     ; Adresse du registre THR et RBR
            TBE EQU 20H       ; Masque pour tester si le registre THR est vide
DEBUT :     IN AL, LSR        ; Lecture LSR
            TEST AL, TBE      ; Tester TBE
            JZ DEBUT
            MOV AL, 'a'       ; Caractère à transmettre
            OUT TXRX, AL      ; Envoie caractère sur le port
```

II.3.4.2 Programmation en utilisant les interruptions BIOS/ DOS : Les interruptions BIOS/DOS sont des mécanismes permettant aux programmes d'accéder à des services matériels (BIOS) ou système d'exploitation (DOS) comme l'affichage, le clavier, le disque, via des appels logiciels (ex : INT 21H pour DOS, INT 10H pour le vidéo BIOS). Comme le montre le tableau IV.7, l'opération à exécuter lors de l'appel à ces interruptions dépend du contenu du registre *AH* du microprocesseur.

AH	Opération
03H	Lecture d'un caractère
04H	Écriture d'un caractère

TABLE IV.6 – Interruption DOS INT 21H

AH	Opération	Entrée	Sortie
00	Initialisation	AL=Paramètre DX=Port	AH : État du port AL : État du modem
01	Écriture d'un caractère	AL= Caractère DX=Port	AH : Bit7= 0 ; succès AH : Bit7=1 ; échec
02	Lecture d'un caractère	DX=Port	AH : Bit7= 0 ; succès AH : Bit7=1 ; échec
03	Lecture de statut	DX=Port	AH : État du port AL : État du modem

TABLE IV.7 – Interruption BIOS INT 14H.

Exemple 1 : Lecture d'un caractère du port COM1

```
        COM1 EQU 0
        COM2 EQU 1
        TBE EQU 20H
        RDA EQU 01H
DEBUT : MOV DX, COM1
        MOV AH, 03H
        INT 14H
        TEST AH, RDA
        JZ DEBUT
        MOV DX, COM1
        MOV AH, 02H
        INT 14H
```

Exemple 2 : Écriture du caractère 'a' sur le port COM1

```
        COM1 EQU 0
        COM2 EQU 1
        TBE EQU 20H
        RDA EQU 01H
DEBUT : MOV DX, COM1
        MOV AH, 03H
        INT 14H
        TEST AH, TBE     ; Test la valeur de TBE JZ DEBUT
        MOV DX, COM1
        MOV AH, 01H
        MOV AL, 'a'      ; Écriture du caractère
        INT 14H
```

III Interruptions du 8086

III.1 Définition

On rappelle que la réception de données provenant des périphériques par le microprocesseur se fait via l'une des deux techniques suivantes :

- **Polling** (ou scrutation périodique) : Cette technique consiste à examiner périodiquement l'état des ports E/S, le programme principal doit donc contenir des instructions qui font cette opération.
  - — Avantage : simplicité de programmation.
  - — Inconvénients :
    - — temps d'examination augmente s'il y a de nombreux périphériques à examiner ;
    - — même si de nouvelles données ne sont pas présentées, le microprocesseur doit vérifier l'état des ports ;
    - — des données peuvent être perdues si elles varient rapidement.
- **Interruption** : dans cette technique, une fois une nouvelle donnée apparaît sur un périphérique, le circuit d'E/S envoie au microprocesseur une demande de lecture de la donnée apparue : on parle de demande d'interruption ***(IRQ : Interrupt Request)***.
  - — Avantage : la lecture des ports d'E/S par le microprocesseur se fait seulement lorsqu'une nouvelle donnée est disponible, ce qui permet d'éviter de perdre des données et de gagner du temps.
  - — Inconvénient : complexité de programmation par rapport au polling.

Exemple : parmi les périphériques qui utilisent les interruptions, on trouve :
- Port série : demande d'interruption lorsqu'une donnée est arrivée sur la ligne de transmission ;
- Clavier : demande d'interruption quand une touche est appuyée.

Remarque : Une demande d'interruption peut être générée par le microprocesseur lui-même en cas de problèmes tels qu'une division par zéro, une erreur d'alimentation ou un circuit mémoire défectueux (erreurs fatales). Dans ces conditions, la demande d'interruption conduit à l'arrêt du microprocesseur.

III.2 Traitement de demandes d'interruptions par le µp

Suite à une demande d'interruptions par le périphérique, le microprocesseur peut *accepter ou refuser* cette demande.

Le microprocesseur suit les étapes suivantes une fois qu'il accepte une interruption :

— Il termine l'exécution de l'instruction en cours ;

— Il sauvegarde le contenu des principaux registres sur la pile ;

— Il envoie un signal d'accusé de réception (INTA : Interrupt Acknowledge) signalant au périphérique que la demande d'interruption est acceptée.

— Il suspend l'exécution du programme en cours et va chercher à exécuter un sous-programme de service de l'interruption (ISR : Interrupt Service Routine) ;

— Une fois l'exécution de l'ISR est terminé, les registres sont restaurés à partir de la pile et le microprocesseur reprend l'exécution du programme qu'il avait suspendu.

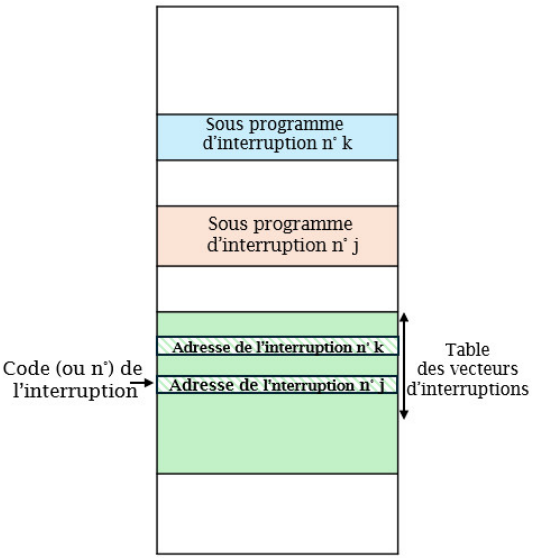
En revanche, le processeur refuse une interruption, c'est souvent parce qu'elle est **masquée** (priorité insuffisante ou temporairement désactivée pour une tâche critique), et dans ce cas, aucun accusé de réception formel n'est envoyé pour la refuser ; le processeur continue simplement son travail. Mais, l'interruption peut rester en attente si elle n'est pas masquée de manière permanente, jusqu'à ce que le processeur soit prêt à la traiter (par exemple, si la priorité actuelle baisse).

Remarques :

- La dernière instruction d'un sous-programme de service d'interruption est IRET (Interrupt Return ou retour d'interruption en français).
- Si plusieurs interruptions sont produites en même temps, alors le microprocesseur commence par le traitement des interruptions de priorité supérieure.

III.3 Adresses des sous-programmes d'interruptions

Chaque interruption correspond à un sous-programme de service rangé dans la mémoire centrale. Lorsqu'une demande d'interruption survient, le microprocesseur a besoin de déterminer l'adresse du sous-programme associé à cette interruption. Pour cela, le périphérique qui émit l'interruption place sur le bus de données un code numérique signalant la nature de l'interruption. Le microprocesseur utilise ce code pour rechercher dans une zone en mémoire centrale, appelée tableau de vecteurs d'interruptions, l'adresse du sous-programme d'interruption à exécuter. Chaque case de ce tableau s'appelle un *vecteur d'interruption*, on dit dans ce cas que les interruptions sont vectorisées. Cette technique de vectorisation des interruptions est très importante, car dans ce cas, l'emplacement d'une ISR peut être n'importe où dans la mémoire et il suffit de spécifier le vecteur d'interruption correspondant.



III.4 Interruptions du 8086

Le µp 8086 peut gérer jusqu'à 256 interruptions. Chaque interruption est désignée par un numéro compris entre 0 et 255 appelé type de l'interruption. On distingue trois types d'interruptions du 8086 :

- **Interruptions matérielles** : générées par l'activation des lignes INTR (Interrupt Request) et NMI (Non-Maskable Interrupt) du microprocesseur ;
- **Interruptions logicielles** : générées par l'instruction INT n, où n est le type de l'interruption (Ex : INT 21) ;
- **Interruptions processeur** : produites par le microprocesseur en cas de division par zéro, de dépassement, ou lors de l'exécution pas à pas d'un programme.

Les interruptions du 8086 sont vectorisées. La table des vecteurs d'interruptions doit obligatoirement commencer à l'adresse physique 00000H dans la mémoire centrale. Chaque vecteur d'interruption est constitué de 4 octets représentant une adresse logique du type CS : IP.

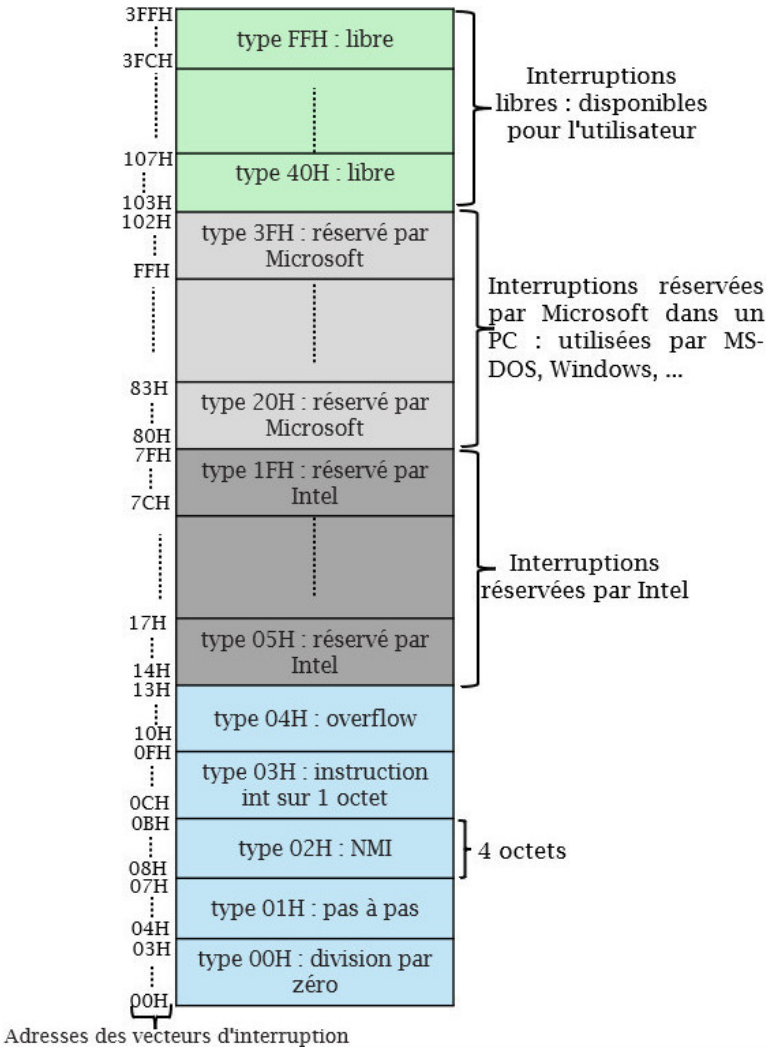


FIGURE 17 – Interruptions du 8086.

Remarques :
- L'adresse du vecteur correspondant à une interruption est liée au numéro de l'interruption comme suit :

$$\textbf{adresse du vecteur correspondant} = 4 \times \textbf{numéro de l'interruption} \qquad (3)$$

Exemple : interruption 02H, adresse du vecteur = 4 × 02H = 08H.
- Au démarrage du système, le programme principal (carte à microprocesseur) ou le système d'exploitation (ordinateur) prend en charge la table des vecteurs d'interruptions. Elle peut être changée en cours de fonctionnement, on parle de détournement des vecteurs d'interruptions.

III.5 Contrôleur d'interruptions 8259

Puisqu'il n'y a que deux lignes de demandes d'interruptions sur le microprocesseur 8086 (INTR et NMI), alors il faut donc utiliser un circuit supplémentaire qui sert à gérer les différentes interruptions lorsque le microprocesseur est relié à plusieurs périphériques qui utilisent des interruptions. Pour cela, on utilise le contrôleur programmable d'interruptions 8259 qui peut générer jusqu'à 8 interruptions matérielles. Il permet :

— d'accueillir des demandes d'interruptions des périphériques ;

— d'identifier les priorités des interruptions ;

— d'informer le microprocesseur via signal INTR ;

— de placer le numéro de l'interruption sur le bus de données.

La figure 18 montre le brochage et le schéma fonctionnel du 8259.

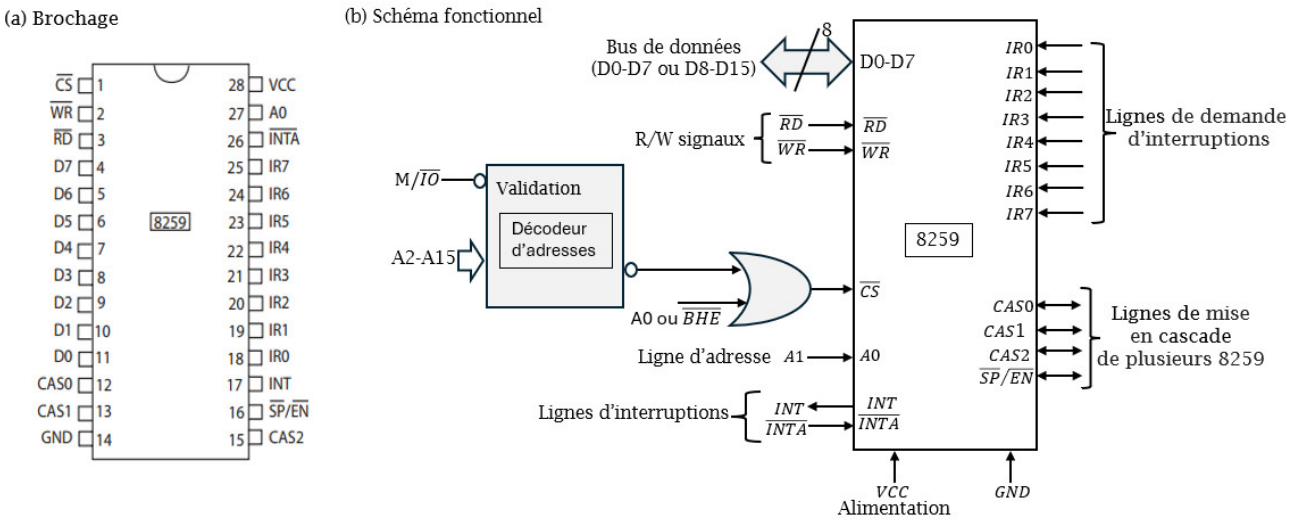


FIGURE 18 – Brochage et schéma fonctionnel de contrôleur d'interruptions 8259.

Remarque : Dans le cas où le nombre d'interruptions est supérieur à 8, on peut mettre en cascade autant nécessaire de contrôleurs 8259 (voir la figure 19).

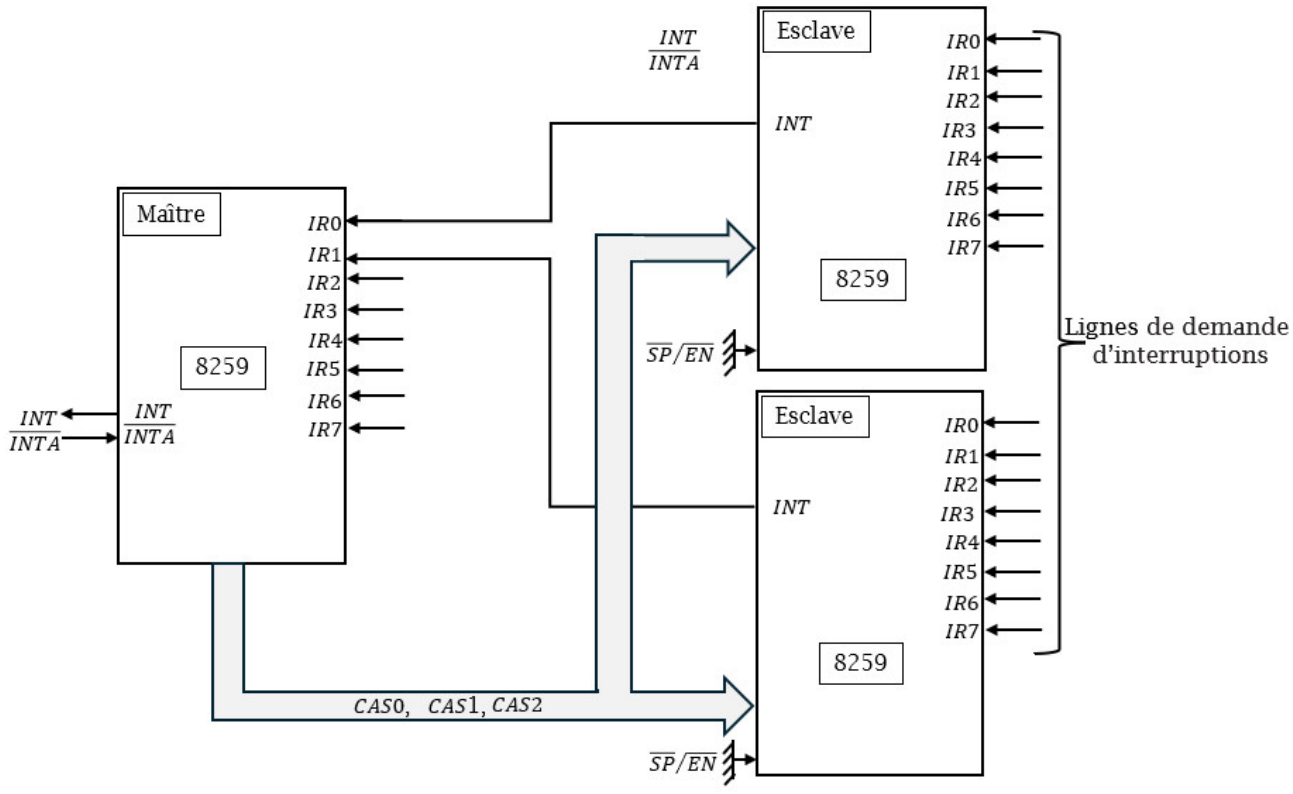


FIGURE 19 – Association des contrôleurs d'interruptions 8259 pour gérer un nombre important d'interruptions.

Partie II

Systèmes à microcontrôleur

Chapitre V

Microcontrôleurs : Arduino uno

L'objectif de ce chapitre est de présenter l'architecture et le principe de fonctionnement des microcontrôleurs. Parmi les différentes familles des micronctrôleurs existantes, nous choisirons d'étudier en détail la carte *Arduino UNO* en présentant leur architecture, programmation et communication avec son environnement.

I Généralités sur les microcontrôleurs

I.1 Définition

Un microcontrôleur est un circuit intégré qui combine un processeur, de la mémoire et des interfaces d'entrées/sorties pour exécuter des tâches spécifiques et contrôlées dans des systèmes embarqués, avec une faible consommation énergétique, le rendant idéal pour les appareils comme les électroménagers, thermostats et drones.

I.2 Domaines d'applications

les microcontrôleurs sont devenus omniprésents dans notre quotidien, intégrés dans presque tous les appareils électroniques pour gérer des fonctions spécifiques avec une faible consommation d'énergie. Par exemple, on les rencontre dans :

▷ **Systèmes embarqués** : qui sont des systèmes électroniques autonomes intégrés dans des appareils pour exécuter des tâches spécifiques. Par exemple, ces systèmes se trouvent dans :
- Appareils électroménagers : Contrôle des fonctions dans les réfrigérateurs, lave-linge et autres appareils ;
- Automobile : systèmes de sécurité, gestion des moteurs et contrôle des dispositifs information-divertissement ;
- Électronique grand public : Télécommandes, jouets électroniques et dispositifs portables.

▷ **Médecine :** Dans le domaine médical, on trouve les microcontrôleurs dans :
- Dispositifs implantables : Pacemakers et autres dispositifs médicaux nécessitant un contrôle précis ;
- Appareils de diagnostic : Équipements pour analyser les échantillons et surveiller les signes vitaux.

▷ **Industrie** : Les microcontrôleurs sont largement utilisés dans le secteur industriel pour :
- Contrôler et automatiser les processus : Surveillance et régulation des machines et des systèmes de production ;

- Contrôler la qualité des produits : Analyse des données en temps réel pour assurer la conformité des produits.

▷ **Communication** : Grâce aux microcontrôleurs les systèmes de communication deviennent plus performants. Ils sont utilisés dans les :
- Protocoles de communication : Gestion des interfaces (UART, I2C, SPI) et des réseaux sans fil (Wi-Fi, Bluetooth) ;
- Dispositifs de réseau : Routeurs, modems et autres équipements de communication.

▷ **Éducation et recherche** : Dans les environnements éducatifs, les microcontrôleurs sont introduits dans les :
- Projets d'apprentissage : Enseignement de la programmation et de l'électronique à travers des kits de développement ;
- Prototypes de recherche : Développement de nouveaux dispositifs et technologies.

▷ **Internet des objets (IoT)** : Les microcontrôleurs jouent un rôle crucial dans des dispositifs IoT, assurant :
- Connectivité : Utilisation de capteurs et d'actionneurs pour collecter, analyser et transmettre des données ;
- Automatisation domestique : Dispositifs de gestion des maisons intelligentes, comme l'éclairage et la sécurité.

I.3 Description matérielle des microcontrôleurs

I.3.1 Architecture interne des microcontrôleurs

La figure 1 présente l'architecture interne d'un microcontrôleur. Elle montre les différents composants sur lesquels repose le fonctionnement du µc. Chacun de ces composants réalise une tâche spécifique, et ensemble, ils permettent au microcontrôleur :

- de réaliser des tâches variées comme le contrôle des appareils externes ;
- de traiter des données et prendre les décisions ;
- de communiquer avec d'autres systèmes (capteurs, actionneurs, modules de communication sans fil, autres microcontrôleurs,...)

Comme le montre la figure 1, les éléments principaux d'un microcontrôleur sont :

▷ **CPU (microprocesseur)**

Le CPU est l'élément principal du microcontrôleur qui effectue tout le travail de traitement. Il s'agit du cerveau de l'ensemble du circuit. Il fonctionne selon les instructions ou les programmes stockés dans la ROM. Au début du traitement, il récupère les données dans la ROM et les exécute.

▷ **Mémoires**

Les mémoires sont utilisées pour stocker les données et les programmes que le microcontrôleur utilise. On distingue entre la mémoire :

* ROM (Read-Only Memory) : est une mémoire permanente utilisée pour stocker le programme principal du microcontrôleur et les instructions nécessaires à son fonctionnement, incluant le firmware et les routines de démarrage (éléments essentiels dans le fonctionnement des microcontrôleurs).

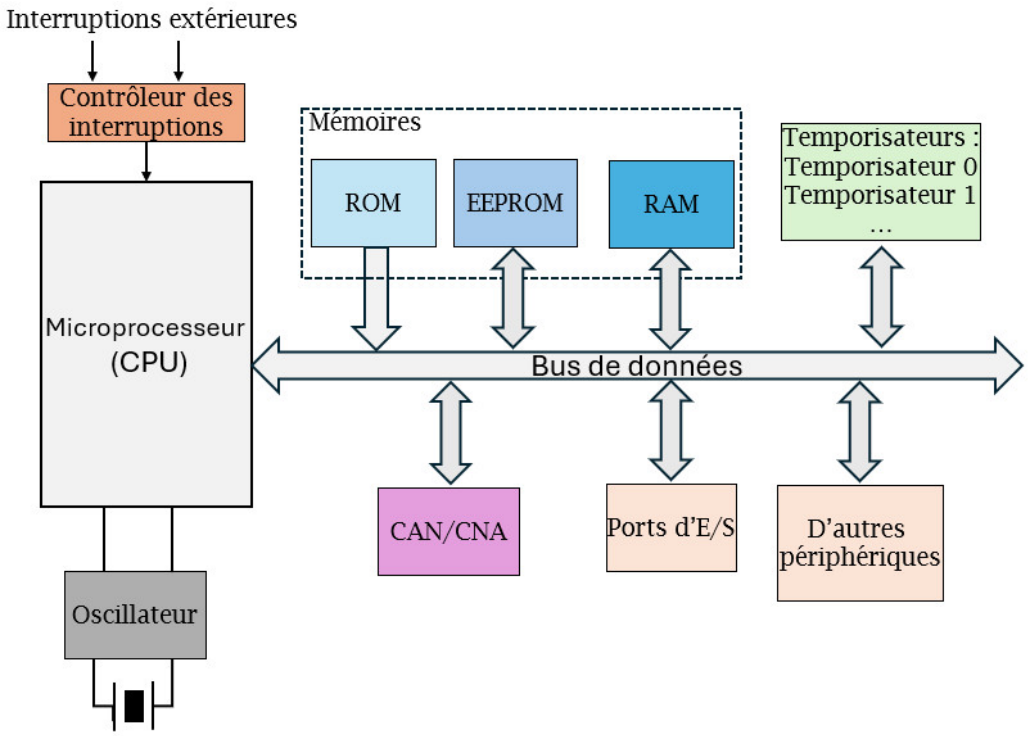


FIGURE 1 – Architecture interne d'un microcontrôleur.

* EEPROM (Electrically Erasable Programmable Read Only Memory) : est une mémoire non-volatile utilisée pour stocker de petites quantités de données qui doivent être conservées hors tension.
* RAM (Random Access Memory) : est une mémoire temporaire utilisée pour stocker temporairement les données pendant l'exécution des programmes.

▷ Temporisateurs (Timers en anglais)/Compteurs(Counters)

* Un temporisateur est un circuit intégré qui compte des impulsions à une fréquence définie, permettant de mesurer des durées des événements, de synchroniser des tâches, de générer des signaux de contrôle (PWM). Par exemple, un temporisateur peut être employé pour faire une action toutes les secondes, comme clignoter une LED toutes les 1 seconde
* Un compteur est utilisé pour compter des événements, par exemple, pour compter le nombre de tours d'un moteur.

▷ Ports d'E/S

Les ports E/S sont des points de connexion entre le microcontrôleur et le milieu extérieur (capteurs, écrans, actionneurs), permettant d'envoyer/recevoir les données, les signaux de contrôle, etc. Ils sont configurés par logiciel pour fonctionner en entrée (lire un signal) ou en sortie (envoyer un signal), généralement via des registres de contrôle (DDRx, PORTx).

▷ Oscillateur

Un oscillateur est un circuit électronique fondamental qui génère le signal d'horloge (un signal périodique) nécessaire pour synchroniser toutes les opérations internes du processeur et des périphériques. Il peut être interne ou externe, et peut être à quartz ou RC, etc.

▷ **Contrôleur des interruptions**

Le contrôleur des interruptions est un circuit intégré qui gère les demandes d'interruption provenant de divers périphériques (capteurs, timers, etc.), les priorise, les masque ou les active, puis informe le processeur (CPU) pour qu'il exécute un sous-programme spécifique.

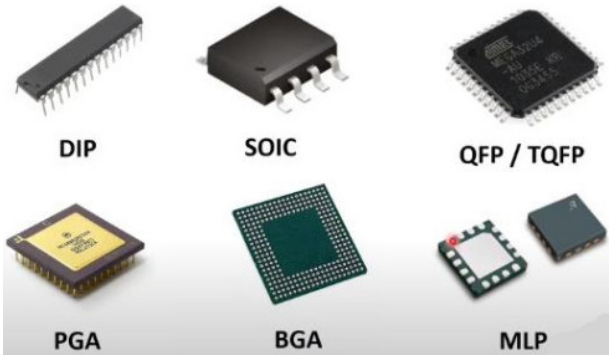
▷ **CAN/CNA**

— CAN : Convertit un signal analogique (comme la température) en un signal numérique (Suite de valeurs numériques).

— CNA : Convertit un signal numérique en un signal analogique.

I.3.2 Boîtiers de microcontrôleurs

Il existe plusieurs types de boîtiers de microcontrôleur, chacun ayant des caractéristiques spécifiques qui les rendent plus ou moins convenables à certaines applications.

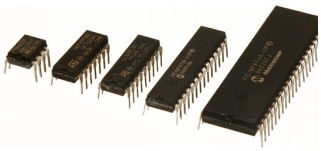


- DIP (Dual Inline Package) :
  + Broches sur les côtés droite et gauche ;
  + Facile à manipuler et à souder.
- PGA (Pin Grid Array) :
  + Broches sous forme de grille ;
  + Un peu plus grand, mais facile à souder.
- SOIC (Small Outline Integrated Circuit) :
  + Plus petit que DIP, broches sur les côtés.
  + Plus difficile à souder que le DIP.
- BGA (Ball Grid Array) :
  + Ne dispose pas de broches visibles, seulement des billes de soudure sous le composant.
  + Très petit et puissant, mais difficile à souder.
- QFP (Quad Flat Package) / TQFP (Thin QFP) :
  + Broches sur les 4 côtés.
  + Plus compact, mais un peu plus compliqué à souder.
- MLP (Micro Lead Frame Package) :
  + Composant plat avec des broches sur les 4 côtés.
  + Convenable pour économiser l'espace.

I.4 Familles de microcontrôleurs

Les microcontrôleurs se groupent en plusieurs familles, chacune ayant des propriétés spécifiques adaptées à différentes applications. Les principales familles de microcontrôleurs sont :

▷ **Famille PIC (Peripheral Interface Controller)** : On distingue quatre catégories des microcontrôleurs PIC :



* *Baseline* : Ont une architecture simple et utilisent un mot d'instruction de 12 bits. Ils sont idéaux pour les applications de base. Exemples de numéros de référence : 10Fxxx, 12Fxxx, 16Fxxx.
* *Mid-Range* : Propose des fonctionnalités avancées avec un mot d'instruction de 14 bits. Exemples de numéros de référence : 10Fxxx, 12Fxxx, 16Fxxx.
* *Enhanced Mid-Range* : Améliore les performances tout en restant compatible avec la famille Mid-Range. Exemples de numéros de référence : 12F1xxx, 16F1xxx.
* *High-End* : Offre des fonctionnalités avancées comme CAN, USB, et Ethernet. Ils Utilisent un mot d'instruction de 16 bits. Exemples de numéros de référence : 18Fxxxx, 18FxxJxx, 18FxxKxx.

▷ **Famille AVR (Advanced Virtual RISC(Reduced Instruction Set Computer))** : Cette famille de microcontrôleurs est conçue par Atmel. Elle est populaire pour sa simplicité et sa flexibilité. Les microcontrôleurs de cette famille sont largement utilisés dans des applications d'éducation et de prototypage, souvent avec des plateformes comme Arduino.



▷ **Famille ARM (Advanced RISC Machine)** Cette famille de microcontrôleurs est basée sur l'architecture ARM. Elle est largement utilisée dans des applications nécessitant des performances élevées et une faible consommation d'énergie, comme des dispositifs mobiles et embarqués.



▷ **Famille MSP430** : Les microcontrôleurs de cette famille sont développés par Texas Instruments. Ils sont connus par leur faible consommation d'énergie, ce qui les rend idéales pour les applications portables et les capteurs.



▷ **Famille 8051** : Cette famille est basée sur l'architecture 8051. Elle est connue par sa simplicité et sa large disponibilité sur le marché. Elle est largement utilisée dans les systèmes embarqués.



Remarques : Chacune de ces familles de microcontrôleurs :
- a ses propres avantages et inconvénients. Ainsi, le choix d'un microcontrôleur dépend souvent des spécifications de l'application visée, telles que la consommation d'énergie, la puissance de traitement, le prix, les interfaces disponibles, etc.
- dispose de son propre *environnement de développement intégré (IDE)* et de ses outils de programmation, PIC : Utilise MPLAB X, Arduino : Utilise Arduino IDE, AVR : Atmel Studio,

ARM utilise IDE comme Keil, IAR Embedded Workbench, MSP430 utilise Code Composer Studio (CCS), 8051 utilise notamment Keil uVision.
- utilise un Programmateur (Programmer) : est un dispositif spécifique pour implanter le code dans la mémoire du microcontroleur.

— PIC utilise PICkit ;

— AVR utilise AVRISP ;

— Arduino Uno utilise un microcontrôleur A mega328P, qui peut être programmé via un programmateur USB-to-serial intégré sur la carte.

— ARM utilise JTAG

- utilise un débogueur : est un outil logiciel qui facilite le développement en testant et corrigeant le code en temps réel.

II Arduino Uno

II.1 Définition

ARDUINO UNO est une carte électronique dont le coeur est un microcontrôleur ATMEL de référence ATMega 328. Ce dernier est un microcontrôleur 8 bits de la famille AVR, dont la programmation peut être réalisée en langage C/C++.

L'avantage principal des cartes Arduino est leur facilité de mise en œuvre. Ils disposent d'un environnement de développement (IDE) qui appuie sur des outils open-source. En outre, le transfert du programme compilé dans la mémoire du microcontrôleur se fait très simplement à l'aide d'un port USB. Enfin, plusieurs bibliothèques de fonctions sont également disponibles pour l'exploitation des E/S courantes : génération de signaux PWM, gestion des convertisseurs ADC, E/S TOR, exploitation de servomoteurs, d'afficheurs LCD, exploitation de bus TWI/I2C...
Remarque : En plus de la carte Arduino UNO, il existe d'autres cartes à savoir Arduino Leonardo, Arduino Mega 2560, Arduino Esplora, Boarduino V2.0, Arduino Nano, Arduino LilyPad, Arduino Due, Arduino Yún, etc. Chacune de ces cartes a ses avantages et inconvénients. Le choix d'une carte dépend des spécifications de l'application, telles que le prix, la consommation énergétique, la surface disponible, etc.

II.2 Présentation de la carte Arduino Uno

La figure 2 présente la carte Arduino Uno.
Les principales caractéristiques de cette carte sont résumées dans le tableau V.1.

II.2.1 Microcontroleur ATmega 328

II.2.1.1 Caractéristiques de ATmega 328 : Les principales caractéristiques du microcontrôleur ATMega 328 de la carte Arduino UNO sont :
+ **Mémoires** :
  - Mémoire programme de 32 Ko (Flash) ;
  - Mémoire volatile de données 2 Ko (SRAM) ;
  - Mémoire non-volatile de données 1 Ko (EEPROM).
+ **Entrées sorties numériques :**

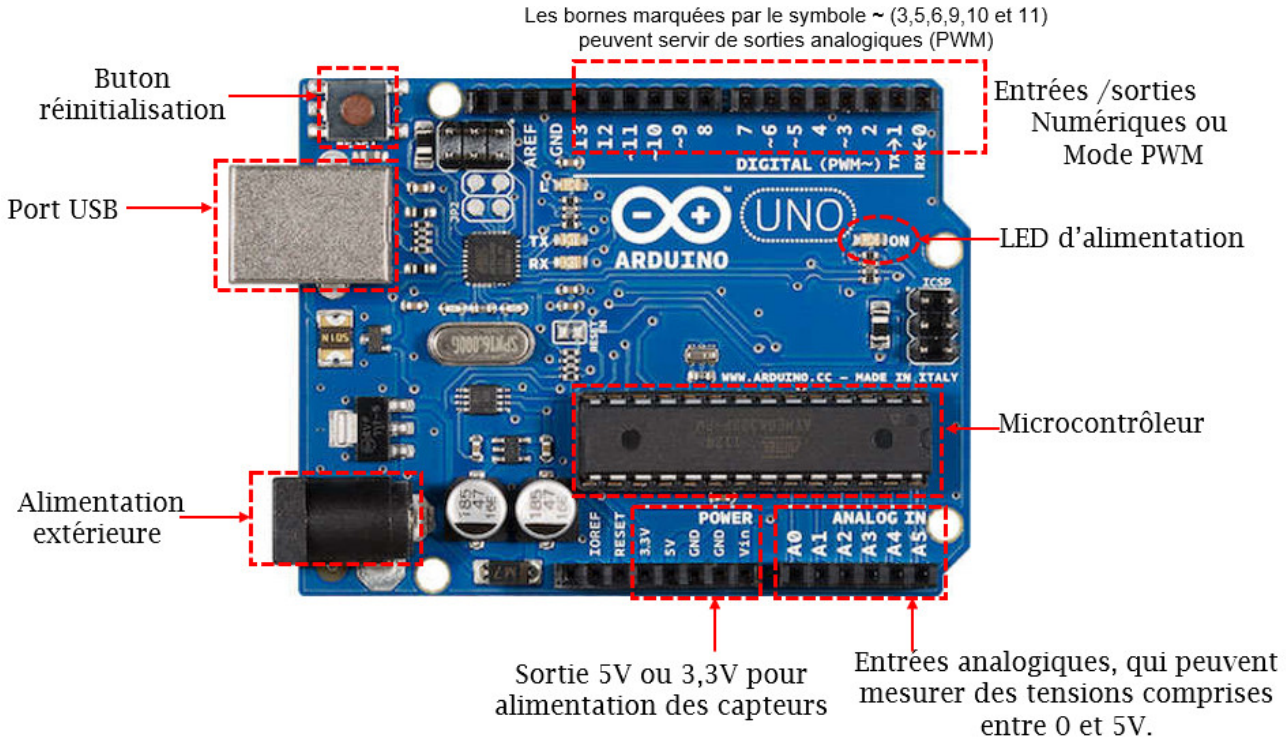


FIGURE 2 – Présentation de la carte Arduino Uno.

Caractéristiques	Valeur/Types
Microcontrôleur	ATmega328
- Fréquence d'horloge	16 MHz
- Tension de sortie	5 V
- Tension d'entrée (recommandée)	7-12 V
- Tension d'entrée (limites)	6-20 V
- Ports numériques	14 Entrées/sorties
- Ports analogiques	6 entrées
- Courant max par broche d'E/S	40 mA
- Courant max par broche 3.3 V	50 mA
- Mémoires	32 Ko Flash, 2 Ko SRAM, 1 Ko EEPROM
- Chargeur d'amorçage	0.5 Ko (en mémoire Flash)
- Interface	USB
- Dimensions	6.86 cm × 5.3 cm
- Prix	250 MAD

TABLE V.1 – Caractéristiques de la carte Arduino Uno.

- 3 ports parallèles : PortB, PortC, PortD (soit 23 broches au total)

+ **Registres :**
- 32 registres de travail d'accès rapide pour l'ALU

+ **Périphériques internes**
- 6 convertisseurs Analogique/Numérique 10 bits alimentés via la broche 20 (AVCC) ;
- 1 comparateur analogique de tension de référence AREF (Broche 21) ;
- 3 temporisateurs : Timer0 et Timer2 (comptage 8 bits), Timer1 (comptage 16 bits) ;
- 6 broches PWM (OCxA/OCxB) ;
- 3 interfaces : SPI, USART, TWI (=I2C).

+ **25 interruptions**
+ **5 modes d'économie d'énergie**

II.2.1.2 Brochage de ATmega 328 : Comme nous pouvons constater sur le schéma de ATmega 328 (Figure 3), la plupart des broches ont différents noms sur le brochage. Elles ont donc plusieurs fonctions différentes, sélectionnées par programmation.

▷ **PortB B0-B7**

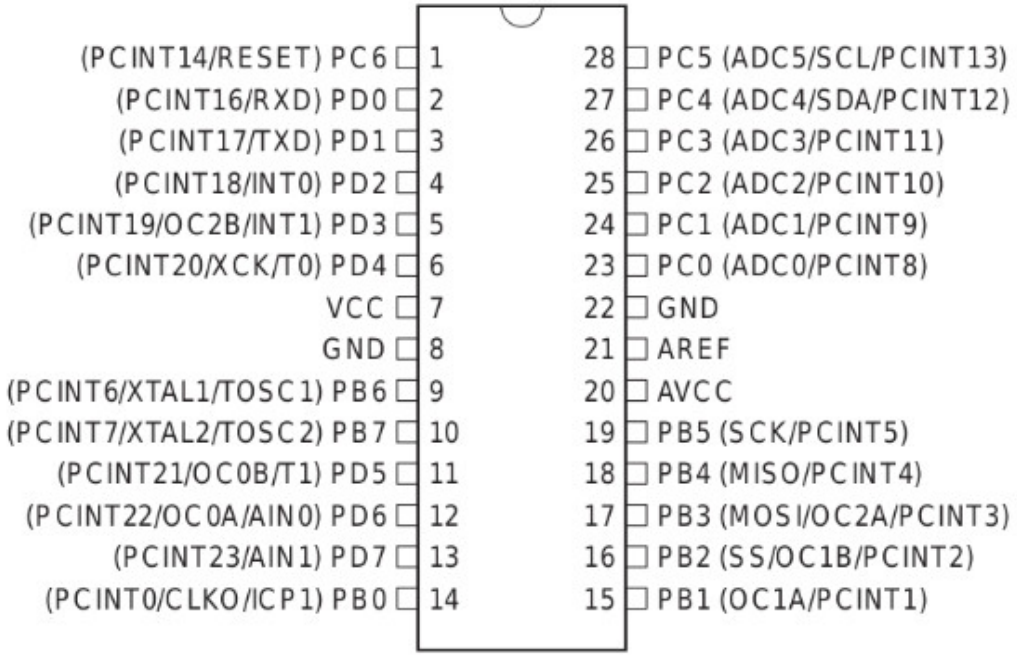


FIGURE 3 – Brochage de microcontrôleur ATmega 328.

• **Fonctions principales des broches PB0-PB7 :**

— *E/S Numériques Générales :* C'est leur fonction de base, permettant de lire des états (HIGH/LOW) ou de piloter des composants (LEDs, relais, etc.).

— *Interruptions (PCINT0-PCINT7)* : En plus, elles peuvent être configurées pour déclencher des routines d'interruption lorsqu'un changement d'état (front montant ou descendant) est détecté, permettant une réactivité rapide aux événements externes.

— *PWM (Pulse Width Modulation)* : Certaines broches (comme PB1, PB2, PB3, PB4) peuvent être utilisées pour générer des signaux PWM, utilisés par exemple pour le contrôle de la vitesse des moteurs.

• **Fonctions Spécifiques :**

— *PB0* : Entrée de capture pour le Timer/Counter1 (ICP1) ou sortie d'horloge système (CLKO).

— *PB1* : Sortie de comparaison *A* pour le Timer/Counter1 (OC1A) permettant la génération de signaux PWM.

— *PB2* : Sortie de comparaison *B* pour le Timer/Counter1 (OC1B) ou entrée de sélection d'esclave pour l'interface SPI (SS).

— *PB3* : Sortie de comparaison *A* pour le Timer/Counter2 (OC2A) ou entrée de données Master Out Slave In pour le bus SPI (MOSI).

— *PB4* : Entrée de données Master In Slave Out pour le bus SPI (MISO).

— *PB5* : Horloge série pour le bus SPI (SCK). Sur une carte Arduino Uno, cette broche pilote souvent la LED intégrée.

— *PB6* : Entrée de l'amplificateur d'oscillateur inverseur (XTAL1) ou entrée d'horloge pour le Timer2 (TOSC1).

— *PB7* : Sortie de l'amplificateur d'oscillateur inverseur (XTAL2) ou sortie d'horloge pour le Timer2 (TOSC2).

▷ **PortC PC0-PC6**

• **Fonctions principales du Port C (PC0-PC6) :**

— *E/S Générales (GPIO)* :

— *Entrées Analogiques (ADC)* Broches : PC0 (ADC0) à PC5 (ADC5) sont directement utilisées comme entrées analogiques pour le convertisseur 10 bits.

— *Interruption de Changement de Broche (PCINT)* : Groupe PCINT8-PCINT15

— *Réinitialisation PC6* (RESET)

• **Fonctions Spécialisées**

— *PC4* (ADC4/SDA) : Ligne de données pour la communication I2C (TWI).

— *PC5* (ADC5/SCL) : Ligne d'horloge pour la communication I2C (TWI).

▷ **PortD : PD0-PD7**

• **Fonctions principales du Port C (PC0-PC7) :**

— *E/S Numérique* : Chaque broche peut être configurée comme une entrée ou une sortie numérique, avec des résistances de pull-up internes activables pour maintenir une logique haute en l'absence de signal externe.

— *Interruption de Changement de Broche* (PCINT) :Groupe PCINT16-PCINT23

• **Fonctions Spécialisées**

— *PD0* (RXD) : Réception de données pour l'interface de communication série UART (Universal Asynchronous Receiver/Transmitter) ;

— *PD1* (TXD) : Transmission de données pour l'UART ;

— *PD2* (INT0) : Entrée pour l'interruption externe 0, réagissant aux changements d'état ;

— *PD3* (INT1 / OC2B) : Entrée pour l'interruption externe 1 ou sortie PWM pour le Timer/Counter 2 (OC2B) ;

— *PD4* (T0 / XCK) : Entrée horloge externe pour le Timer/Counter 0 ou signal d'horloge pour le transfert SPI (XCK) ;

— *PD5* (T1 / OC0B) : Entrée horloge externe pour le Timer/Counter 1 ou sortie PWM pour le Timer/Counter 0 (OC0B) ;

— *PD6* (AIN0 / OC0A) : Entrée pour le comparateur analogique (AIN0) ou sortie PWM pour le Timer/Counter 0 (OC0A) ;

— *PD7* (AIN1) : Entrée pour le comparateur analogique (AIN1).

▷ **Alimentation :**

— *VCC* (Broche 7, 20) : 5V

— *GND* (Broche 8, 22) : Masse (Ground)

— *AREF* (Analog Reference, broche 21) : est une broche qui fournit une tension de référence stable pour le convertisseur analogique-numérique (ADC)

II.2.2 Schéma simplifié de la carte Arduino UNO

Les broches du microcontrôleur ATmega sont reliées avec les connecteurs de la carte Arduino selon le schéma de la figure 4. En recoupant avec le schéma de la carte arduino, on a les

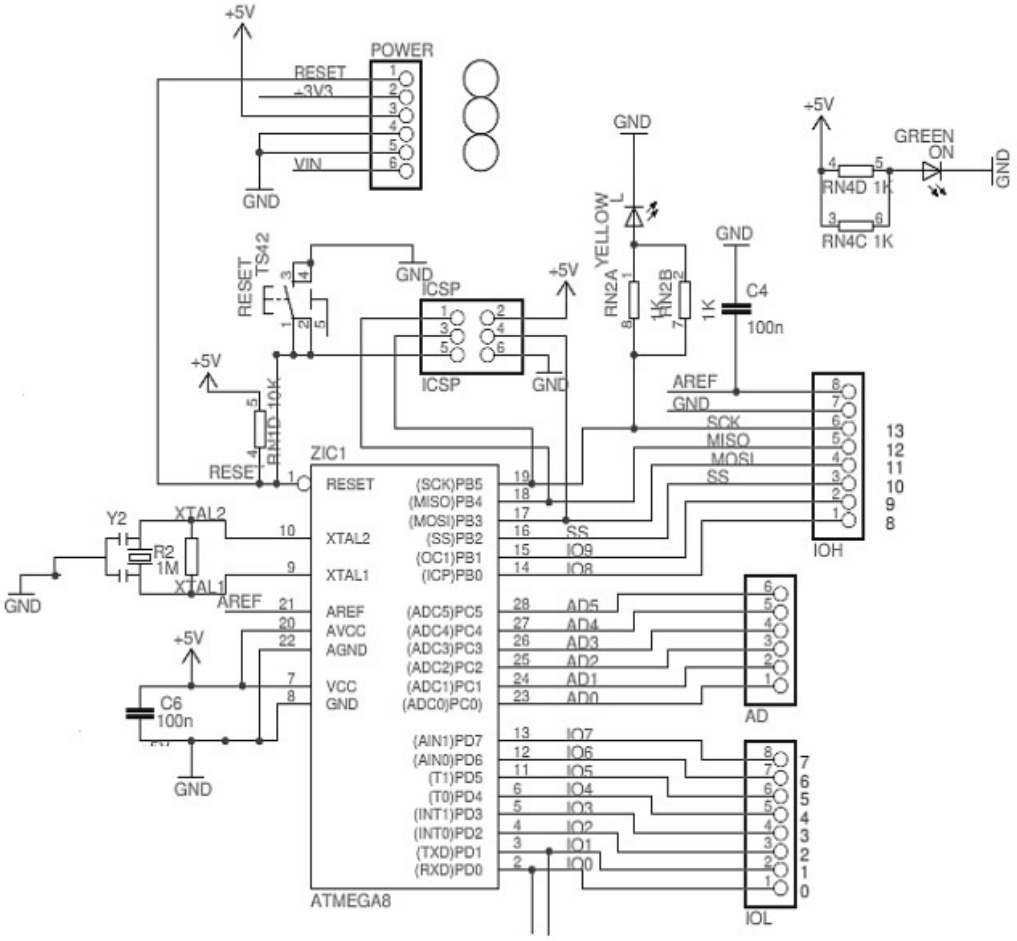


FIGURE 4 – Schéma simplifié de la carte Arduino UNO.

équivalences suivantes :

- Connecteurs numéros 0 à 7 (Arduino)⟶ Broches PD0 à PD7 (microcontrôleur)
- Connecteurs numéros 8 à 13 (Arduino)⟶ Broches PB0 à PB5 (microcontrôleur)
- Connecteurs ANALOG IN A0 à A5 (Arduino)⟶ Broches PC0 à PC5 (microcontrôleur)

Remarque : Lors de la programmation de la carte en utilisant les fonctions Arduino comme pinMode, digitalRead, digitalWrite ..., les signaux sont repérés selon la numérotation

des connecteurs (partie gauche). En revanche, lorsque l'on programme en bas niveau, on utilise le nom des registres/des broches du microcontrôleur (partie droite).

II.3 Programmation de la carte Arduino Uno

II.3.1 Environnement de développement (IDE)

La programmation de la carte Arduino se fait via l'environnement de développement (IDE, Integrated Development Environment en anglais) constituant d'un éditeur de source, des boutons pour les opérations de compilation et de chargement des programmes (sketches) dans la mémoire du microcontrôleur (voir la figure 5). Le fichier de l'installation de l'IDE ainsi que les drivers de la carte Arduino sont téléchargeables librement sur : https ://docs.arduino.cc/software/

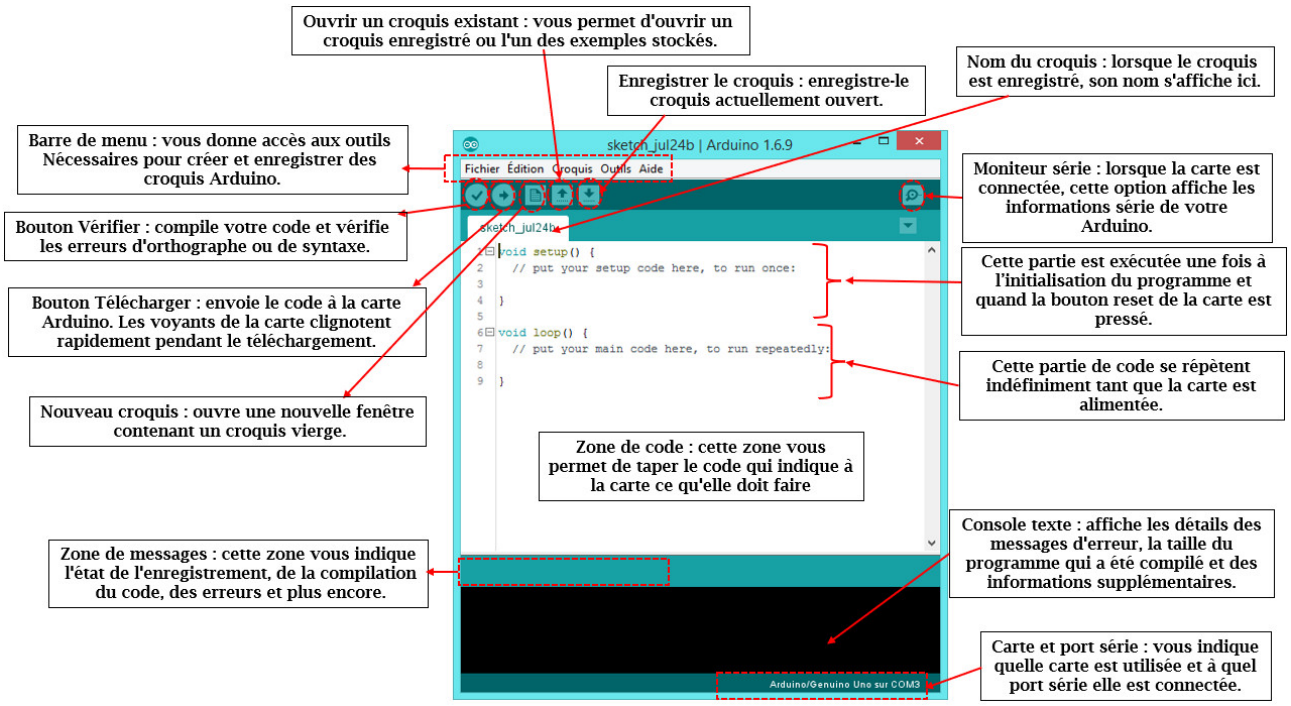


FIGURE 5 – Description de l'IDE de la carte Arduino.

II.3.2 Programmation et configuration de la carte Arduino

La programmation et la configuration de la carte Arduino se font en trois étapes :

Etape 1 : **Préparation et configuration**

- Installer l'IDE Arduino : Téléchargez et installez le logiciel Arduino IDE depuis le site officiel https ://docs.arduino.cc/software/
- Connecter la carte : Branchez l'Arduino Uno avec l'ordinateur via un câble USB (voir la figure 6).
- Configurer l'IDE : Dans le menu Outils > Type de carte, choisissez "Arduino Uno". Ensuite, dans Outils > Port, sélectionnez le port série correspondant à la carte utilisée (voir la figure 6).

Etbpe 2 : **Écriture du code (Sketch)** : Un code Arduino (sketch) contient généralement :

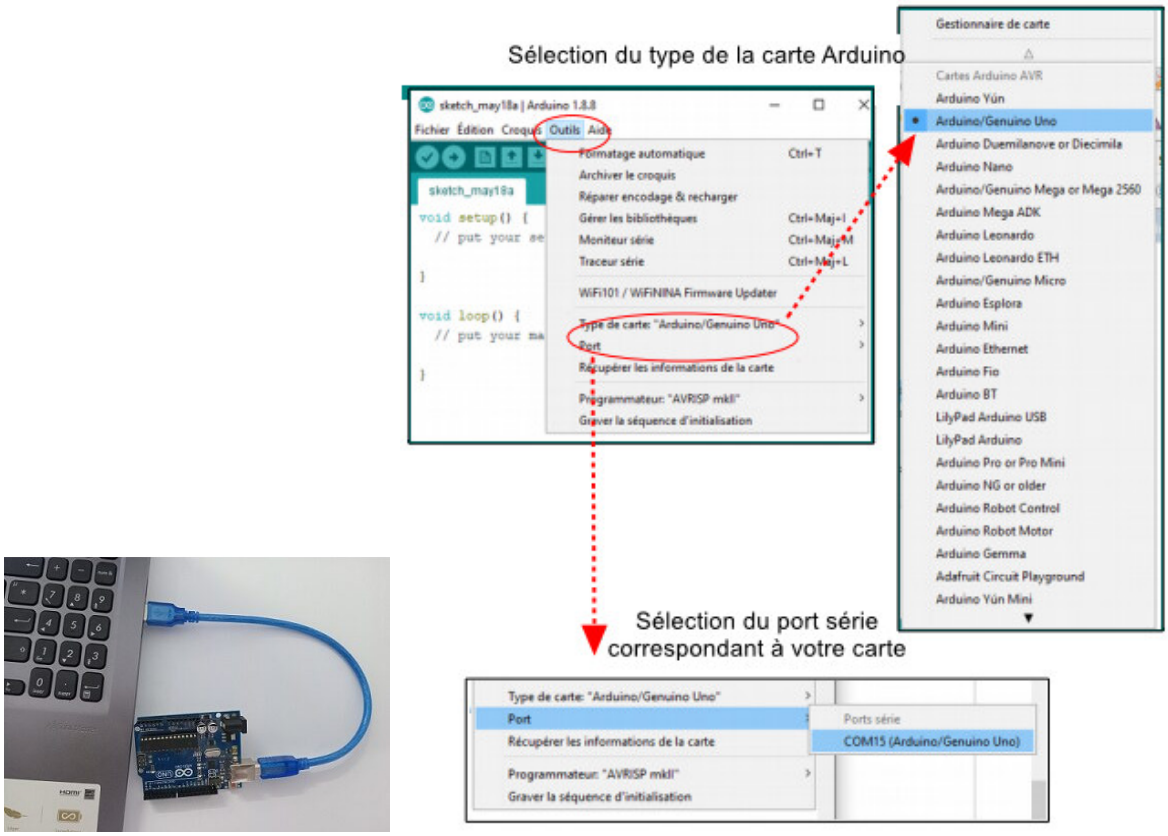


FIGURE 6 – Configuration de l'IDE de l'Arduino.

— Des déclarations de variables (optionnelles).

— La fonction void setup() : s'exécute une seule fois au démarrage pour initialiser les broches et les composants.

— La fonction void loop() : s'exécute indéfiniment tant que la carte Arduino est fonctionnée.

Etcpe 3 : Compilation et téléversement du code

— Vérifier/Compiler : Cliquez sur l'icône de coche √ pour vérifier la syntaxe de code et le compiler.

— Téléverser : Cliquez sur l'icône de flèche droite ⟶ pour envoyer le programme compilé (sketch) vers la carte Arduino.

II.3.3 Langage Arduino

Le langage Arduino est très proche des langages C et C++ auxquels s'ajoutent les fonctions des nombreuses bibliothèques (libraries) Arduino. On distingue entre les fonctions, les variables et les opérateurs :

▷ **Fonctions** : sont utilisées pour structurer le programme. Le tableau V.2 résume les principales fonctions de langages Arduino.

▷ **Variables** (types int, float, boolean, char) : sont utilisées dans un programme pour stocker des données. Le tableau (V.3) regroupe les différents types de variables et types de données.

▷ **Opérateurs** : sont utilisés pour réaliser les calculs et les conditions (if, for, while). Le tableau (V.4) présente les principaux opérateurs de langage Arduino.

Fonction	Description
void setup()	S'exécute une seule fois au démarrage pour initialiser les broches, la communication série, etc..
void loop()	S'exécute en continu après setup(), c'est le cœur du programme
digitalWrite(pin, valeur)	Définit une broche numérique sur HIGH (5V) ou LOW (0V)
pinMode(pin, mode)	Définit une broche comme INPUT ou OUTPUT
analogRead(pin)	Lit une valeur analogique (0-1023) sur une broche analogique (A0-A5).
digitalRead(pin)	Lit l'état (HIGH/LOW) d'une broche numérique
analogWrite(pin, value)	Envoie une valeur PWM (0-255) à une broche PWM (tilde).
Serial.begin(baudrate) / Serial.println()	Pour la communication série avec un ordinateur.
delay(ms)	Met le programme en pause pour un nombre de millisecondes spécifié.

TABLE V.2 – Fonctions principales du langage Arduino.

Variable et types de donnée	Description
int	Entiers (ex : 10, -50)
float	Nombres décimaux (ex : 3.14)
boolean	true ou false (vrai/faux)
char	Caractères (ex : 'A')
byte	Entiers non signés de 0 à 255
unsigned long	Pour les grands nombres, souvent utilisé pour le temps (millis())

TABLE V.3 – Variables et types de données dans le langage Arduino.

Opérateur	Description
Opérateurs arithmétiques	+, -, *, /, % (modulo)
Opérateurs de comparaison	== (égal), != (différent), < (inférieur), > (supérieur), <= , >=
Opérateurs logiques	&& (ET), || (OU), ! (NON)
Conditions	if (condition) ... else ...
Boucles	for (init ; condition ; incrément) ... , while (condition) ...
Constantes	HIGH, LOW, INPUT, OUTPUT, true, false

TABLE V.4 – Opérations et structures de contrôle dans le langage Arduino.

Remarques :

— Pour se familiariser avec le langage Arduino plusieurs exemples pré-enregistrés peuvent être chargés et testés : pour ouvrir un exemple sur l'IDE allez vers : Fichier > Exemples >01.Basics > ...

— Langage Arduino se différencie entre les majuscules et les minuscules

— En langage Arduino, la plupart des lignes se termine par un point virgule ;

— Les blocs, boucles, fonctions sont encadrées par des accolades { .}, apprendre à bien les repérer !

— Toutes les variables doivent être déclarées (nom et type).

— L'indentation (décalage) n'est pas obligatoire mais conseillée pour bien se repérer dans le programme.

— Les commentaires sont définis par // en début de ligne. Les blocs de commentaires par /* au début et */ à la fin

Exemple : Clignoter la LED intégrée (broche 13) sur Arduino

```
void setup() {
  pinMode(13, OUTPUT); // Initialise la broche 13 comme une sortie (pour la LED)
}

void loop() {
  digitalWrite(13, HIGH); // Allume la LED (état HAUT)
  delay(1000);            // Attend 1000 millisecondes (1 seconde)
  digitalWrite(13, LOW);  // Éteint la LED (état BAS)
  delay(1000);            // Attend 1000 millisecondes (1 seconde)
}
```

Bibliographie

[1] M. Sunil, MICROPROCESSOR 8086 Architecture, Programming and Interfacing, PHI LEARNING, 2011

[2] K. J. Ayala, The 8086 Microprocessor : Programming and Interfacing the PC. Delmar Publishers, New York, (1995).

[3] Arduino. (2025). Arduino Programming Basics. https ://www.arduino.cc

[4] J.-M. Hugues, Arduino, le guide complet. Éditions Dunod, 2018.

[5] Aperçu de l'IA. Google. (2025).

[6] HAGGEGE, cours de microprocesseur, ISET Radès, 2003.